CIS Center for Internet Security®

CIS Benchmarks™

CIS Amazon Elastic Kubernetes Service (EKS) Benchmark

v2.0.0 - 05-28-2026

Terms of Use

Please see the below link for our current terms of use:

https://www.cisecurity.org/cis-securesuite/cis-securesuite-membership-terms-of-use/

For information on referencing and/or citing CIS Benchmarks in 3rd party documentation (including using portions of Benchmark Recommendations) please contact CIS Legal (legalnotices@cisecurity.org) and request guidance on copyright usage.

NOTE: It is ***NEVER*** acceptable to host a CIS Benchmark in ***ANY*** format (PDF, etc.) on a 3rd party (non-CIS owned) site.

Table of Contents

Overview

All CIS Benchmarks™ (Benchmarks) focus on technical configuration settings used to maintain and/or increase the security of the addressed technology, and they should be used in **conjunction** with other essential cyber hygiene tasks like:

- Monitoring the base operating system and applications for vulnerabilities and quickly updating with the latest security patches.
- End-point protection (Antivirus software, Endpoint Detection and Response (EDR), etc.).
- Logging and monitoring user and system activity.

In the end, the Benchmarks are designed to be a key **component** of a comprehensive cybersecurity program.

Important Usage Information

All Benchmarks are available free for non-commercial use from the CIS Website. They can be used to manually assess and remediate systems and applications. In lieu of manual assessment and remediation, there are several tools available to assist with assessment:

- CIS Configuration Assessment Tool (CIS-CAT® Pro Assessor)
- CIS Benchmarks™ Certified 3rd Party Tooling

These tools make the hardening process much more scalable for large numbers of systems and applications.

NOTE: Some tooling focuses only on the Benchmark Recommendations that can be fully automated (skipping ones marked **Manual**). It is important that ***ALL*** Recommendations (**Automated** and **Manual**) be addressed since all are important for properly securing systems and are typically in scope for audits.

Key Stakeholders

Cybersecurity is a collaborative effort, and cross functional cooperation is imperative within an organization to discuss, test, and deploy Benchmarks in an effective and efficient way. The Benchmarks are developed to be best practice configuration guidelines applicable to a wide range of use cases. In some organizations, exceptions to specific Recommendations will be needed, and this team should work to prioritize the problematic Recommendations based on several factors like risk, time, cost, and labor. These exceptions should be properly categorized and documented for auditing purposes.

Apply the Correct Version of a Benchmark

Benchmarks are developed and tested for a specific set of products and versions and applying an incorrect Benchmark to a system can cause the resulting pass/fail score to be incorrect. This is due to the assessment of settings that do not apply to the target systems. To assure the correct Benchmark is being assessed:

- **Deploy the Benchmark applicable to the way settings are managed in the environment:** An example of this is the Microsoft Windows family of Benchmarks, which have separate Benchmarks for Group Policy, Intune, and Stand-alone systems based upon how system management is deployed. Applying the wrong Benchmark in this case will give invalid results.

- **Use the most recent version of a Benchmark**: This is true for all Benchmarks, but especially true for cloud technologies. Cloud technologies change frequently and using an older version of a Benchmark may have invalid methods for auditing and remediation.

Exceptions

The guidance items in the Benchmarks are called recommendations and not requirements, and exceptions to some of them are expected and acceptable. The Benchmarks strive to be a secure baseline, or starting point, for a specific technology, with known issues identified during Benchmark development are documented in the Impact section of each Recommendation. In addition, organizational, system specific requirements, or local site policy may require changes as well, or an exception to a Recommendation or group of Recommendations (e.g. A Benchmark could Recommend that a Web server not be installed on the system, but if a system's primary purpose is to function as a Webserver, there should be a documented exception to this Recommendation for that specific server).

In the end, exceptions to some Benchmark Recommendations are common and acceptable, and should be handled as follows:

- The reasons for the exception should be reviewed cross-functionally and be well documented for audit purposes.
- A plan should be developed for mitigating, or eliminating, the exception in the future, if applicable.
- If the organization decides to accept the risk of this exception (not work toward mitigation or elimination), this should be documented for audit purposes.

It is the responsibility of the organization to determine their overall security policy, and which settings are applicable to their unique needs based on the overall risk profile for the organization.

Remediation

CIS has developed Build Kits for many technologies to assist in the automation of hardening systems. Build Kits are designed to correspond to Benchmark's "Remediation" section, which provides the manual remediation steps necessary to make that Recommendation compliant to the Benchmark.

When remediating systems (changing configuration settings on deployed systems as per the Benchmark's Recommendations), please approach this with caution and test thoroughly.

The following is a reasonable remediation approach to follow:

- CIS Build Kits, or internally developed remediation methods should never be applied to production systems without proper testing.
- Proper testing consists of the following:
  - Understand the configuration (including installed applications) of the targeted systems. Various parts of the organization may need different configurations (e.g., software developers vs standard office workers).
  - Read the Impact section of the given Recommendation to help determine if there might be an issue with the targeted systems.
  - Test the configuration changes with representative lab system(s). If issues arise during testing, they can be resolved prior to deploying to any production systems.
  - When testing is complete, initially deploy to a small sub-set of production systems and monitor closely for issues. If there are issues, they can be resolved prior to deploying more broadly.
  - When the initial deployment above is completed successfully, iteratively deploy to additional systems and monitor closely for issues. Repeat this process until the full deployment is complete.

Summary

Using the Benchmarks Certified tools, working as a team with key stakeholders, being selective with exceptions, and being careful with remediation deployment, it is possible to harden large numbers of deployed systems in a cost effective, efficient, and safe manner.

NOTE: As previously stated, the PDF versions of the CIS Benchmarks™ are available for free, non-commercial use on the CIS Website. All other formats of the CIS Benchmarks™ (MS Word, Excel, and Build Kits) are available for CIS SecureSuite® members.

CIS-CAT® Pro is also available to CIS SecureSuite® members.

Target Technology Details

This document provides prescriptive guidance for running Amazon Elastic Kubernetes Service (EKS) following recommended security controls. This benchmark only includes controls which can be modified by an end user of Amazon EKS.

To obtain the latest version of this guide, please visit www.cisecurity.org. If you have questions, comments, or have identified ways to improve this guide, please write us at support@cisecurity.org.

Intended Audience

This document is intended for cluster administrators, security specialists, auditors, and any personnel who plan to develop, deploy, assess, or secure solutions that incorporate Amazon EKS using managed and self-managed nodes.

Customers using Amazon EKS on AWS Fargate are not responsible for node management. Hence, this document is not scoped for Amazon EKS on AWS Fargate customers.

Consensus Guidance

This CIS Benchmark™ was created using a consensus review process comprised of a global community of subject matter experts. The process combines real world experience with data-based information to create technology specific guidance to assist users to secure their environments. Consensus participants provide perspective from a diverse set of backgrounds including consulting, software development, audit and compliance, security research, operations, government, and legal.

Each CIS Benchmark undergoes two phases of consensus review. The first phase occurs during initial Benchmark development. During this phase, subject matter experts convene to discuss, create, and test working drafts of the Benchmark. This discussion occurs until consensus has been reached on Benchmark recommendations. The second phase begins after the Benchmark has been published. During this phase, all feedback provided by the Internet community is reviewed by the consensus team for incorporation in the Benchmark. If you are interested in participating in the consensus process, please visit https://workbench.cisecurity.org/.

Typographical Conventions

The following typographical conventions are used throughout this guide:

Convention	Meaning
`Stylized Monospace font`	Used for blocks of code, command, and script examples. Text should be interpreted exactly as presented.
`Monospace font`	Used for inline code, commands, UI/Menu selections or examples. Text should be interpreted exactly as presented.
`<Monospace font in brackets>`	Text set in angle brackets denote a variable requiring substitution for a real value.
Italic font	Used to reference other relevant settings, CIS Benchmarks and/or Benchmark Communities. Also, used to denote the title of a book, article, or other publication.
Bold font	Additional information or caveats things like **Notes**, **Warnings**, or **Cautions** (usually just the word itself and the rest of the text normal).

Recommendation Definitions

The following defines the various components included in a CIS recommendation as applicable. If any of the components are not applicable it will be noted, or the component will not be included in the recommendation.

Title

Concise description for the recommendation's intended configuration.

Assessment Status

An assessment status is included for every recommendation. The assessment status indicates whether the given recommendation can be automated or requires manual steps to implement. Both statuses are equally important and are determined and supported as defined below:

Automated

Represents recommendations for which assessment of a technical control can be fully automated and validated to a pass/fail state. Recommendations will include the necessary information to implement automation.

Manual

Represents recommendations for which assessment of a technical control cannot be fully automated and requires all or some manual steps to validate that the configured state is set as expected. The expected state can vary depending on the environment.

Profile

A collection of recommendations for securing a technology or a supporting platform. Most benchmarks include at least a Level 1 and Level 2 Profile. Level 2 extends Level 1 recommendations and is not a standalone profile. The Profile Definitions section in the benchmark provides the definitions as they pertain to the recommendations included for the technology.

Description

Detailed information pertaining to the setting with which the recommendation is concerned. In some cases, the description will include the recommended value.

Rationale Statement

Detailed reasoning for the recommendation to provide the user a clear and concise understanding on the importance of the recommendation.

Impact Statement

Any security, functionality, or operational consequences that can result from following the recommendation.

Audit Procedure

Systematic instructions for determining if the target system complies with the recommendation.

Remediation Procedure

Systematic instructions for applying recommendations to the target system to bring it into compliance according to the recommendation.

Default Value

Default value for the given setting in this recommendation, if known. If not known, either not configured or not defined will be applied.

References

Additional documentation relative to the recommendation.

CIS Critical Security Controls® (CIS Controls®)

The mapping between a recommendation and the CIS Controls is organized by CIS Controls version, Safeguard, and Implementation Group (IG). The Benchmark in its entirety addresses the CIS Controls safeguards of (v7) “5.1 - Establish Secure Configurations” and (v8) '4.1 - Establish and Maintain a Secure Configuration Process” so individual recommendations will not be mapped to these safeguards.

Additional Information

Supplementary information that does not correspond to any other field but may be useful to the user.

Profile Definitions

The following configuration profiles are defined by this Benchmark:

- **Level 1**

  Level 1 Configuration Profile

- **Level 2**
  Extends Level 1

Acknowledgements

This Benchmark exemplifies the great things a community of users, vendors, and subject matter experts can accomplish through consensus collaboration. The CIS community thanks the entire consensus team with special recognition to the following individuals who contributed greatly to the creation of this guide:

Thanks to Authors: Paavan Mistry and Randall Mowen
with special thanks to contributors: Rory MCcune and Mark Larinde

Authors:

Paavan Mistry
Randall Mowen

Editor:
Mark Larinde

Contributors
Randall Mowen
Rory MCcune
Tony Wilwerding
James Stocks
Daniel Burns
Joe Bowbeer

Recommendations

1 Control Plane Components

Security in Amazon EKS follows the AWS shared responsibility model. AWS is responsible for the security and operation of the managed Kubernetes control plane, including the control plane nodes and the etcd datastore that support the cluster. In standard EKS, each cluster has its own dedicated control plane, and AWS distributes control plane components across multiple Availability Zones for availability and durability.

Although AWS secures and operates the Amazon EKS control plane, the customer is responsible for securing and managing the cluster environment within the customer AWS account. This includes configuring the data plane, controlling access, securing workloads and applications, managing network access to the Kubernetes API, and protecting the data processed by the cluster. As a result, the security of an EKS deployment depends on both AWS's protection of the managed control plane and the customer's configuration of the surrounding cluster resources.

The customer remains responsible for security in the cloud, including:

- Configuration and protection of the cluster data plane
- Security of worker nodes, containers, and workloads
- Workload permissions and network controls
- IAM and Kubernetes access management
- Protection of applications and data
- Configuration of Kubernetes API access, including public or private endpoint exposure and allowed CIDR ranges where applicable

For Amazon EKS managed node groups, AWS is responsible for producing updated EKS-optimized AMIs that include applicable security patches and Kubernetes updates, while the customer is responsible for deploying those updated AMI versions to the node groups. AWS secures the control plane itself, while customers are responsible for configuring the control-plane-related options that are exposed to them and addressed elsewhere in this benchmark.

2 Control Plane Configuration

In Amazon EKS, control plane configuration refers to the customer-manageable settings that govern how the managed Kubernetes API is exposed, monitored, and protected. While AWS operates the underlying control plane in an AWS-managed environment, customers are responsible for configuring the options that Amazon EKS makes available for cluster administration, such as API endpoint access, authentication and access management, control plane logging, and encryption settings for Kubernetes API data stored in etcd.

Although AWS secures and operates the Amazon EKS control plane infrastructure, the customer is responsible for selecting and managing the control-plane-related settings exposed through the service. This includes determining how the Kubernetes API can be reached, how administrative access is granted, whether control plane activity is logged to the customer account, and how Kubernetes API data is protected through available encryption options. As a result, the security posture of the EKS control plane depends on both AWS's management of the service and the customer's configuration of the available control plane options.

The customer remains responsible for control plane configuration, including:

- Kubernetes API endpoint exposure, including public access, private access, and public access CIDR restrictions
- Cluster access configuration, including authentication mode, access entries, access policies, and IAM-to-Kubernetes access management
- Control plane logging configuration, including whether audit and diagnostic logs are sent to CloudWatch Logs
- Encryption configuration for Kubernetes API data, including default envelope encryption and customer-managed AWS KMS keys where applicable

Amazon EKS secures the managed control plane itself, but customers are responsible for choosing and maintaining the control-plane settings that Amazon EKS exposes for customer administration and that are addressed elsewhere in this benchmark. AWS documentation also identifies EKS access entries and access policies as the current access-management approach, while the legacy aws-auth ConfigMap is deprecated.

2.1 Logging

Logging in AKS is primarily handled through Azure Monitor. In practice, AKS logging combines Azure Activity Logs for Azure resource operations, control plane resource logs for managed Kubernetes components, audit logs for Kubernetes API server activity, and container or cluster logs collected from nodes and workloads. These logs are commonly routed to a Log Analytics workspace for querying, correlation, and alerting.

Key concepts:

- Control plane logs are implemented as Azure Monitor resource logs for AKS-managed components.
- Audit logs are a subset of control plane logging and capture Kubernetes API server activity.
- The main audit categories are `kube-audit` and `kube-audit-admin`.
- `kube-audit` includes all API audit events, while `kube-audit-admin` excludes `get` and `list` operations and is more focused on administrative or modifying activity.
- Control plane and audit logs are not collected by default and must be enabled through Diagnostic settings.
- Container insights is commonly used to collect workload and cluster logs from nodes and containers.
- In Log Analytics, audit data can be stored in tables such as AKSAudit and AKSAuditAdmin.

2.1.1 Enable EKS control plane logging (Automated)

Profile Applicability:

- Level 1

Description:

Enable control plane logging in Amazon EKS to provide visibility into the operation of the managed Kubernetes control plane and activity processed through its core components. Control plane logs create a persistent record of administrative activity, authentication events, scheduling behavior, and other platform operations to support monitoring, troubleshooting, investigation, and accountability. In EKS, control plane log types are enabled on a per-cluster basis and exported to Amazon CloudWatch Logs for retention, search, and analysis.

Rationale:

In Amazon EKS, control plane logging must be explicitly enabled for each cluster and for each desired log type. EKS provides separate control plane log types for the Kubernetes API server `api`, audit activity `audit`, EKS authenticator events `authenticator`, controller manager operations `controllerManager`, and scheduler activity `scheduler`, allowing organizations to capture visibility across the managed control plane rather than relying on a single source of diagnostic data. When enabled, these logs are exported to Amazon CloudWatch Logs, where they support investigation of authentication events, administrative actions, control plane errors, scheduling behavior, and other operational conditions that may indicate misuse, misconfiguration, or service issues. Collecting these log streams improves the ability to monitor cluster activity, investigate abnormal behavior, validate administrative actions, and perform more targeted remediation using a centralized and retained record of control plane events.

Impact:

Without EKS control plane logging, visibility into API activity, authentication events, scheduler actions, and other managed control plane operations is reduced, which can hinder troubleshooting, delay incident investigation, and weaken accountability for administrative activity. Enabling these logs improves detection and response capabilities, but it also increases Amazon CloudWatch Logs ingestion and storage costs.

Audit:

From Console:

1. For each EKS Cluster in each region;
2. Go to **Amazon Elastic Kubernetes Service** > **Clusters** > **CLUSTER_NAME** > **Observability** > **Control plane logs**

3. This will show the control plane logging configuration:
    - API server (Logs pertaining to API requests to the cluster)
    - Audit (Logs pertaining to cluster access via the Kubernetes API.)
    - Authenticator (Logs pertaining to authentication requests into the cluster.)
    - Controller Manager (Logs pertaining to state of cluster controllers.)
    - Scheduler (Logs pertaining to scheduling decisions.)
4. Ensure that all options are set to 'Enabled' for logging you want to capture

From AWS CLI:

1. For each EKS Cluster in each region:

```
aws eks describe-cluster \
  --name $CLUSTER_NAME \
  --region $REGION_CODE \
  --query 'cluster.logging.clusterLogging'
```

Remediation:

From Console:

1. For each EKS Cluster in each region;
2. Go to **Amazon Elastic Kubernetes Service** > **Clusters** > **CLUSTER_NAME** > **Observability** > **Control plane logs**
3. Click **Manage** to enable or disable logging.
4. Ensure that one or more options are toggled to `Enabled` and Save Changes.
    - API server: Enabled
    - Audit: Enabled
    - Authenticator: Enabled
    - Controller manager: Enabled
    - Scheduler: Enabled
5. Click 'Save Changes'.

From the AWS CLI:

1. For each EKS Cluster in each region:

```
aws eks update-cluster-config \
  --name $CLUSTER_NAME \
  --region $REGION_CODE \
  --logging
'{"clusterLogging":[{"types":["api","audit","authenticator","controllerManage
r","scheduler"],"enabled":true}]}'
```

Default Value:

Control Plane Logging is disabled by default.

```
API server: Disabled
Audit: Disabled
Authenticator: Disabled
Controller manager: Disabled
Scheduler: Disabled
```

References:

1. https://docs.aws.amazon.com/eks/latest/userguide/control-plane-logs.html
2. https://docs.aws.amazon.com/cli/latest/reference/eks/update-cluster-config.html
3. https://docs.aws.amazon.com/eks/latest/APIReference/API_DescribeCluster.html
4. https://docs.aws.amazon.com/eks/latest/best-practices/auditing-and-logging.html

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	8.1 Establish and Maintain an Audit Log Management Process Establish and maintain an audit log management process that defines the enterprise's logging requirements. At a minimum, address the collection, review, and retention of audit logs for enterprise assets. Review and update documentation annually, or when significant enterprise changes occur that could impact this Safeguard.	●	●	●
v8	8.2 Collect Audit Logs Collect audit logs. Ensure that logging, per the enterprise's audit log management process, has been enabled across enterprise assets.	●	●	●
v7	6.2 Activate audit logging Ensure that local logging has been enabled on all systems and networking devices.	●	●	●
v7	6.3 Enable Detailed Logging Enable system logging to include detailed information such as an event source, date, user, timestamp, source addresses, destination addresses, and other useful elements.		●	●

3 Worker Nodes

This section consists of security recommendations for the components that run on Amazon EKS worker nodes.

3.1 Worker Node Configuration Files

In AWS EKS, worker node configuration files are the operating system and bootstrap settings that prepare each data-plane node to join the cluster and run core services such as `kubelet` and `containerd`. These files control how the node is initialized, how it registers with the cluster, and how baseline runtime and networking behavior are applied. In practice, EKS worker node configuration is typically provided through node bootstrap configuration, user data, and host-level service settings rather than through a single centralized file.

3.1.1 Ensure that the kubeconfig file permissions are set to 644 or more restrictive (Automated)

Profile Applicability:

- Level 1

Description:

If kubelet is running, and if it is configured by a kubeconfig file, ensure that the proxy kubeconfig file has permissions of 644 or more restrictive.

Rationale:

The `kubelet` kubeconfig file controls various parameters of the `kubelet` service in the worker node. You should restrict its file permissions to maintain the integrity of the file. The file should be writable by only the administrators on the system.

It is possible to run `kubelet` with the kubeconfig parameters configured as a Kubernetes ConfigMap instead of a file. In this case, there is no proxy kubeconfig file.

Impact:

Ensuring that the kubeconfig file permissions are set to 644 or more restrictive significantly strengthens the security posture of the Kubernetes environment by preventing unauthorized modifications. This restricts write access to the kubeconfig file, ensuring only administrators can alter crucial kubelet configurations, thereby reducing the risk of malicious alterations that could compromise the cluster's integrity.

However, this configuration may slightly impact usability, as it limits the ability for non-administrative users to make quick adjustments to the kubelet settings. Administrators will need to balance security needs with operational flexibility, potentially requiring adjustments to workflows for managing kubelet configurations.

Audit:

Method 1

Optional: Run the following command to enter **debug mode** where NODE_NAME and IMAGE are defined variables:

```
kubectl debug node/$NODE_NAME -it --image=$IMAGE --profile=general
```

Run the following command on each node to find the appropriate kubeconfig file:

```
ps -efww | grep kubelet | grep -- '--kubeconfig'
```

The command line output should show a path for kubeconfig similar to `--kubeconfig=/var/lib/kubelet/kubeconfig` which is the location of the kubeconfig file.

Run this command to obtain the kubeconfig file permissions:

```
stat -c %a /var/lib/kubelet/kubeconfig (or stat -c %a
/host/var/lib/kubelet/kubeconfig if in debub mode)
```

The output of the above command gives you the kubeconfig file's permissions.

Verify that if a file is specified and it exists, the permissions are 644 or more restrictive.

Method 2

Create and Run a Privileged Pod.

You will need to run a pod that is privileged enough to access the host's file system. This can be achieved by deploying a pod that uses the hostPath volume to mount the node's file system into the pod.

Here's an example of a simple pod definition that mounts the root of the host to /host within the pod:

```
apiVersion: v1
kind: Pod
metadata:
  name: file-check
spec:
  volumes:
  - name: host-root
    hostPath:
      path: /
      type: Directory
  containers:
  - name: nsenter
    image: busybox
    command: ["sleep", "3600"]
    volumeMounts:
    - name: host-root
      mountPath: /host
    securityContext:
      privileged: true
```

Save this to a file (e.g., file-check-pod.yaml) and create the pod:

```
kubectl apply -f file-check-pod.yaml
```

Once the pod is running, you can exec into it to check file permissions on the node:

```
kubectl exec -it -n kube-public file-check -- sh
```

Now you are in a shell inside the pod, but you can access the node's file system through the /host directory and check the permission level of the file:

```
ls -l /host/var/lib/kubelet/kubeconfig
```

Verify that if a file is specified and it exists, the permissions are 644 or more restrictive.

Remediation:

Run the below command (based on the file location on your system) on the each worker node. For example,

```
chmod 644 <kubeconfig file>
```

Default Value:

See the AWS EKS documentation for the default value.

References:

1. https://kubernetes.io/docs/reference/command-line-tools-reference/kube-proxy/

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	3.3 Configure Data Access Control Lists Configure data access control lists based on a user's need to know. Apply data access control lists, also known as access permissions, to local and remote file systems, databases, and applications.	●	●	●
v7	5.2 Maintain Secure Images Maintain secure images or templates for all systems in the enterprise based on the organization's approved configuration standards. Any new system deployment or existing system that becomes compromised should be imaged using one of those images or templates.		●	●

3.1.2 Ensure that the kubeconfig file ownership is set to root:root (Automated)

Profile Applicability:

- Level 1

Description:

If `kubelet` is running, ensure that the file ownership of its kubeconfig file is set to `root:root`.

Rationale:

The kubeconfig file for `kubelet` controls various parameters for the `kubelet` service in the worker node. You should set its file ownership to maintain the integrity of the file. The file should be owned by `root:root`.

Impact:

None

Audit:

Method 1

Optional: Run the following command to enter **debug mode** where NODE_NAME and IMAGE are defined variables:

```
kubectl debug node/$NODE_NAME -it --image=$IMAGE --profile=general
```

Run the following command on each node to find the appropriate kubeconfig file:

```
ps -efww | grep kubelet | grep -- '--kubeconfig'
```

The command line output should show a path for kubeconfig similar to `--kubeconfig=/var/lib/kubelet/kubeconfig` which is the location of the kubeconfig file.

Run this command to obtain the kubeconfig file permissions:

```
stat -c %U:%G /var/lib/kubelet/kubeconfig (or stat -c %U:%G
/host/var/lib/kubelet/kubeconfig if in debub mode)
```

The output of the above command gives you the kubeconfig file's ownership. Verify that the ownership is set to `root:root`.

Method 2

Create and Run a Privileged Pod.

You will need to run a pod that is privileged enough to access the host's file system. This can be achieved by deploying a pod that uses the hostPath volume to mount the node's file system into the pod.

Here's an example of a simple pod definition that mounts the root of the host to /host within the pod:

```
apiVersion: v1
kind: Pod
metadata:
  name: file-check
spec:
  volumes:
  - name: host-root
    hostPath:
      path: /
      type: Directory
  containers:
  - name: nsenter
    image: busybox
    command: ["sleep", "3600"]
    volumeMounts:
    - name: host-root
      mountPath: /host
    securityContext:
      privileged: true
```

Save this to a file (e.g., file-check-pod.yaml) and create the pod:

```
kubectl apply -f file-check-pod.yaml
```

Once the pod is running, you can exec into it to check file ownership on the node:

```
kubectl exec -it -n kube-public file-check -- sh
```

Now you are in a shell inside the pod, but you can access the node's file system through the /host directory and check the ownership of the file:

```
ls -l /host/var/lib/kubelet/kubeconfig
```

The output of the above command gives you the kubeconfig file's ownership. Verify that the ownership is set to `root:root`.

Remediation:

Run the below command (based on the file location on your system) on each worker node.

For example,

```
chown root:root <proxy kubeconfig file>
```

Default Value:

See the AWS EKS documentation for the default value.

References:

1. https://kubernetes.io/docs/reference/command-line-tools-reference/kube-proxy/

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	3.3 Configure Data Access Control Lists Configure data access control lists based on a user's need to know. Apply data access control lists, also known as access permissions, to local and remote file systems, databases, and applications.	●	●	●
v7	5.2 Maintain Secure Images Maintain secure images or templates for all systems in the enterprise based on the organization's approved configuration standards. Any new system deployment or existing system that becomes compromised should be imaged using one of those images or templates.		●	●

3.1.3 Ensure that the kubelet configuration file has permissions set to 644 or more restrictive (Automated)

Profile Applicability:

- Level 1

Description:

Ensure that if the kubelet refers to a configuration file with the `--config` argument, that file has permissions of 644 or more restrictive.

Rationale:

The kubelet reads various parameters, including security settings, from a config file specified by the `--config` argument. If this file is specified you should restrict its file permissions to maintain the integrity of the file. The file should be writable by only the administrators on the system.

Impact:

None.

Audit:

Method 1

Optional: Run the following command to enter **debug mode** where NODE_NAME and IMAGE are defined variables:

```
kubectl debug node/$NODE_NAME -it --image=$IMAGE --profile=general
```

Run the following command on each node to find the appropriate config file:

```
ps -efww | grep kubelet | grep -- '--config'
```

The command line output should show a path for config similar to `--config /etc/kubernetes/kubelet/config.json` which is the location of the config file.

Run this command to obtain the config file permissions:

```
stat -c %a /etc/kubernetes/kubelet/config.json (or stat -c %a
/host/etc/kubernetes/kubelet/config.json if in debub mode)
```

The output of the above command is the Kubelet config file's permissions. Verify that the permissions are `644` or more restrictive.

Method 2

Create and Run a Privileged Pod.

You will need to run a pod that is privileged enough to access the host's file system. This can be achieved by deploying a pod that uses the hostPath volume to mount the node's file system into the pod.

Here's an example of a simple pod definition that mounts the root of the host to /host within the pod:

```
apiVersion: v1
kind: Pod
metadata:
  name: file-check
spec:
  volumes:
  - name: host-root
    hostPath:
      path: /
      type: Directory
  containers:
  - name: nsenter
    image: busybox
    command: ["sleep", "3600"]
    volumeMounts:
    - name: host-root
      mountPath: /host
    securityContext:
      privileged: true
```

Save this to a file (e.g., file-check-pod.yaml) and create the pod:

```
kubectl apply -f file-check-pod.yaml
```

Once the pod is running, you can exec into it to check file permissions on the node:

```
kubectl exec -it -n kube-public file-check -- sh
```

Now you are in a shell inside the pod, but you can access the node's file system through the /host directory and check the permission level of the file:

```
ls -l /host/etc/kubernetes/kubelet/config.json
```

Verify that if a file is specified and it exists, the permissions are 644 or more restrictive.

Remediation:

Run the following command (using the config file location identified in the Audit step)

```
chmod 644 /etc/kubernetes/kubelet/config.json
```

Default Value:

See the AWS EKS documentation for the default value.

References:

1. https://kubernetes.io/docs/tasks/administer-cluster/kubelet-config-file/

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	3.3 Configure Data Access Control Lists Configure data access control lists based on a user's need to know. Apply data access control lists, also known as access permissions, to local and remote file systems, databases, and applications.	●	●	●
v7	5.2 Maintain Secure Images Maintain secure images or templates for all systems in the enterprise based on the organization's approved configuration standards. Any new system deployment or existing system that becomes compromised should be imaged using one of those images or templates.		●	●

3.1.4 Ensure that the kubelet configuration file ownership is set to root:root (Automated)

Profile Applicability:

- Level 1

Description:

Ensure that if the kubelet refers to a configuration file with the `--config` argument, that file is owned by root:root.

Rationale:

The kubelet reads various parameters, including security settings, from a config file specified by the `--config` argument. If this file is specified you should restrict its file permissions to maintain the integrity of the file. The file should be writable by only the administrators on the system.

Impact:

None

Audit:

Method 1

Optional: Run the following command to enter **debug mode** where NODE_NAME and IMAGE are defined variables:

```
kubectl debug node/$NODE_NAME -it --image=$IMAGE --profile=general
```

Run the following command on each node to find the appropriate config file:

```
ps -efww | grep kubelet | grep -- '--config'
```

The command line output should show a path for config similar to `--config /etc/kubernetes/kubelet/config.json` which is the location of the config file.

Run this command to obtain the config file permissions:

```
stat -c %U:%G /etc/kubernetes/kubelet/config.json (or stat -c %U:%G
/host/etc/kubernetes/kubelet/config.json if in debub mode)
```

The output of the above command is the Kubelet config file's ownership. Verify that the ownership is set to `root:root`

Method 2

Create and Run a Privileged Pod.

You will need to run a pod that is privileged enough to access the host's file system. This can be achieved by deploying a pod that uses the hostPath volume to mount the node's file system into the pod.

Here's an example of a simple pod definition that mounts the root of the host to /host within the pod:

```
apiVersion: v1
kind: Pod
metadata:
  name: file-check
spec:
  volumes:
  - name: host-root
    hostPath:
      path: /
      type: Directory
  containers:
  - name: nsenter
    image: busybox
    command: ["sleep", "3600"]
    volumeMounts:
    - name: host-root
      mountPath: /host
    securityContext:
      privileged: true
```

Save this to a file (e.g., file-check-pod.yaml) and create the pod:

```
kubectl apply -f file-check-pod.yaml
```

Once the pod is running, you can exec into it to check file ownership on the node:

```
kubectl exec -it -n kube-public file-check -- sh
```

Now you are in a shell inside the pod, but you can access the node's file system through the /host directory and check the ownership of the file:

```
ls -l /etc/kubernetes/kubelet/config.json
```

The output of the above command gives you the azure.json file's ownership. Verify that the ownership is set to `root:root`.

Remediation:

Run the following command (using the config file location identified in the Audit step)

```
chown root:root /etc/kubernetes/kubelet/config.json
```

Default Value:

See the AWS EKS documentation for the default value.

References:

1. https://kubernetes.io/docs/reference/command-line-tools-reference/kube-proxy/

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	3.3 Configure Data Access Control Lists Configure data access control lists based on a user's need to know. Apply data access control lists, also known as access permissions, to local and remote file systems, databases, and applications.	●	●	●
v7	5.2 Maintain Secure Images Maintain secure images or templates for all systems in the enterprise based on the organization's approved configuration standards. Any new system deployment or existing system that becomes compromised should be imaged using one of those images or templates.		●	●

3.2 Kubelet

Kubelets can accept configuration via a configuration file and in some cases via command line arguments. It is important to note that parameters provided as command line arguments will override their counterpart parameters in the configuration file (see `--config` details in the Kubelet CLI Reference for more info, where you can also find out which configuration parameters can be supplied as a command line argument).

With this in mind, it is important to check for the existence of command line arguments as well as configuration file entries when auditing Kubelet configuration.

Firstly, SSH to each node and execute the following command to find the Kubelet process:

```
ps -efww | grep kubelet | grep -- '--config'
```

The output of the above command provides details of the active Kubelet process, from which we can see the command line arguments provided to the process. Also note the location of the configuration file, provided with the `--config` argument, as this will be needed to verify configuration. The file can be viewed with a command such as `more` or `less`, like so:

```
sudo less /path/to/kubelet-config.json
```

This config file could be in JSON or YAML format depending on your distribution.

3.2.1 Ensure kubelet anonymous authentication is disabled (Automated)

Profile Applicability:

- Level 1

Description:

In Amazon EKS, this control means the worker node kubelet must not accept unauthenticated requests. In current EKS node bootstrapping, kubelet settings are applied through the node's effective `KubeletConfiguration`, so this requirement is represented as `authentication.anonymous.enabled: false` in the live kubelet configuration rather than depending on a legacy command-line flag being visible in the process list.

Rationale:

The kubelet exposes an HTTPS API on each worker node, and Kubernetes documents that requests not rejected by other configured authentication methods are otherwise treated as `system:anonymous` in the `system:unauthenticated` group. The `KubeletConfiguration` schema defines `authentication.anonymous.enabled` as the setting that controls this behavior, while EKS `nodeadm` merges kubelet configuration with defaults, so the technically correct review point in EKS is the effective kubelet config on the node.

Impact:

Disabling kubelet anonymous authentication causes unauthenticated requests to be rejected with `401 Unauthorized` instead of being processed as anonymous access, which reduces exposure of sensitive node-local kubelet endpoints and forces callers to use authenticated access paths.

Audit:

To audit whether kubelet anonymous authentication is disabled on an Amazon EKS worker node, query the live actuated kubelet configuration through the Kubernetes API server proxy and inspect the `configz` output. This is the preferred method over grepping the kubelet process because Kubernetes documents that kubelet settings can come from a config file, drop-in files, and command-line flags, and that `configz` shows the final effective configuration.

Using the api configz endpoint search for the status of `authentication... "anonymous":{"enabled":false}` by extracting the live configuration from the nodes running kubelet.

Set the local proxy port and the following variables and provide proxy port number and node name; `HOSTNAME_PORT="localhost-and-port-number"` `NODE_NAME="The-Name-Of-Node-To-Extract-Configuration"` from the output of `kubectl get nodes`

```
kubectl proxy --port=8001 &

export HOSTNAME_PORT=localhost:8001 (example host and port number)
export NODE_NAME=ip-192-168-20-214.us-west-2.compute.internal (example node
name from "kubectl get nodes")

curl -sSL "http://${HOSTNAME_PORT}/api/v1/nodes/${NODE_NAME}/proxy/configz"

or

curl -sSL "http://${HOSTNAME_PORT}/api/v1/nodes/${NODE_NAME}/proxy/configz" |
jq (for a more readable output)
```

Output should show something similar to `"authentication"..."anonymous":{"enabled":false}` in the output

To interpret the result:

- **false** = Pass. Anonymous requests to the kubelet are disabled. Kubernetes states that disabling anonymous access causes unauthenticated requests to receive 401 Unauthorized.
- **true** = Fail. Requests not rejected by another authentication method can be treated as system:anonymous in the system:unauthenticated group.
- **null or missing** = Investigate further. That would be unexpected for a normal kubelet config response because authentication.anonymous is part of the kubelet configuration structure; verify that the request reached the kubelet successfully and that you are querying the intended node.

Remediation:

If the audit returns `true`, the worker node is allowing anonymous requests to the kubelet and should be treated as **noncompliant**. If the result is `null` or the field is `missing`, first confirm the `configz` response is complete, because Kubernetes identifies that endpoint as the view of the kubelet's final actuated configuration. Once the result is validated, remediation in EKS should be performed through the supported node provisioning or node update workflow, not by directly editing the kubelet config file on the host.

Remediation of new cluster

When creating a new EKS cluster or node group, the goal is to ensure worker nodes join the cluster with kubelet anonymous authentication already disabled. This is done by defining the setting in the bootstrap configuration that EKS applies during node creation.

- Define `authentication.anonymous.enabled: false` in the worker node bootstrap configuration so the setting is present when nodes first join the cluster.

- Use a custom launch template when kubelet customization is required at cluster or node group creation time.
- Place the setting in the supported bootstrap path, such as launch template user data or `NodeConfig`, so it becomes part of the node's durable build configuration.
- Validate the live effective kubelet configuration after deployment to confirm the setting resolves to `false`.

Remediation of existing running cluster

For an existing EKS cluster, the objective is to change the node launch configuration and then replace the current nodes so the corrected kubelet setting is applied consistently across the node group. This ensures the fix is durable and survives restart, scale-out, and node replacement events.

- Update the custom launch template or equivalent node bootstrap configuration so the effective kubelet setting becomes `authentication.anonymous.enabled: false`.
- Apply the updated launch template to the managed node group so EKS launches replacement nodes with the corrected configuration.
- Replace or roll the existing nodes so the running fleet is rebuilt from the updated launch configuration.
- Re-audit the replacement nodes using the live kubelet configuration to confirm the effective value is `false`.

Note: Even though it is tempting to do so, direct edits to the kubelet config file on EKS worker nodes are discouraged because EKS applies kubelet settings through bootstrap user data and launch configuration, and the effective kubelet values can be influenced by multiple configuration sources. As a result, a host-level edit may be overridden, may not match the live actuated configuration, and can be lost when the node is recycled or replaced.

Default Value:

EKS anonymous authentication is disabled (false) by default

References:

1. https://docs.aws.amazon.com/eks/latest/userguide/launch-templates.html
2. https://docs.aws.amazon.com/eks/latest/userguide/update-managed-node-group.html
3. https://docs.aws.amazon.com/eks/latest/eksctl/customizing-the-kubelet.html
4. https://docs.aws.amazon.com/eks/latest/eksctl/node-bootstrapping.html

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	5.3 Disable Dormant Accounts Delete or disable any dormant accounts after a period of 45 days of inactivity, where supported.	●	●	●
v7	14.6 Protect Information through Access Control Lists Protect all information stored on systems with file system, network share, claims, application, or database specific access control lists. These controls will enforce the principle that only authorized individuals should have access to the information based on their need to access the information as a part of their responsibilities.	●	●	●

3.2.2 Ensure kubelet authorization mode is set to Webhook (Automated)

Profile Applicability:

- Level 1

Description:

In EKS, the available kubelet authorization modes are `AlwaysAllow` and `Webhook`. For worker nodes, the recommended setting is `Webhook`, which uses the Kubernetes authorization flow to evaluate access to the kubelet API. in contrast, `AlwaysAllow` is not optimal because it bypasses that authorization decision and permits kubelet requests without a webhook-based access check. This is also consistent with AWS guidance for EKS-related kubelet behavior, including AWS documentation that treats kubelet authorization as managed node configuration and AWS monitoring guidance that expects kubelet Webhook authorization for supported integrations such as Container Insights.

Rationale:

AWS documents that kubelet `authorization` is a managed field in EKS node configuration and not one that `eksctl` normally lets you overwrite, which reflects that authorization behavior is part of the node's baseline configuration. AWS also documents that EKS node customization should be delivered through launch templates and bootstrap configuration, and that managed node group user data is merged with Amazon EKS user data. In that model, the technically correct target is the effective kubelet configuration on the node, with `authorization.mode` set to `Webhook` instead of an unrestricted mode such as `AlwaysAllow`.

Impact:

Setting kubelet authorization mode to `Webhook` keeps kubelet access aligned with AWS-supported EKS authorization behavior. `AlwaysAllow` is not recommended because it bypasses that authorization check and can also affect supported observability scenarios, since AWS documents kubelet Webhook authorization as a prerequisite for Container Insights.

Audit:

To audit whether kubelet authentication is set to Webhook on an Amazon EKS worker node, query the live effective kubelet configuration through the Kubernetes API server proxy and inspect the kubelet `configz` output for `authorization.mode`. Kubernetes identifies `configz` as the view of the kubelet's final actuated configuration, and the kubelet config schema defines the valid authorization modes as `AlwaysAllow` and `Webhook`. In EKS, this is the right place to verify the control because kubelet settings are applied through managed node configuration and bootstrap behavior rather than only through visible process flags.

Using the api configz endpoint search for the status of `authentication.."webhook".."enabled":true` by extracting the live configuration from the nodes running kubelet.

Set the local proxy port and the following variables and provide proxy port number and node name; `HOSTNAME_PORT="localhost-and-port-number" NODE_NAME="The-Name-Of-Node-To-Extract-Configuration"` from the output of `kubectl get nodes`

```
kubectl proxy --port=8001 &

export HOSTNAME_PORT=localhost:8001 (example host and port number)
export NODE_NAME=ip-192-168-20-214.us-west-2.compute.internal (example node
name from "kubectl get nodes")

curl -sSL "http://${HOSTNAME_PORT}/api/v1/nodes/${NODE_NAME}/proxy/configz"

or

curl -sSL "http://${HOSTNAME_PORT}/api/v1/nodes/${NODE_NAME}/proxy/configz" |
jq (for a more readable output)
```

Output should show something similar to `authentication.."webhook".."enabled":true` in the output

To interpret the result:

- **Webhook** = Pass. This is the compliant result. Kubernetes documents that Webhook mode uses the SubjectAccessReview API to determine authorization.
- **AlwaysAllow** = Fail. This is noncompliant because requests are allowed without webhook-based authorization. AWS also documents kubelet Webhook authorization as a prerequisite for Container Insights.
- **null or missing** = Investigate further. Recheck the raw configz output and confirm the proxy path and node name are correct, because configz is the authoritative view of the final kubelet configuration.

Remediation:

If the audit returns `AlwaysAllow`, the worker node is permitting kubelet requests without webhook-based authorization and should be treated as **noncompliant**. If the result is null or the field is missing, first confirm the `configz` response is complete, because Kubernetes identifies that endpoint as the view of the kubelet's final actuated configuration. Once the result is validated, remediation in EKS should be performed through the supported node provisioning or node update workflow, not by directly editing the kubelet config file on the host. AWS documents that EKS merges managed node bootstrap user data, treats kubelet `authorization` as a managed field in EKS tooling, and applies launch template updates to running managed node groups by recycling nodes.

Remediation of new cluster

When creating a new EKS cluster or node group, the goal is to ensure worker nodes join the cluster with kubelet authorization already set to Webhook. This is done by defining the setting in the bootstrap configuration that EKS applies during node creation.

- Define `authorization.mode: Webhook` in the worker node bootstrap configuration so the setting is present when nodes first join the cluster.
- Use a custom launch template when kubelet customization is required at cluster or node group creation time. AWS documents launch templates as the supported path for deeper managed node customization.
- Place the setting in the supported bootstrap path, such as launch template user data or `NodeConfig`, so it becomes part of the node's durable build configuration.
- Validate the live effective kubelet configuration after deployment to confirm the setting resolves to `Webhook`.

Remediation of existing running cluster

For an existing EKS cluster, the objective is to change the node launch configuration and then replace the current nodes so the corrected kubelet setting is applied consistently across the node group. This ensures the fix is durable and survives restart, scale-out, and node replacement events.

- Update the custom launch template or equivalent node bootstrap configuration so the effective kubelet setting becomes `authorization.mode: Webhook`.
- Apply the updated launch template to the managed node group so EKS launches replacement nodes with the corrected configuration.
- Replace or roll the existing nodes so the running fleet is rebuilt from the updated launch configuration. AWS states that when a node group is updated to a newer launch template version, every node is recycled to match that version.
- Re-audit the replacement nodes using the live kubelet configuration to confirm the effective value is `Webhook`.

Note: Even though it is tempting to do so, direct edits to the kubelet config file on EKS worker nodes are discouraged because EKS applies kubelet settings through bootstrap user data and launch configuration, while Kubernetes allows effective values to be influenced by multiple configuration sources. As a result, a host-level edit may be overridden, may not match the live actuated configuration, and can be lost when the node is recycled or replaced.

Default Value:

The default is authorization set to Webhook and enabled true

References:

1. https://docs.aws.amazon.com/eks/latest/eksctl/customizing-the-kubelet.html
2. https://docs.aws.amazon.com/eks/latest/userguide/launch-templates.html

3. https://docs.aws.amazon.com/eks/latest/userguide/update-managed-node-group.html
4. https://docs.aws.amazon.com/AmazonCloudWatch/latest/monitoring/Container-Insights-prerequisites.html

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	5.4 Restrict Administrator Privileges to Dedicated Administrator Accounts Restrict administrator privileges to dedicated administrator accounts on enterprise assets. Conduct general computing activities, such as internet browsing, email, and productivity suite use, from the user's primary, non-privileged account.	●	●	●
v7	4.2 Change Default Passwords Before deploying any new asset, change all default passwords to have values consistent with administrative level accounts.	●	●	●

3.2.3 Ensure the kubelet X.509 client CA file is configured (Automated)

Profile Applicability:

- Level 1

Description:

This setting in EKS is about configuring the kubelet with a trusted X.509 client CA source so the kubelet can validate client certificates presented to its HTTPS endpoint. It is not the same as validating the kubelet's own serving certificate from the API server side. In EKS, kubelet authentication settings are part of the node's managed configuration baseline and should be evaluated in the effective kubelet configuration applied during bootstrap.

Rationale:

This setting establishes the trust anchor the kubelet uses for certificate-based client authentication at the node level. For EKS, that trust source is not something you should assume the control plane will generate automatically, because AWS states that client certificate signing is not supported by the Amazon EKS CA signer. AWS also documents that kubelet authentication is a managed field in EKS tooling, and that deeper node customization is delivered through launch templates, bootstrap arguments, merged user data, or a custom AMI when needed. As a result, if your EKS worker node design depends on X.509 client authentication to the kubelet, the client CA file must be intentionally provisioned through the supported node bootstrap and launch path so the node starts with a consistent trust source rather than relying on manual host edits or an unsupported EKS CA workflow.

Impact:

If the kubelet X.509 client CA file is not configured, certificate-based clients cannot be validated against a trusted CA when connecting to the kubelet, which can break intended X.509-based access patterns or leave that authentication path unavailable. In EKS, remediation should be made through bootstrap or launch configuration and then applied through node replacement, because AWS documents that node configuration is delivered through merged bootstrap behavior and updated launch template versions are rolled out by recycling nodes.

Audit:

To audit whether kubelet authentication X.509 client CA file is configured on an Amazon EKS worker node, query the live effective kubelet configuration through the Kubernetes API server proxy and inspect the kubelet `configz` output for `authentication.x509.clientCAFile`. Kubernetes identifies `configz` as the view of the kubelet's final actuated configuration, and the kubelet config schema defines `authentication.x509.clientCAFile` as the setting used for X.509 client certificate authentication. In EKS, this is the right place to verify the control because kubelet authentication settings are part of managed node configuration and node bootstrap behavior rather than only visible as process flags.

Using the api configz endpoint search for the status of `authentication...clientCAFile` by extracting the live configuration from the nodes running kubelet.

Set the local proxy port and the following variables and provide proxy port number and node name; `HOSTNAME_PORT="localhost-and-port-number" NODE_NAME="The-Name-Of-Node-To-Extract-Configuration"` from the output of `kubectl get nodes`

```
kubectl proxy --port=8001 &

export HOSTNAME_PORT=localhost:8001 (example host and port number)
export NODE_NAME=ip-192-168-20-214.us-west-2.compute.internal (example node
name from "kubectl get nodes")

curl -sSL "http://${HOSTNAME_PORT}/api/v1/nodes/${NODE_NAME}/proxy/configz"

or

curl -sSL "http://${HOSTNAME_PORT}/api/v1/nodes/${NODE_NAME}/proxy/configz" |
jq (for a more readable output)
```

Output should show something similar to `authentication":{"x509":{"clientCAFile":"/etc/kubernetes/pki/ca.crt"}` in the output

To interpret the result:

- A **non-empty** path, such as `/etc/kubernetes/pki/ca.crt` or another valid file path, means the setting is configured. This is the expected result for this control because the kubelet has a CA bundle path defined for validating client certificates.
- An **empty** string means the field is present but not configured. Treat this as noncompliant for this control because the kubelet does not have a CA bundle path configured for X.509 client certificate validation.
- **null** or a **missing** field should be treated as investigate further. Recheck the raw `configz` response and confirm the proxy path and node name are correct, because `configz` is the authoritative view of the final kubelet configuration. In EKS, if the value still cannot be verified, review the node bootstrap and launch customization path because authentication settings are managed there.

Remediation:

If the audit shows that the kubelet X.509 client CA file is not configured, the worker node should be treated as **noncompliant**. If the result is empty, null, or missing, first confirm the inspection reflects the effective node configuration. In Amazon EKS, remediation should be performed through the supported node provisioning or node update workflow, because AWS documents that kubelet `authentication` is a managed field in `eksctl`, launch templates are the supported path for deeper managed node customization, and client certificate signing is not supported by the Amazon EKS CA signer.

Remediation of new cluster

When creating a new EKS cluster or node group, the goal is to ensure each worker node starts with a trusted client CA bundle already present and with the kubelet configured to use it for certificate-based client authentication.

- Select and manage the CA that will anchor trust for kubelet client certificates, because Amazon EKS does not support client certificate signing through the Amazon EKS CA signer.
- Use a custom launch template for this kubelet authentication customization, since AWS uses launch templates for deeper managed node configuration.
- Use a custom AMI or equivalent bootstrap customization, when needed, to place the CA bundle on the node and point kubelet to that file path. AWS notes that with a custom AMI, you are responsible for the required bootstrap logic.
- Validate after deployment that the node has the CA file at the expected path and that the kubelet is using the intended authentication configuration.

Remediation of existing running cluster

For an existing EKS cluster, the objective is to update the node launch configuration so future nodes include the required client CA bundle and kubelet reference, then replace the current nodes so the change is applied consistently across the node group.

- Prepare an updated custom launch template, and a new custom AMI if required, that places the CA bundle on the node and configures kubelet to use it.
- If the managed node group already uses a custom launch template, update it to a new version so EKS can roll the node group to that configuration.
- If the managed node group was not created with a launch template, build a new node group with the required launch template or custom AMI path and migrate workloads to it.
- Re-audit the replacement nodes to confirm the client CA file is now configured through the intended node build path.

Note: Even though it is tempting to do so, direct edits to the kubelet config file on EKS worker nodes are discouraged because node behavior is applied through bootstrap, launch configuration, and managed node replacement workflows, so host-level edits can be lost when nodes are recycled or replaced.

Default Value:

No fixed EKS default is documented; verify the live kubelet config.

References:

1. https://docs.aws.amazon.com/eks/latest/eksctl/customizing-the-kubelet.html
2. https://docs.aws.amazon.com/eks/latest/userguide/launch-templates.html
3. https://docs.aws.amazon.com/eks/latest/userguide/cert-signing.html
4. https://docs.aws.amazon.com/eks/latest/eksctl/node-bootstrapping.html

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	3.10 Encrypt Sensitive Data in Transit Encrypt sensitive data in transit. Example implementations can include: Transport Layer Security (TLS) and Open Secure Shell (OpenSSH).		●	●
v7	14.4 Encrypt All Sensitive Information in Transit Encrypt all sensitive information in transit.		●	●

3.2.4 Ensure the kubelet read-only port is disabled (Automated)

Profile Applicability:

- Level 1

Description:

In EKS, this setting is intended to keep kubelet access limited to the standard secured management path used for node operations, instead of exposing an additional legacy interface on the worker node. In practice, it should be assessed in the node's effective kubelet configuration because AWS applies node behavior through bootstrap configuration, launch templates, and managed node initialization rather than relying only on visible runtime flags.

Rationale:

Kubernetes defines `readOnlyPort` as a read-only kubelet service with no authentication or authorization, and specifies that setting `readOnlyPort: 0` disables that service. A nonzero value creates an extra listener that is separate from the main kubelet serving port. In EKS, AWS documents that normal control plane to node operations such as logs, exec, and port-forward use TCP port `10250`, so the preferred design is to keep kubelet access consolidated on that standard endpoint and leave the unauthenticated read-only service disabled. Because AWS uses launch templates and bootstrap configuration for deeper node customization, this state should be enforced through the node's managed configuration path.

Impact:

Disabling the kubelet `readOnlyPort` removes an unnecessary unauthenticated kubelet service while preserving the standard kubelet access path EKS uses on port 10250

Audit:

To audit whether the kubelet read-only port is disabled on an Amazon EKS worker node, query the live effective kubelet configuration through the Kubernetes API server proxy and inspect the kubelet configz output for `readOnlyPort` set to `0`. Kubernetes identifies configz as the view of the kubelet's final actuated configuration, and the kubelet configuration schema defines `readOnlyPort` as the setting that controls the unauthenticated read-only kubelet service. In EKS, this is the appropriate place to verify the control because kubelet settings are applied through managed node configuration and bootstrap behavior, rather than being reliably visible only as process flags.

Using the api configz endpoint search for the status of `readOnlyPort` by extracting the live configuration from the nodes running kubelet.

Set the local proxy port and the following variables and provide proxy port number and node name; `HOSTNAME_PORT="localhost-and-port-number" NODE_NAME="The-Name-Of-Node-To-Extract-Configuration"` from the output of `kubectl get nodes`

```
kubectl proxy --port=8001 &

export HOSTNAME_PORT=localhost:8001 (example host and port number)
export NODE_NAME=ip-192-168-20-214.us-west-2.compute.internal (example node
name from "kubectl get nodes")

curl -sSL "http://${HOSTNAME_PORT}/api/v1/nodes/${NODE_NAME}/proxy/configz"

or

curl -sSL "http://${HOSTNAME_PORT}/api/v1/nodes/${NODE_NAME}/proxy/configz" |
jq (for a more readable output)
```

Output should show something similar to `readOnlyPort: 0` in the output

To interpret the result:

- **0** = Pass. Kubernetes defines 0 as disabled, which is the expected state for this control.
- A **nonzero** integer such as 10255 = Fail. That means the kubelet read-only service is enabled and is serving without authentication or authorization.
- **null** or a **missing** field = If `readOnlyPort` is absent from configz, it commonly indicates the effective value is 0, which disables the unauthenticated read-only kubelet service.

Remediation:

If the audit returns a nonzero `readOnlyPort`, the worker node is exposing the kubelet read-only service and should be treated as **noncompliant**. If the field is missing from `configz`, first confirm whether it is simply omitted because the effective value is `0`, since the kubelet configuration schema defines `readOnlyPort` as the unauthenticated read-only service and states that `0` disables it. Once a noncompliant state is confirmed, remediation in EKS should be performed through the supported node provisioning or node update workflow, not by directly editing the kubelet config file on the host. AWS documents that managed node customization is applied through launch templates and bootstrap logic, and that updating a managed node group to a new launch template version replaces the nodes to match that configuration.

Remediation of new cluster

When creating a new EKS cluster or node group, the goal is to ensure worker nodes start with the kubelet read-only service disabled in the effective kubelet configuration. In practice, that means setting the node bootstrap configuration so readOnlyPort resolves to 0 from the time the node joins the cluster.

- Define the kubelet configuration so `readOnlyPort: 0`, which disables the unauthenticated read-only kubelet service.
- Use a custom launch template when kubelet customization is required at cluster or node group creation time, because AWS uses launch templates for deeper managed node customization.

- Place the setting in the supported bootstrap path, such as launch template user data, `NodeConfig`, or a custom AMI workflow when needed, so it becomes part of the node's durable build configuration.
- Validate after deployment that the live kubelet configuration reflects the intended state and that the read-only service is not enabled on the node.

Remediation of existing running cluster

For an existing EKS cluster, the objective is to update the node launch configuration so replacement nodes start with the kubelet read-only service disabled, then roll or replace the current nodes so the corrected state is applied consistently across the node group. This makes the fix durable across restart, scale-out, and node replacement events.

- Update the custom launch template or equivalent node bootstrap configuration so the effective kubelet setting resolves to `readOnlyPort: 0`.
- Apply the updated launch template version to the managed node group so EKS launches replacement nodes with the corrected configuration.
- Replace or roll the existing nodes so the running fleet is rebuilt from the updated launch configuration, because AWS applies node group configuration changes by replacing nodes.
- Re-audit the replacement nodes using the live kubelet configuration and, if needed, confirm that no service is listening on the read-only kubelet port.

Note: Even though it is tempting to do so, direct edits to the kubelet config file on EKS worker nodes are discouraged because EKS applies node behavior through managed bootstrap, launch templates, and node replacement workflows. A host-level edit can therefore be overridden, may not reflect the final actuated kubelet configuration, and can be lost when the node is recycled or replaced.

Default Value:

The default setting for EKS Cluster nodes is readOnlyPort: 0 (disabled). If missing from configz, that can still be consistent with a disabled state because zero-value fields may be omitted from serialized config output.

References:

1. https://docs.aws.amazon.com/eks/latest/userguide/launch-templates.html
2. https://docs.aws.amazon.com/eks/latest/eksctl/customizing-the-kubelet.html
3. https://docs.aws.amazon.com/eks/latest/eksctl/node-bootstrapping.html
4. https://docs.aws.amazon.com/AWSCloudFormation/latest/TemplateReference/aws-properties-eks-cluster-remotenetworkconfig.html

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	12.6 Use of Secure Network Management and Communication Protocols Use secure network management and communication protocols (e.g., 802.1X, Wi-Fi Protected Access 2 (WPA2) Enterprise or greater).		●	●
v7	9.2 Ensure Only Approved Ports, Protocols and Services Are Running Ensure that only network ports, protocols, and services listening on a system with validated business needs, are running on each system.		●	●

3.2.5 Ensure the kubelet iptables utility chains are enabled (Automated)

Profile Applicability:

- Level 1

Description:

This control is intended to keep kubelet-managed `iptables` support enabled on worker nodes so the node maintains the expected networking behavior for Kubernetes traffic handling. The recommended state is `makeIPTablesUtilChains: true`, and in EKS this should be enforced through the node's managed bootstrap and launch configuration, because AWS applies deeper node customization through launch templates and merged user data.

Rationale:

Amazon EKS worker nodes rely on Kubernetes-managed `iptables` state for normal network operation. AWS documents that `kube-proxy` maintains network rules on each node to enable network communication to pods, and that if the required `iptables` rules are not set correctly, Service routing can fail and supporting components such as the CNI can be disrupted. Keeping kubelet `iptables` utility chains enabled helps preserve that expected host networking state and reduces the risk that manual or inconsistent `iptables` handling will break pod communication or service connectivity.

Impact:

Keeping kubelet `iptables` utility chains enabled helps preserve the node-level networking behavior that Amazon EKS expects for pod communication and Service routing. AWS documents that `kube-proxy` runs on each EC2 node and maintains the network rules that enable communication to pods, so disabling this setting can make host packet handling and service connectivity less predictable.

Audit:

To audit whether the kubelet `makeIPTablesUtilChains` is enabled on an Amazon EKS worker node, query the live effective kubelet configuration through the Kubernetes API server proxy and inspect the kubelet configz output for `makeIPTablesUtilChains`. Kubernetes identifies `configz` as the view of the kubelet's final actuated configuration, and the kubelet configuration schema defines `makeIPTablesUtilChains` with a default of `true`. In EKS, this is the right place to verify the control because AWS applies deeper node customization through launch templates and merged bootstrap user data rather than relying only on legacy process flags.

Using the api configz endpoint search for the status of `makeIPTablesUtilChains` by extracting the live configuration from the nodes running kubelet.

Set the local proxy port and the following variables and provide proxy port number and node name; `HOSTNAME_PORT="localhost-and-port-number"` `NODE_NAME="The-Name-Of-Node-To-Extract-Configuration"` from the output of `kubectl get nodes`

```
kubectl proxy --port=8001 &

export HOSTNAME_PORT=localhost:8001 (example host and port number)
export NODE_NAME=ip-192-168-20-214.us-west-2.compute.internal (example node
name from "kubectl get nodes")

curl -sSL "http://${HOSTNAME_PORT}/api/v1/nodes/${NODE_NAME}/proxy/configz"

or

curl -sSL "http://${HOSTNAME_PORT}/api/v1/nodes/${NODE_NAME}/proxy/configz" |
jq (for a more readable output)
```

Output should show something similar to `makeIPTablesUtilChains.:true` in the output

To interpret the result:

- **true** = Pass. This is the expected state for the control. Kubernetes defines true as enabling kubelet iptables utility chain creation, and AWS documents that EKS node networking relies on components such as `kube-proxy` to maintain node network rules.
- **false** = Fail. This means kubelet iptables utility chains are disabled and the node should be treated as noncompliant. Since AWS documents that EKS worker nodes rely on node-level networking rules to enable pod communication, disabling related kubelet iptables support is not the preferred state.
- **null** or **missing** = Investigate further. Recheck the raw configz output and confirm the proxy path and node name are correct, because `configz` is the authoritative view of the kubelet's final actuated configuration. If the value still cannot be verified, review the node bootstrap and launch-template customization path used by that node group.

Remediation:

If the audit returns false, the worker node is not keeping kubelet iptables utility chains enabled and should be treated as **noncompliant**. If the result is null or the field is missing, first confirm the configz response is complete, because Kubernetes identifies that endpoint as the view of the kubelet's final actuated configuration, and the kubelet configuration schema defines `makeIPTablesUtilChains` with a default of `true`. Once the result is validated, remediation in EKS should be performed through the supported node provisioning or node update workflow, not by directly editing the kubelet config file on the host. AWS documents that managed node customization is applied through launch templates and bootstrap logic, and that managed node group updates to a new launch template version replace the nodes to match that configuration.

Remediation of new cluster

When creating a new EKS cluster or node group, the goal is to ensure worker nodes start with kubelet iptables utility chains enabled as part of the node's baseline configuration. In practice, that means setting the node bootstrap configuration so `makeIPTablesUtilChains` resolves to `true` when the node joins the cluster. AWS documents launch templates as the supported path for deeper managed node customization, including bootstrap arguments and custom node configuration.

- Define the kubelet configuration so `makeIPTablesUtilChains: true`, which keeps kubelet `iptables` utility chain creation enabled.
- Use a custom launch template when kubelet customization is required at cluster or node group creation time, because AWS uses launch templates for deeper managed node customization.
- Place the setting in the supported bootstrap path, such as launch template user data, NodeConfig, or a custom AMI workflow when needed, so it becomes part of the node's durable build configuration.
- Validate after deployment that the live kubelet configuration reflects the intended state.

Remediation of existing running cluster

For an existing EKS cluster, the objective is to update the node launch configuration so replacement nodes start with kubelet `iptables` utility chains enabled, then roll or replace the current nodes so the corrected state is applied consistently across the node group. This makes the fix durable across restart, scale-out, and node replacement events. AWS documents that managed node groups can be updated to a newer version of the same launch template, and that launch template changes are applied by replacing the nodes.

- Update the custom launch template or equivalent node bootstrap configuration so the effective kubelet setting resolves to `makeIPTablesUtilChains: true`.
- Apply the updated launch template version to the managed node group so EKS launches replacement nodes with the corrected configuration.
- Replace or roll the existing nodes so the running fleet is rebuilt from the updated launch configuration.
- Re-audit the replacement nodes using the live kubelet configuration to confirm the effective value is `true`.

Note: Even though it is tempting to do so, direct edits to the kubelet config file on EKS worker nodes are discouraged because AWS applies node behavior through bootstrap user data, launch configuration, and managed node replacement workflows. A host-level edit can therefore be overridden, may not reflect the final actuated kubelet configuration, and can be lost when the node is recycled or replaced.

Default Value:

The default setting for this control is `makeIPTablesUtilChains: true`. Kubernetes defines this field as causing the kubelet to create the `KUBE-IPTABLES-HINT` chain in `iptables` as a hint to other components about the system's `iptables` configuration, and the schema lists the default as `true`.

References:

1. https://docs.aws.amazon.com/eks/latest/userguide/launch-templates.html
2. https://docs.aws.amazon.com/eks/latest/userguide/managing-kube-proxy.html
3. https://docs.aws.amazon.com/eks/latest/eksctl/customizing-the-kubelet.html
4. https://docs.aws.amazon.com/eks/latest/eksctl/node-bootstrapping.html

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	12.3 Securely Manage Network Infrastructure Securely manage network infrastructure. Example implementations include version-controlled-infrastructure-as-code, and the use of secure network protocols, such as SSH and HTTPS.		●	●
v7	11.1 Maintain Standard Security Configurations for Network Devices Maintain standard, documented security configuration standards for all authorized network devices.		●	●

3.2.6 Ensure the kubelet event recording QPS is set to an appropriate value (Automated)

Profile Applicability:

- Level 1

Description:

In EKS, this control is about tuning how quickly the kubelet records events so worker nodes capture useful operational data without generating unnecessary event volume. AWS documents the kubelet as a customer-configurable component in EKS and supports kubelet tuning through managed node configuration paths such as launch templates, while the Kubernetes kubelet configuration defines `eventRecordQPS` with a default value of `50` events per second.

Rationale:

The setting `eventRecordQPS` is the kubelet rate limit for event creation. Kubernetes defines `50` as the default and states that `0` removes the limit entirely. AWS's EKS scaling guidance explains that Kubernetes components use QPS-based controls to avoid overwhelming downstream services and warns that removing a kubelet-side bottleneck can shift pressure to the next link in the service chain. In practice, that means a value that is too low can suppress event creation during burst conditions such as node instability, container restart loops, or rapid scheduling churn, while a value of `0` can allow unrestricted event generation and increase load on the API path and related control plane components. This setting should therefore be tuned through supported node configuration rather than treated as an arbitrary host-level change.

Impact:

Setting `eventRecordQPS` too low can throttle or suppress relevant kubelet events during burst conditions, which reduces diagnostic visibility. Leaving it at an appropriate bounded value, such as the default of `50`, helps preserve event capture while limiting event traffic. Setting it to `0` removes the rate limit entirely, which can allow unbounded event generation from the node in EKS. AWS warns that removing a kubelet-side QPS bottleneck can overwhelm downstream components above a certain rate, so this can contribute to denial-of-service conditions caused by load on the API path or related services rather than simply improving event capture.

Audit:

To audit whether the kubelet event recording QPS is appropriately set on an Amazon EKS worker node, query the live effective kubelet configuration through the Kubernetes API server proxy and inspect the kubelet `configz` output for `eventRecordQPS`. Kubernetes identifies `configz` as the view of the kubelet's final actuated configuration, and AWS notes that kubelet QPS settings are part of the customer-configurable side of EKS and can affect downstream components if changed carelessly.

Using the api configz endpoint search for the status of `eventRecordQPS` by extracting the live configuration from the nodes running kubelet.

Set the local proxy port and the following variables and provide proxy port number and node name; `HOSTNAME_PORT="localhost-and-port-number" NODE_NAME="The-Name-Of-Node-To-Extract-Configuration"` from the output of `kubectl get nodes`

```
kubectl proxy --port=8001 &

export HOSTNAME_PORT=localhost:8001 (example host and port number)
export NODE_NAME=ip-192-168-20-214.us-west-2.compute.internal (example node
name from "kubectl get nodes")

curl -sSL "http://${HOSTNAME_PORT}/api/v1/nodes/${NODE_NAME}/proxy/configz"

or

curl -sSL "http://${HOSTNAME_PORT}/api/v1/nodes/${NODE_NAME}/proxy/configz" |
jq (for a more readable output)
```

Output should show something similar to `eventRecordQPS":50` in the output

To interpret the result:

- **50** = Using the kubelet schema default unless it has been explicitly overridden. Kubernetes defines `50` as the default value for `eventRecordQPS`.
- **0** = No rate limit is enforced. This should be reviewed carefully because Kubernetes defines `0` as unlimited event creation, and AWS warns that removing a kubelet-side QPS bottleneck can overwhelm downstream components above a certain rate.
- **Value greater than 50** = Event creation is rate-limited to that configured value and should be evaluated against the cluster's event volume and diagnostic needs.
- **null** or **missing** = Investigate further by reviewing the raw `configz` response and the node bootstrap path, because `configz` is the authoritative view of the kubelet's final actuated configuration.

Remediation:

If the audit shows `eventRecordQPS` is set too low, the worker node may throttle useful event creation during burst conditions and should be adjusted. If the value is `0`, Kubernetes defines that as no limit enforced, so it should be used only when the additional event volume is intentional and downstream capacity has been considered. If the field is missing or null, first confirm the `configz` response is complete, because Kubernetes identifies `configz` as the view of the kubelet's final actuated configuration. In EKS, remediation should be made through the supported node provisioning or node update workflow, not by directly editing the kubelet config file on the host, because AWS applies deeper node customization through launch templates and merged bootstrap user data.

Remediation of new cluster

When creating a new EKS cluster or node group, set `eventRecordQPS` so worker nodes capture useful events without creating unnecessary downstream load. Kubernetes defines the default as `50` and `0` as unlimited, so the starting point is usually the default unless observed event loss justifies a different value. In EKS, apply this through supported node configuration such as launch-template-based bootstrap customization.

- Define `eventRecordQPS` in the node bootstrap configuration so the intended rate limit is applied when nodes join the cluster.
- Start from the default of `50` unless measured event loss or burst conditions justify a change.
- Avoid `0` unless unlimited event creation is intentional and monitored, because it removes the kubelet-side rate limit entirely.
- Use a custom launch template or equivalent supported node customization path when this setting must be enforced as part of node build or bootstrap behavior.
- Validate after deployment that the live kubelet configuration reflects the intended `eventRecordQPS` value.

Remediation of existing running cluster

For an existing EKS cluster, update the node launch configuration so replacement nodes start with the corrected `eventRecordQPS` value, then roll or replace the current nodes so the change is applied consistently across the node group. AWS documents that when a managed node group uses your custom launch template, updating to a new version of that template recycles all nodes in the group to match the new configuration.

- Update the custom launch template or equivalent bootstrap configuration so the effective kubelet setting resolves to the intended `eventRecordQPS` value.
- Raise the value when relevant events are being throttled lower it when event volume is excessive avoid `0` unless unlimited event creation is justified and downstream impact is understood.
- Apply the new launch template version to the managed node group so EKS replaces nodes with the corrected configuration.
- If the existing node group was not created with a custom launch template, build a new node group with the required configuration and migrate workloads to it, because AWS says those existing groups cannot be updated directly to use a custom launch template.
- Re-audit the replacement nodes against the live kubelet configuration and confirm the new value provides the intended balance between event capture and downstream load.

Note: Even though it is tempting to do so, direct edits to the kubelet config file on EKS worker nodes are discouraged because the effective kubelet configuration can come from multiple sources, and AWS applies node behavior through bootstrap user data and launch configuration. A host-level edit can therefore be overridden and lost when the node is recycled or replaced.

Default Value:

The default setting is `eventRecordQPS: 50`. Kubernetes defines `eventRecordQPS` as the maximum event creations per second and states that `0` means no limit is enforced.

References:

1. https://docs.aws.amazon.com/eks/latest/best-practices/kubernetes_scaling_theory.html
2. https://docs.aws.amazon.com/eks/latest/userguide/launch-templates.html
3. https://docs.aws.amazon.com/eks/latest/eksctl/customizing-the-kubelet.html
4. https://docs.aws.amazon.com/eks/latest/eksctl/node-bootstrapping.html

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	8.2 Collect Audit Logs Collect audit logs. Ensure that logging, per the enterprise's audit log management process, has been enabled across enterprise assets.	●	●	●
v8	8.5 Collect Detailed Audit Logs Configure detailed audit logging for enterprise assets containing sensitive data. Include event source, date, username, timestamp, source addresses, destination addresses, and other useful elements that could assist in a forensic investigation.		●	●
v7	6.2 Activate audit logging Ensure that local logging has been enabled on all systems and networking devices.	●	●	●
v7	6.3 Enable Detailed Logging Enable system logging to include detailed information such as an event source, date, user, timestamp, source addresses, destination addresses, and other useful elements.		●	●

3.2.7 Ensure the kubelet client certificate rotation is enabled (Automated)

Profile Applicability:

- Level 1

Description:

In EKS, the setting `rotateCertificates` is about whether the kubelet renews its client certificate before that certificate expires. The EKS default is that certificate rotation is not enabled by default for self-managed EKS nodes, and AWS warns that a node running longer than one year can stop authenticating to the Kubernetes API if rotation stays disabled. In practice, that means `true` is the recommended operational state for long-lived nodes, while `false` or a `not present` setting should be treated as rotation not enabled.

In addition, `rotateCertificates` should be not be confused with a similar setting 'RotateKubeletServerCertificate'. Client certificate rotation affects the certificate the kubelet uses when it authenticates to the Kubernetes API server. By contrast, `RotateKubeletServerCertificate` refers to the kubelet's serving certificate, which is the certificate the kubelet presents from its own HTTPS endpoint when another component connects to it. AWS's certificate-signing guidance reflects that distinction by noting that Amazon EKS does not support client certificate signing with the Amazon EKS CA signer, while AWS also shows certificate-signing workflows for server-auth use cases, and AWS Auto Mode separately highlights automated certificate management and rotation as part of the node security model.

Rationale:

Enabling kubelet client certificate rotation allows the kubelet to automatically generate a new key and submit a new certificate signing request through the Kubernetes API before its current client certificate expires. AWS documents that kubelet certificates on self-managed EKS nodes have a one-year lifetime and that certificate rotation is not enabled by default, so a long-lived node can lose the ability to authenticate to the Kubernetes API unless the node is regularly recreated or client certificate rotation is enabled. In practical terms, a setting of `true` means the kubelet is configured to renew its client certificate, while `false` or `not present` should be interpreted conservatively as rotation not enabled unless the platform is explicitly managing certificate renewal for you. AWS also shows that some environments, such as EKS Auto Mode, can handle certificate management and rotation automatically.

This setting is distinct from `RotateKubeletServerCertificate`. Client certificate rotation protects the certificate the node uses to authenticate to the API server, while server certificate rotation separately governs the certificate the kubelet presents from its HTTPS endpoint to callers.

Impact:

When kubelet client certificate rotation is enabled, long-lived EKS worker nodes are less likely to lose access to the Kubernetes API because the kubelet can renew its client certificate before it expires. When the setting is `false` or `not present`, the node depends on replacement or another renewal process to avoid certificate expiry, and AWS explicitly warns that self-managed EKS nodes running longer than one year can stop authenticating to the API if rotation is not enabled.

Audit:

To audit whether the kubelet client certificate rotation is enabled on an Amazon EKS worker node, query the live effective kubelet configuration through the Kubernetes API server proxy and inspect the kubelet `configz` output for `rotateCertificates`. Kubernetes identifies `configz` as the view of the kubelet's final actuated configuration, and AWS documents that EKS nodes can rely on certificate rotation or regular node replacement to avoid kubelet client certificate expiry. For self-managed EKS nodes, AWS explicitly states that certificate rotation is not enabled by default and that nodes running longer than one year can lose the ability to authenticate to the Kubernetes API if rotation is not enabled.

Using the api `configz` endpoint search for the status of `rotateCertificates` by extracting the live configuration from the nodes running kubelet.

Set the local proxy port and the following variables and provide proxy port number and node name; `HOSTNAME_PORT="localhost-and-port-number"` `NODE_NAME="The-Name-Of-Node-To-Extract-Configuration"` from the output of `kubectl get nodes`

```
kubectl proxy --port=8001 &

export HOSTNAME_PORT=localhost:8001 (example host and port number)
export NODE_NAME=ip-192-168-20-214.us-west-2.compute.internal (example node
name from "kubectl get nodes")

curl -sSL "http://${HOSTNAME_PORT}/api/v1/nodes/${NODE_NAME}/proxy/configz"

or

curl -sSL "http://${HOSTNAME_PORT}/api/v1/nodes/${NODE_NAME}/proxy/configz" |
jq (for a more readable output)
```

Output should show something similar to `"rotateCertificates":true` in the output

To interpret the result:

- **true** = Pass. Client certificate rotation is enabled, so the kubelet is configured to renew its client certificate through the Kubernetes certificate lifecycle before expiry.
- **false** = Fail. Client certificate rotation is disabled. AWS warns that for self-managed EKS nodes on Outposts, nodes can lose Kubernetes API

authentication after certificate expiry if rotation is not enabled and nodes are not recreated.
- **null** or **missing** = Investigate further, but treat it conservatively as not enabled unless your platform explicitly manages certificate rotation for you. Kubernetes documents rotateCertificates with a default of false, and AWS documents automated certificate management and rotation for EKS Auto Mode as a separate managed behavior.

Remediation:

If the audit returns `false`, the worker node is not configured to rotate its kubelet client certificate and should be treated as **noncompliant**. If the field is not present, interpret that conservatively as rotation not enabled unless your EKS platform explicitly manages certificate rotation for you. AWS's self-managed EKS node guidance states that kubelet certificates have a one-year expiration, that certificate rotation is not enabled by default, and that nodes can lose the ability to authenticate to the Kubernetes API if they remain in service beyond that period without renewal. Kubernetes also defines `rotateCertificates` as the kubelet setting that enables client certificate rotation and gives it a default of `false`.

Remediation of new cluster

When creating a new EKS cluster or node group, the goal is to ensure worker nodes start with kubelet client certificate rotation enabled if the nodes may remain in service long enough for certificate expiry to matter. In EKS, this should be applied through the supported node bootstrap and launch configuration path rather than as a host-level edit, because AWS documents launch templates as the mechanism for deeper managed node customization.

- Set `rotateCertificates: true` in the worker node's kubelet configuration.
- Use a custom launch template when this setting must be enforced at node creation.
- Apply the setting through the supported bootstrap path, not by editing the host after deployment.
- Verify the live kubelet configuration shows rotation enabled.

Remediation of existing running cluster

For an existing EKS cluster, the objective is to update the node launch configuration so replacement nodes start with kubelet client certificate rotation enabled, then replace the current nodes so the corrected state is applied consistently across the node group. AWS documents that when a managed node group was created with your own launch template, you can update it to a new version of that same template, and all nodes in the group are recycled to match the new configuration.

- Update the launch template or bootstrap configuration so `rotateCertificates: true` is applied.

- Roll the managed node group so replacement nodes launch with the corrected setting.
- If the node group was not created from your own launch template, create a new node group with the required configuration and migrate workloads.
- Re-audit replacement nodes to confirm client certificate rotation is enabled.

Note: Even though it is tempting to do so, direct edits to the kubelet config file on EKS worker nodes are discouraged because AWS applies durable node behavior through launch templates and node replacement workflows. A host-level change can be bypassed by the node's launch configuration and lost when the node is recycled or replaced.

Default Value:

AWS default is false or not present, but the recommended secure state is true for long-lived nodes.

References:

1. https://docs.aws.amazon.com/eks/latest/userguide/eks-outposts-self-managed-nodes.html
2. https://docs.aws.amazon.com/eks/latest/best-practices/autosecure.html
3. https://docs.aws.amazon.com/eks/latest/userguide/launch-templates.html
4. https://docs.aws.amazon.com/eks/latest/userguide/update-managed-node-group.html

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	3.10 Encrypt Sensitive Data in Transit Encrypt sensitive data in transit. Example implementations can include: Transport Layer Security (TLS) and Open Secure Shell (OpenSSH).		●	●
v7	14.4 Encrypt All Sensitive Information in Transit Encrypt all sensitive information in transit.		●	●

3.2.8 Ensure serverTLSBootstrap and RotateKubeletServerCertificate are enabled (Automated)

Profile Applicability:

- Level 1

Description:

In EKS, these settings are about the kubelet's serving certificate, which is the certificate the kubelet presents from its HTTPS endpoint to callers. `serverTLSBootstrap` controls whether the kubelet uses the Kubernetes certificates API to bootstrap that serving certificate, while `RotateKubeletServerCertificate` keeps the related rotation support enabled when feature gates are managed. In EKS, these settings should remain enabled together so the kubelet can participate in an automated serving-certificate lifecycle instead of relying on a static or separately managed certificate path. AWS documents `serverTLSBootstrap` as a managed kubelet field in `eksctl`, and also warns that if custom feature gates are supplied, `featureGates.RotateKubeletServerCertificate=true` should still be included unless you intentionally disable it.

Rationale:

These two settings are tightly coupled, but they perform different functions. `serverTLSBootstrap=true` puts the kubelet on the Kubernetes certificate-signing workflow for its serving certificate, rather than leaving that HTTPS endpoint on a self-signed or separately managed certificate path. `RotateKubeletServerCertificate=true` keeps the related rotation support enabled, which matters because AWS documents that if custom feature gates are supplied in `eksctl`, `featureGates.RotateKubeletServerCertificate=true` should still be included unless you intentionally disable it.

The Kubernetes Certificates API automates X.509 provisioning through CSRs, supports server auth certificates, and uses a one-year default and maximum certificate lifetime for that signer profile. In addition, client certificate signing is not supported, which helps distinguish this control from kubelet client certificate rotation. When both settings are true, the kubelet is configured to bootstrap into a CSR-based serving-certificate lifecycle and to keep the server-certificate rotation feature available for that lifecycle. If `serverTLSBootstrap` is `false`, the kubelet does not enter that bootstrap flow. If `RotateKubeletServerCertificate` is not enabled when custom feature gates are managed, the intended serving-certificate automation can be weakened or disabled.

Impact:

Enabling both settings helps keep the kubelet HTTPS endpoint on an automated serving-certificate lifecycle instead of depending on a static or self-managed certificate path. In practical terms, that improves continuity for long-lived nodes because the certificate workflow is bootstrapped and rotation support remains available. If either setting is not enabled, the intended serving-certificate automation is incomplete, which can leave the kubelet endpoint relying on a less durable certificate management approach.

Audit:

To audit whether kubelet serving certificate bootstrap and server certificate rotation are enabled on an Amazon EKS worker node, query the live effective kubelet configuration through the Kubernetes API server proxy and inspect the kubelet `configz` output for both `serverTLSBootstrap` and `featureGates.RotateKubeletServerCertificate`. Kubernetes identifies `configz` as the view of the kubelet's final actuated configuration. AWS documents that `serverTLSBootstrap` is a managed kubelet field in eksctl, and also warns that when custom featureGates are supplied, `RotateKubeletServerCertificate` can be unset unless you include it explicitly.

Using the api configz endpoint search for the status of `serverTLSBootstrap` and `RotateKubeletServerCertificate` by extracting the live configuration from the nodes running kubelet.

Set the local proxy port and the following variables and provide proxy port number and node name; `HOSTNAME_PORT="localhost-and-port-number"` `NODE_NAME="The-Name-Of-Node-To-Extract-Configuration"` from the output of `kubectl get nodes`

```
kubectl proxy --port=8001 &

export HOSTNAME_PORT=localhost:8001 (example host and port number)
export NODE_NAME=ip-192-168-20-214.us-west-2.compute.internal (example node
name from "kubectl get nodes")

curl -sSL "http://${HOSTNAME_PORT}/api/v1/nodes/${NODE_NAME}/proxy/configz"

or

curl -sSL "http://${HOSTNAME_PORT}/api/v1/nodes/${NODE_NAME}/proxy/configz" |
jq (for a more readable output)
```

Output should show something similar to `"serverTLSBootstrap":true` and `"RotateKubeletServerCertificate":true` in the output

To interpret the result:

- `serverTLSBootstrap=true` and `RotateKubeletServerCertificate=true` = Pass. The kubelet is configured to bootstrap its serving certificate through the Kubernetes certificates API, and the related rotation support is enabled.

- `serverTLSBootstrap=true` and `RotateKubeletServerCertificate=false` = Fail. Kubernetes says the feature must be enabled when `serverTLSBootstrap` is set, and AWS warns this gate can become unset when custom feature gates are provided.
- `serverTLSBootstrap=false` and `RotateKubeletServerCertificate=true` = Fail. Rotation support is available, but the kubelet is not configured to enter the serving-certificate bootstrap workflow. Kubernetes lists the default for `serverTLSBootstrap` as `false`.
- `serverTLSBootstrap=false` and `RotateKubeletServerCertificate=false` = Fail. Neither the bootstrap setting nor the related rotation support is enabled.
- `serverTLSBootstrap` is `null` or `missing` = Treat as not enabled unless you have another authoritative source showing it is being set outside the visible kubelet config. Kubernetes lists the default as `false`.
- `RotateKubeletServerCertificate` is `null` or `missing` = Investigate further. In current Kubernetes, the feature may be enabled by default, but AWS specifically says that if you provide custom featureGates in `eksctl`, you should explicitly include `RotateKubeletServerCertificate=true` or it will be unset.

Remediation:

If the audit shows `serverTLSBootstrap` is `false`, the kubelet is not entering the Kubernetes certificates API workflow for its serving certificate and the node should be treated as **noncompliant**. If `RotateKubeletServerCertificate` is `false`, or becomes unset because custom feature gates were supplied without re-including it, the related server certificate rotation support is not being preserved. If either field is missing, first confirm the configz response is complete, because Kubernetes identifies that endpoint as the view of the kubelet's final actuated configuration. In EKS, remediation should be performed through the supported node provisioning or node update workflow, not by directly editing the kubelet config file on the host, because AWS applies durable node behavior through launch templates, merged user data, and managed node replacement workflows. AWS also documents that `serverTLSBootstrap` is a managed kubelet field in `eksctl`, and warns that when custom `featureGates` are provided you should explicitly include `RotateKubeletServerCertificate=true` or it will be unset.

Remediation of new cluster

When creating a new EKS cluster or node group, the goal is to ensure the kubelet starts with serving certificate bootstrap enabled and the related rotation support preserved. As the two settings work together, `serverTLSBootstrap=true` places the kubelet on the CSR-based serving-certificate workflow, while `RotateKubeletServerCertificate=true` keeps the required rotation feature enabled when feature gates are managed. AWS documents launch templates as the supported path for deeper managed node customization.

- Set `serverTLSBootstrap: true` in the worker node kubelet configuration.
- Ensure `featureGates.RotateKubeletServerCertificate: true` is included, especially if custom feature gates are supplied.

- Apply both settings through the supported bootstrap path, such as your launch-template-based node configuration, rather than by changing the host after deployment.
- Verify the live kubelet configuration shows both settings enabled.

Remediation of existing running cluster

For an existing EKS cluster, the objective is to update the node launch configuration so replacement nodes start with both serving certificate bootstrap and rotation support enabled, then replace the current nodes so the corrected state is applied consistently across the node group. AWS documents that when a managed node group uses your own launch template, updating to a new version of that same launch template recycles all nodes to match the new configuration.

- Update the launch template or equivalent bootstrap configuration so `serverTLSBootstrap: true` is applied.
- Add or preserve `featureGates.RotateKubeletServerCertificate: true` if custom feature gates are present.
- Roll the managed node group so replacement nodes launch with the corrected settings.
- If the node group was not created from your own launch template, create a new node group with the required configuration and migrate workloads, because AWS says existing node groups without a custom launch template cannot be updated directly that way.
- Re-audit replacement nodes to confirm both settings are enabled in the live kubelet configuration.

Note: Even though it is tempting to do so, direct edits to the kubelet config file on EKS worker nodes are discouraged because Amazon EKS merges required user data for managed nodes, says not to include commands that start or modify kubelet in that merged user data path, and applies durable configuration changes through launch-template updates and node replacement. A host-level change can therefore be overridden and lost when the node is recycled or replaced.

Default Value:

serverTLSBootstrap and RotateKubeletServerCertificate are enabled, set to true, by default

References:

1. https://docs.aws.amazon.com/eks/latest/eksctl/customizing-the-kubelet.html
2. https://docs.aws.amazon.com/eks/latest/userguide/hybrid-nodes-bottlerocket.html
3. https://docs.aws.amazon.com/eks/latest/userguide/launch-templates.html
4. https://docs.aws.amazon.com/eks/latest/userguide/update-managed-node-group.html

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	3.10 Encrypt Sensitive Data in Transit Encrypt sensitive data in transit. Example implementations can include: Transport Layer Security (TLS) and Open Secure Shell (OpenSSH).		●	●
v7	14.4 Encrypt All Sensitive Information in Transit Encrypt all sensitive information in transit.		●	●

4 Policies

This section contains recommendations for various Kubernetes policies which are important to the security of Amazon EKS customer environment.

4.1 RBAC and Service Accounts

4.1.1 Ensure that the cluster-admin role is only used where required (Manual)

Profile Applicability:

- Level 1

Description:

Verify that cluster-wide administrative access in Amazon EKS is granted only to identities with a documented need for full cluster administration. In EKS, this review should cover access entries associated with `AmazonEKSClusterAdminPolicy`, legacy `aws-auth` and Kubernetes group mappings that lead to `ClusterRoleBinding` assignments, and any service accounts bound to cluster-wide administrative permissions. Where administration is limited to a team, application, or namespace boundary, use a narrower authorization model instead of cluster-wide admin access.

Rationale:

This setting enforces least privilege across the most sensitive EKS authorization paths. AWS states that access entries are the best way to grant users access to the Kubernetes API and that they support granular permission assignment, including modifying or revoking `cluster-admin` access from the original cluster creator. AWS also distinguishes cluster-wide administration through `AmazonEKSClusterAdminPolicy` from the more limited `AmazonEKSAdminPolicy`, which is typically scoped to one or more namespaces. In parallel, AWS notes that the IAM principal that creates the cluster is automatically granted `system:masters` permissions, and its RBAC hardening guidance warns that DaemonSet and CSI driver service account permissions can be effectively exposed from every node in the cluster. Together, these EKS behaviors make it important to keep cluster-admin use exceptional, tightly bounded, and reviewable.

Impact:

If cluster-admin is assigned too broadly in EKS, a compromised IAM principal or overprivileged service account can gain effective control over resources across all namespaces, increasing the chance that a single credential exposure or node-level compromise becomes a cluster-wide security or availability incident. The risk is greater when the affected workload also has AWS IAM permissions through IRSA or Pod Identity, because exposure can extend beyond Kubernetes RBAC into AWS API access.

Audit:

This audit reviews Kubernetes `ClusterRoleBinding` objects in the Amazon EKS cluster to identify principals that are directly granted the `cluster-admin` `ClusterRole`. It lists each matching binding and its associated users, groups, or service accounts so the reviewer can verify whether full cluster-wide administrative access is justified. Because EKS can also grant Kubernetes API access through access entries, access policies, and legacy IAM-to-Kubernetes group mappings, this check should be used as a partial review of administrative privilege rather than the sole test of cluster-admin exposure.

```
kubectl get clusterrolebindings -o json | jq -r '
  (["BINDING","KIND","NAMESPACE","NAME"] | join(" | ")),
  (
    .items[]
    | select(.roleRef.name=="cluster-admin")
    | .metadata.name as $binding
    | (.subjects // [])[]
    | [$binding, .kind, (.namespace // "-"), .name]
    | join(" | ")
  )
'
```

Example output with headers and piped separation of values. Review each principal listed and ensure that `cluster-admin` privilege is required for it.

```
BINDING | KIND | NAMESPACE | NAME
cluster-admin | Group | - | system:masters
my-admin-binding | User | - | admin-user
ci-cluster-admin | ServiceAccount | cicd | deploy-bot
eks:addon-cluster-admin | User | - | eks:addon-manager
```

Remediation:

Review each `ClusterRoleBinding` identified by the audit that grants the Kubernetes `cluster-admin` `ClusterRole`, and determine whether the listed user, group, or service account still requires full cluster-wide administrative access. In Amazon EKS, this review should also account for access granted through EKS access entries and access policies, because AWS recommends access entries as the preferred way to grant Kubernetes API access. Where responsibilities are limited to a namespace, workload, or operational function, replace cluster-wide administration with a narrower authorization model.

- Review each principal returned by the audit and confirm whether full cluster-wide administrative access is still justified for that identity.
- For IAM principals managed through EKS access entries, remove or downgrade any association to `AmazonEKSClusterAdminPolicy` and use a narrower access policy where possible, such as `AmazonEKSAdminPolicy` with namespace scope.
- For principals granted access through Kubernetes RBAC, replace unnecessary cluster-admin bindings with a lower-privilege `Role`, `ClusterRole`, or namespace-scoped `RoleBinding` that matches the approved operational need.

- Validate that the replacement access allows the user, group, or service account to perform its approved tasks before removing the original `cluster-admin` binding.
- Remove the unneeded `ClusterRoleBinding` only after the reduced-privilege access has been confirmed to work as intended.
- Do not remove EKS-managed bindings such as `eks:addon-cluster-admin` unless you intentionally want Amazon EKS to stop managing add-ons in the cluster.
- Review the original cluster creator separately, because AWS notes that this principal is automatically granted elevated cluster access and should not remain the default path for routine administration.

Where possible, first assign a lower-privilege role, validate required access, and then remove the `ClusterRoleBinding` that grants `cluster-admin`:

```
kubectl delete clusterrolebinding [name]
```

Default Value:

By default, `cluster-admin` is bound to `system:masters`, and the original cluster creator in EKS also has `system:masters` permissions.

References:

1. https://docs.aws.amazon.com/eks/latest/best-practices/identity-and-access-management.html
2. https://docs.aws.amazon.com/eks/latest/userguide/access-entries.html
3. https://docs.aws.amazon.com/eks/latest/userguide/access-policies.html
4. https://docs.aws.amazon.com/eks/latest/userguide/auth-configmap.html

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	5.4 Restrict Administrator Privileges to Dedicated Administrator Accounts Restrict administrator privileges to dedicated administrator accounts on enterprise assets. Conduct general computing activities, such as internet browsing, email, and productivity suite use, from the user's primary, non-privileged account.	●	●	●
v7	4.3 Ensure the Use of Dedicated Administrative Accounts Ensure that all users with administrative account access use a dedicated or secondary account for elevated activities. This account should only be used for administrative activities and not internet browsing, email, or similar activities.	●	●	●

4.1.2 Ensure access to Secrets is minimized and granted only where required (Manual)

Profile Applicability:

- Level 1

Description:

Verify that access to Kubernetes `Secret` objects in EKS is granted only to IAM principals, Kubernetes users or groups, and service accounts with a defined operational or workload need. The review should cover EKS access entries and associated access policies, Kubernetes RBAC roles and bindings, and service account permissions, with particular attention to permissions that allow `get`, `list`, or `watch` access to Secrets. Where access is needed, scope it as narrowly as possible, because EKS access policies can be limited to specific namespaces instead of the entire cluster.

Rationale:

This requirement reduces unnecessary exposure to one of the most sensitive resource types in an EKS cluster. `AmazonEKSAdminViewPolicy` allows an IAM principal to list and view all cluster resources, and explicitly notes that this includes Kubernetes Secrets. Access policies attached to EKS access entries can be scoped to all resources in the cluster or to specific namespaces, and that when an access entry also uses Kubernetes group names, the principal receives the combined permissions from both the associated access policies and any Kubernetes RBAC objects bound to those groups. Minimizing Secrets access therefore helps prevent broad read access from being granted through overlapping EKS and Kubernetes authorization paths.

For workloads, least-privilege access should be applied to both Kubernetes service accounts and AWS IAM roles. IAM roles for service accounts help isolate credentials and scope AWS access, but unauthorized node access can expose projected service account tokens and, when Pod Identity or IRSA is used, may also allow retrieval of AWS IAM credentials. Excessive Secrets access combined with overprivileged workload identities can then expand a pod or node compromise beyond Kubernetes RBAC into AWS API access.

Impact:

If access to Secrets is granted too broadly in EKS, a compromised IAM principal, Kubernetes subject, or service account can retrieve sensitive cluster data and use it to expand the scope of an incident across namespaces or into AWS services when workload IAM credentials are also available. That increases the likelihood that a single credential exposure, pod compromise, or node-level breach becomes a wider security event.

Audit:

Start by reviewing every path that can grant access to Kubernetes `Secret` objects in the EKS cluster. Start with EKS access entries and associated access policies, because EKS policies can grant Kubernetes permissions without native RBAC objects, and `AmazonEKSAdminViewPolicy` includes permission to view Kubernetes Secrets. Then review Kubernetes RBAC, service account permissions, and any remaining legacy `aws-auth` mappings, because EKS can authorize access through multiple overlapping models and policy scope can be either cluster-wide or namespace-scoped.

- Review EKS access entries and associated access policies for IAM principals that can access the Kubernetes API, and identify any policy scope broad enough to expose Secrets. Check whether access is cluster-wide or limited to specific namespaces.
- Review Kubernetes RBAC roles and bindings that grant `get`, `list`, or `watch` on `secrets`, and determine which users, groups, and service accounts actually receive that access. Give extra scrutiny to cluster-scoped roles and broadly shared bindings.
- Review workload service accounts that can access Secrets and confirm that each one has a defined workload need. Pay particular attention to DaemonSets, CSI drivers, and other broadly deployed components because node compromise can expose projected service account tokens and related permissions.
- Review any remaining legacy `aws-auth` mappings, because they can still place IAM principals into Kubernetes groups that later receive Secrets access through RBAC bindings.
- Confirm that each access path is tied to a documented requirement, scoped to the smallest practical namespace or function, and no broader than necessary. Flag any unnecessary Secrets access through access policies, RBAC, or service accounts for remediation.

Summary: Review EKS access entries, access policies, RBAC, and service accounts to verify that Secrets access is approved, minimally scoped, and granted only where required.

Remediation:

Start by reducing every access path that allows reading Kubernetes `Secret` objects in the EKS cluster. Start with EKS access entries and associated access policies, because EKS policies can grant Secrets access without native RBAC objects, and policies such as `AmazonEKSAdminViewPolicy`, `AmazonEKSSecretReaderPolicy`, and broader administrative policies can expose Secrets depending on scope. Then reduce unnecessary access in Kubernetes RBAC, service accounts, and any remaining legacy `aws-auth` mappings, and replace broad permissions with the narrowest namespace or workload scope that still supports the required function.

- Remove or narrow EKS access policy associations that expose Secrets, and prefer namespace-scoped access over cluster-wide scope wherever possible.

- Replace broad RBAC permissions on `secrets` with the smallest practical `Role` or `ClusterRole`, and remove unnecessary `get`, `list`, or `watch` access from users, groups, and service accounts.
- Reduce service account access to Secrets to only the workloads that require it, and tighten any related IRSA or Pod Identity IAM permissions to least privilege.
- Review DaemonSet and CSI driver service accounts first, because their tokens and permissions can be exposed from any node where those pods run.
- Migrate remaining legacy `aws-auth` mappings to access entries where appropriate, and remove stale mappings that still place IAM principals into Kubernetes groups with Secrets access.
- Validate that each remaining Secrets access path is tied to a documented operational or workload requirement, and flag any unnecessary access for removal.

Default Value:

By default, EKS creates platform identities for cluster components, but access to Secrets is controlled through associated access policies and Kubernetes RBAC bindings rather than a single cluster setting.

References:

1. https://docs.aws.amazon.com/eks/latest/userguide/access-policy-permissions.html
2. https://docs.aws.amazon.com/eks/latest/userguide/access-policies.html
3. https://docs.aws.amazon.com/eks/latest/userguide/iam-roles-for-service-accounts.html
4. https://docs.aws.amazon.com/eks/latest/userguide/rbac-hardening.html

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	4.1 Establish and Maintain a Secure Configuration Process Establish and maintain a secure configuration process for enterprise assets (end-user devices, including portable and mobile, non-computing/IoT devices, and servers) and software (operating systems and applications). Review and update documentation annually, or when significant enterprise changes occur that could impact this Safeguard.	●	●	●
v7	5.2 Maintain Secure Images Maintain secure images or templates for all systems in the enterprise based on the organization's approved configuration standards. Any new system deployment or existing system that becomes compromised should be imaged using one of those images or templates.		●	●

4.1.3 Minimize wildcard use in Roles and ClusterRoles (Manual)

Profile Applicability:

- Level 1

Description:

Verify that Kubernetes `Role` and `ClusterRole` objects in EKS do not use wildcard entries such as `*` for `verbs`, `resources`, or `apiGroups` unless a documented administrative or platform requirement exists. The review should also include the `RoleBinding` and `ClusterRoleBinding` objects that assign those roles to users, groups, or service accounts, because EKS can authorize access through both access entries and native Kubernetes RBAC, and principals can receive the combined permissions of associated access policies and RBAC bindings.

Rationale:

This recommendation supports least-privilege authorization in the parts of EKS that still rely on Kubernetes RBAC. EKS guidance recommends access entries as the best way to grant users access to the Kubernetes API and highlights fine-grained permissions management, but it also makes clear that when predefined access policies do not meet a requirement, you should use Kubernetes groups and create your own `Role` and `ClusterRole` objects. EKS further notes that if an access entry has both associated access policies and Kubernetes group mappings, the IAM principal receives all permissions from both paths. In that model, wildcard rules make the effective privilege set harder to predict, harder to review, and easier to overextend across current and future resources. The risk is higher for service accounts used by DaemonSets and CSI drivers, because EKS RBAC hardening guidance treats those Kubernetes RBAC permissions as effectively reachable from every node where those pods run.

Impact:

If wildcard permissions are used broadly in EKS, a user or service account can receive more access than intended across namespaces, resource types, or API actions, and that excess access can combine with other policy associations or workload credentials to widen the scope of compromise. When the affected identity is a broadly deployed service account or one linked to IRSA or Pod Identity, an overbroad RBAC rule can turn a single pod or node issue into unauthorized cluster changes or access to AWS APIs beyond the workload's intended boundary.

Audit:

Begin by reviewing custom Kubernetes `Role` and `ClusterRole` objects in EKS to identify any use of wildcard entries such as `*` for `verbs`, `resources`, or `apiGroups`. In EKS, this review should cover native RBAC objects, access entries that reference Kubernetes groups, service accounts, and any remaining `aws-auth` mappings, because EKS supports access through both access entries and Kubernetes RBAC, and it also creates its own platform RBAC objects that should be distinguished from customer-defined permissions.

- Review custom `Role` and `ClusterRole` definitions for wildcard use in `verbs`, `resources`, or `apiGroups`, and prioritize cluster-scoped roles because `ClusterRoles` apply across the cluster.

```
kubectl get roles,clusterroles -A -o json | jq -r '
  (["KIND","NAMESPACE","NAME","WILDCARD_FIELDS"] | join(" | ")),
  (
    .items[]
    | . as $r
    | [
        (if any(.rules[]?; any(.verbs[]?; .=="*")) then "verbs" else empty
end),
        (if any(.rules[]?; any(.resources[]?; .=="*")) then "resources" else
empty end),
        (if any(.rules[]?; any(.apiGroups[]?; .=="*")) then "apiGroups" else
empty end)
      ] as $wild
    | select(($wild | length) > 0)
    | [
        $r.kind,
        ($r.metadata.namespace // "-"),
        $r.metadata.name,
        ($wild | join(","))
      ]
    | join(" | ")
  )
'
```

Sample Output:

```
KIND | NAMESPACE | NAME | WILDCARD_FIELDS
ClusterRole | - | cluster-admin | verbs,resources,apiGroups
ClusterRole | - | eks:addon-manager | resources
ClusterRole | - | eks:service-operations | resources
ClusterRole | - | system:controller:disruption-controller | apiGroups
ClusterRole | - | system:controller:generic-garbage-collector |
resources,apiGroups
ClusterRole | - | system:controller:horizontal-pod-autoscaler |
resources,apiGroups
ClusterRole | - | system:controller:namespace-controller |
resources,apiGroups
ClusterRole | - | system:controller:resourcequota-controller |
resources,apiGroups
ClusterRole | - | system:kube-controller-manager | resources,apiGroups
ClusterRole | - | system:kubelet-api-admin | verbs
```

- Review the `RoleBinding` and `ClusterRoleBinding` assignments for those roles to identify which users, groups, and service accounts actually receive the wildcard permissions, and separate EKS-created bindings from customer-managed ones during the review.
- Review access entries that reference Kubernetes groups, because EKS allows permissions to be attached through access policies or through Kubernetes groups backed by RBAC objects, which means wildcard roles can still be granted through group-based authorization paths.
- Review service accounts for DaemonSets, CSI drivers, and other broadly deployed controllers first, because EKS RBAC hardening guidance specifically calls for reviewing the ClusterRoles bound to those service accounts and removing permissions that are not required.
- Review any remaining legacy `aws-auth` mappings, because IAM principals can still be mapped into Kubernetes groups that may be bound to wildcard roles.
- Confirm that each wildcard rule has a documented operational or platform need. Where it does not, flag it for remediation and replace it with enumerated resources, verbs, API groups, or narrower namespace-scoped roles. EKS guidance favors fine-grained permissions and least-privilege review of RBAC assignments.

Summary: Review custom RBAC roles, bindings, group mappings, and service accounts to verify that wildcard permissions are rare, justified, and no broader than required.

Remediation:

Start by replacing wildcard permissions in custom `Role` and `ClusterRole` objects with explicitly enumerated `verbs`, `resources`, and `apiGroups` that match the approved operational need. In EKS, this remediation should focus on customer-managed RBAC objects, the bindings that assign them, access entries that reference Kubernetes groups, and service accounts used by workloads or cluster components. EKS guidance favors fine-grained access through access entries and least-privilege review of RBAC permissions, especially for service accounts used by add-ons, DaemonSets, and CSI drivers.

- Replace wildcard values in custom `Role` and `ClusterRole` rules with the specific verbs, resources, and apiGroups the function actually requires.
- Reduce or remove any RoleBinding and ClusterRoleBinding that assign those wildcard roles more broadly than necessary, and prefer namespace-scoped bindings where possible.
- Review access entries that use Kubernetes groups, because those groups can still receive wildcard permissions through native RBAC even when IAM authentication is managed through EKS.
- Review service accounts for DaemonSets, CSI drivers, and other broadly deployed controllers first, and remove wildcard permissions that are not required for their runtime behavior.
- Distinguish customer-managed roles from EKS-created platform roles before making changes, and avoid modifying EKS-managed RBAC objects unless you intentionally want to change how a platform component operates.
- After narrowing the role definition, validate that the user, group, or service account can still perform its approved tasks, then remove the superseded wildcard role or binding.

References:

1. https://docs.aws.amazon.com/eks/latest/userguide/rbac-hardening.html
2. https://docs.aws.amazon.com/eks/latest/userguide/access-entries.html
3. https://docs.aws.amazon.com/eks/latest/userguide/default-roles-users.html
4. https://docs.aws.amazon.com/eks/latest/best-practices/identity-and-access-management.html

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	5.2 Use Unique Passwords Use unique passwords for all enterprise assets. Best practice implementation includes, at a minimum, an 8-character password for accounts using MFA and a 14-character password for accounts not using MFA.	●	●	●

Controls Version	Control	IG 1	IG 2	IG 3
v7	4.4 Use Unique Passwords Where multi-factor authentication is not supported (such as local administrator, root, or service accounts), accounts will use passwords that are unique to that system.		●	●

4.1.4 Minimize access to create pods (Manual)

Profile Applicability:

- Level 1

Description:

The ability to create Pods in EKS is a high-risk permission that should be limited to approved deployment and operational functions. In EKS, Pod creation can be granted through associated access policies or native Kubernetes RBAC, and policies such as `AmazonEKSAdminPolicy` and `AmazonEKSEditPolicy` include create access on pods. Because Pods run within a namespace, use a service account, and can carry workload-level permissions, Pod creation should be scoped to the smallest practical namespace and workload boundary.

Rationale:

Creating a Pod can allow a principal to run code with the permissions of the service account assigned to that Pod. EKS Pods use Kubernetes service accounts, that dedicated service accounts should be used for applications, and that token auto-mounting should be disabled when Kubernetes API access is not needed. Those behaviors make Pod creation more than a routine write permission, because it can become a path to exercise Kubernetes API permissions indirectly through workload identity.

The risk can extend beyond Kubernetes RBAC into AWS access. With IRSA or EKS Pod Identity, a Kubernetes service account can be associated with an IAM role, allowing Pods to obtain AWS credentials for API calls. EKS pod security guidance also identifies higher-risk Pod behaviors such as privileged execution and `hostPath` usage, which means the ability to create Pods can also expose host-level or node-adjacent attack paths if security controls do not constrain the Pod spec.

Impact:

If Pod creation is granted too broadly in EKS, a user or workload can launch Pods under available service accounts and gain unintended Kubernetes or AWS access, increasing the risk of privilege escalation and wider cluster impact.

Audit:

Begin by reviewing every path that can grant the ability to create Pods in EKS. Start with access entries and associated access policies, because `AmazonEKSAdminPolicy` and `AmazonEKSEditPolicy` include create access for pods. Then review Kubernetes RBAC, service accounts, and Pod security controls, because Pod creation can allow an identity to launch workloads under available service accounts and potentially exercise both Kubernetes and AWS permissions.

- Review access entries and associated access policies for principals that can access the Kubernetes API, and identify any policy scope that allows create on pods. Check whether the scope is cluster-wide or namespace-scoped.
- Review `Role` and `ClusterRole` assignments that allow create on pods, and also review `deployments`, `daemonsets`, `statefulsets`, jobs, and `cronjobs`, because those resources can launch Pods through controllers. Map each role to the principals that receive it.
- Review service accounts used by workloads that can create or run Pods. Check for use of the default service account, and confirm whether token auto-mounting is disabled where Kubernetes API access is not needed.
- Review workloads that use IRSA or EKS Pod Identity, because a Pod created under a linked service account can obtain AWS credentials in addition to Kubernetes API access. Confirm the IAM role is limited to the intended workload scope.
- Review whether Pod creation can be paired with privileged execution, `hostPath`, or `hostNetwork`, and confirm that Pod Security Admission or policy-as-code controls restrict those settings where required.
- Review any remaining legacy `aws-auth` usage if the cluster still uses `CONFIG_MAP` or `API_AND_CONFIG_MAP`, because IAM principals can still reach Kubernetes authorization through older mappings.

Summary: Review access policies, RBAC, service accounts, and Pod controls to verify that Pod creation is tightly scoped and granted only where required.

Remediation:

Start by reducing every access path that allows Pod creation in EKS. Start with access entries and associated access policies, because `AmazonEKSAdminPolicy` and `AmazonEKSEditPolicy` include create access for pods, and access policies can be scoped to the cluster or to specific namespaces. Then reduce unnecessary permissions in Kubernetes RBAC, service accounts, and Pod security controls so Pod creation is limited to approved deployment paths and the smallest practical workload boundary.

- Remove or narrow access policy associations that allow Pod creation, and prefer namespace-scoped access over cluster-wide scope wherever possible.
- Replace broad RBAC permissions on `pods`, `deployments`, `daemonsets`, `statefulsets`, `jobs`, and `cronjobs` with the smallest `Role` or `ClusterRole` that supports the approved function.
- Move workloads off the default service account, use dedicated service accounts, and disable automountServiceAccountToken where Kubernetes API access is not needed.
- Tighten IRSA or EKS Pod Identity permissions for any service account that can run Pods so the linked IAM role is least privilege and workload-specific.
- Restrict Pod specs that can increase the blast radius of Pod creation, including privileged mode, `hostPath`, and similar settings, by using Pod Security Admission or policy-as-code controls.

- Review DaemonSet and other broadly deployed workloads first, because a service account used across many nodes can widen the impact of an overprivileged Pod creator.
- Remove stale legacy `aws-auth` mappings where applicable so Pod creation is not still reachable through older group mappings in parallel with access entries.

Default Value:

By default, the following list of principals have `create` privileges on `pod` objects

```
BINDING_KIND        BINDING_NS  BINDING_NAME
ROLE_KIND    ROLE_NAME                                  SUBJECT_KIND
ClusterRoleBinding  -           system:controller:daemon-set-controller
ClusterRole  system:controller:daemon-set-controller     ServiceAccount
ClusterRoleBinding  -           system:controller:job-controller
ClusterRole  system:controller:job-controller            ServiceAccount
ClusterRoleBinding  -           system:controller:persistent-volume-binder
ClusterRole  system:controller:persistent-volume-binder  ServiceAccount
ClusterRoleBinding  -           system:controller:replicaset-controller
ClusterRole  system:controller:replicaset-controller     ServiceAccount
ClusterRoleBinding  -           system:controller:replication-controller
ClusterRole  system:controller:replication-controller    ServiceAccount
ClusterRoleBinding  -           system:controller:statefulset-controller
ClusterRole  system:controller:statefulset-controller    ServiceAccount
```

References:

1. https://docs.aws.amazon.com/eks/latest/userguide/access-policy-permissions.html
2. https://docs.aws.amazon.com/eks/latest/userguide/access-entries.html
3. https://docs.aws.amazon.com/eks/latest/best-practices/identity-and-access-management.html
4. https://docs.aws.amazon.com/eks/latest/best-practices/pod-security.html

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	2.7 Allowlist Authorized Scripts Use technical controls, such as digital signatures and version control, to ensure that only authorized scripts, such as specific .ps1, .py, etc., files, are allowed to execute. Block unauthorized scripts from executing. Reassess bi-annually, or more frequently.			●
v7	4.7 Limit Access to Script Tools Limit access to scripting tools (such as Microsoft PowerShell and Python) to only administrative or development users with the need to access those capabilities.		●	●

4.1.5 Ensure that default service accounts are not actively used. (Automated)

Profile Applicability:

- Level 1

Description:

Default service accounts in EKS should not be used as the active identity for ordinary application or automation workloads. If a Pod does not specify a service account, Kubernetes automatically assigns the namespace default service account, and that account can authenticate to the Kubernetes API through its mounted token. Workloads that need Kubernetes or AWS access should instead use an explicitly assigned, dedicated service account that matches the intended namespace, application, and permission boundary.

Rationale:

This supports least privilege at the workload identity layer by using a dedicated service account for each application, including workloads that need Kubernetes API access, IRSA, or EKS Pod Identity. This allows RBAC and workload identity to be scoped to the specific resources and actions required by each application, instead of sharing one namespace-level identity across unrelated Pods.

The default service account is especially risky when Pods inherit it implicitly. In EKS, workloads are automatically assigned a service account if one is not specified, and service account tokens are automatically mounted unless token auto-mounting is disabled. For workloads that need AWS access, EKS Pod Identity and IRSA both bind AWS permissions to a Kubernetes service account, so relying on the default account makes it harder to isolate AWS permissions cleanly and increases the chance that multiple workloads share the same Kubernetes and AWS identity path.

Impact:

If the default service account is actively used in EKS, multiple Pods in the same namespace can inherit the same Kubernetes identity and any linked AWS permissions, which increases blast radius, weakens workload isolation, and reduces audit clarity when a Pod is compromised or misconfigured.

Audit:

Begin by identifying workloads in EKS that implicitly or explicitly use the namespace default service account. This matters because Pods that do not specify `serviceAccountName` are assigned the default service account, workloads that need AWS access should use a dedicated service account for IRSA or EKS Pod Identity, and workloads that do not need Kubernetes API access can disable token auto-mounting with `automountServiceAccountToken: false`.

- Review `Deployments`, `StatefulSets`, `DaemonSets`, `Jobs`, `CronJobs`, and standalone `Pods` for omitted `serviceAccountName` values or explicit use of `default`.

```
kubectl get deploy,sts,ds,job,cronjob,pod -A -o json | jq -r '
  (["NAMESPACE","KIND","NAME","SERVICEACCOUNT"] | @tsv),
  (
    .items[]
    | if .kind=="CronJob" then
        [.metadata.namespace, .kind, .metadata.name,
(.spec.jobTemplate.spec.template.spec.serviceAccountName // "default")]
      elif .kind=="Pod" then
        [.metadata.namespace, .kind, .metadata.name,
(.spec.serviceAccountName // "default")]
      else
        [.metadata.namespace, .kind, .metadata.name,
(.spec.template.spec.serviceAccountName // "default")]
      end
    | select(.[3]=="default")
    | @tsv
  )
' | column -ts $'\t'
```

Sample output:

```
NAMESPACE    KIND         NAME              SERVICEACCOUNT
kube-public  Pod          file-check        default
default      Deployment   web-app           default
apps         StatefulSet  orders-db         default
logging      DaemonSet    fluentd           default
batch        Job          nightly-report    default
```

- Review the default service account in each namespace and confirm it is not bound to unnecessary `RoleBinding` or `ClusterRoleBinding` permissions.
- Review workload specs for `automountServiceAccountToken: false` and confirm it is set wherever the workload does not need Kubernetes API access. Use the command below to find the `automountServiceAccountToken setting`:

```
kubectl get serviceaccount $SERVICE_ACCOUNT -n $NAMESPACE -o yaml
```

- Review workloads that use IRSA or EKS Pod Identity and confirm they use a dedicated service account rather than the namespace default service account.
- Flag any workload that still uses the default service account or still auto-mounts tokens without a documented need for remediation.

Remediation:

Create dedicated service accounts for workloads that need Kubernetes API access, IRSA, or EKS Pod Identity, and update those workloads to explicitly set `serviceAccountNam` rather than relying on the namespace `default` service account. For workloads that do not need Kubernetes API access, disable service account token auto-mounting and avoid using the `default` service account as the active workload identity. Also review the `default` service account in each namespace and ensure it is not bound to unnecessary RBAC permissions.

Modify the `default` service account in each application namespace to set:

```
automountServiceAccountToken: false
```

Then validate workload manifests, because a Pod-level `automountServiceAccountToken` setting can override the service account setting, and workloads that actually need Kubernetes API access must use an explicitly assigned service account instead of inheriting `default`.

Automatic remediation for one namespace:

```
kubectl patch serviceaccount default -n <namespace> -p
'{"automountServiceAccountToken": false}'
```

Default Value:

By default the `default` service account allows for its service account token to be mounted in pods in its namespace.

References:

1. https://docs.aws.amazon.com/eks/latest/best-practices/identity-and-access-management.html
2. https://docs.aws.amazon.com/eks/latest/userguide/service-accounts.html
3. https://docs.aws.amazon.com/eks/latest/userguide/pod-identities.html
4. https://docs.aws.amazon.com/eks/latest/best-practices/scale-workloads.html

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	5.3 Disable Dormant Accounts Delete or disable any dormant accounts after a period of 45 days of inactivity, where supported.	●	●	●
v7	4.3 Ensure the Use of Dedicated Administrative Accounts Ensure that all users with administrative account access use a dedicated or secondary account for elevated activities. This account should only be used for administrative activities and not internet browsing, email, or similar activities.	●	●	●

Controls Version	Control	IG 1	IG 2	IG 3
v7	16.9 Disable Dormant Accounts Automatically disable dormant accounts after a set period of inactivity.	●	●	●

4.1.6 Ensure that Service Account Tokens are only mounted where necessary (Automated)

Profile Applicability:

- Level 1

Description:

Service account tokens in EKS should be mounted only for Pods that require Kubernetes API access or an explicitly configured workload identity flow. EKS supports disabling automatic token mounting at the Pod or ServiceAccount level, and workloads that need AWS access through IRSA or EKS Pod Identity should use a dedicated service account rather than relying on broad default token mounting.

Rationale:

This setting is important because a mounted service account token is an authentication credential. EKS documentation notes that Pods can authenticate to the Kubernetes API server with an auto-mounted token, and AWS recommends setting `automountServiceAccountToken: false` for applications that do not need access to Kubernetes resources. EKS also allows this behavior to be controlled either on the Pod spec or on the ServiceAccount, which supports making token mounting an intentional workload decision instead of a default behavior.

The risk is higher when workload identity is involved. With IRSA, a Pod uses a web identity token associated with a Kubernetes service account to obtain AWS credentials, and with EKS Pod Identity, an IAM role is associated to a Kubernetes service account and credentials are then made available to the workload. AWS also documents that projected service account tokens can be exposed from node compromise scenarios, and if the service account is associated with an IAM role through IRSA or EKS Pod Identity, an exposed token can be used to obtain AWS IAM credentials.

Impact:

If service account tokens are mounted more broadly than necessary in EKS, more Pods than required receive credentials that can be used for Kubernetes API access and, in some cases, AWS IAM access, increasing the blast radius of a compromised or misconfigured workload.

Audit:

Begin by reviewing where service account tokens are mounted in EKS and whether each mounted token supports a defined Kubernetes API or workload identity requirement. In EKS, Pods can receive a service account token automatically, `automountServiceAccountToken` can be disabled at the Pod or `ServiceAccount` level, and workloads that need AWS access through IRSA or EKS Pod Identity should use a dedicated service account instead of broad default token mounting.

- Review `Deployments`, `StatefulSets`, `DaemonSets`, `Jobs`, `CronJobs`, and standalone `Pods` for `serviceAccountName` and `automountServiceAccountToken`, and identify workloads that still rely on default token mounting.
- Review the `automountServiceAccountToken` setting on both the `Pod` and its associated `ServiceAccount`, and treat the Pod setting as authoritative where both are defined.

```
kubectl get serviceaccount $SERVICE_ACCOUNT -n $NAMESPACE -o yaml

kubectl get pod $POD_NAME -n $NAMESPACE -o yaml
```

Expected Output:

```
automountServiceAccountToken: false
```

- Review workloads that use IRSA or EKS Pod Identity and confirm they use a dedicated service account tied to the intended application and permission boundary.
- Review `RoleBindings` and `ClusterRoleBindings` for service accounts that still mount tokens, and confirm the bound RBAC permissions are limited to the API resources and actions the workload actually needs.
- Review `DaemonSets`, `CSI drivers`, and other broadly deployed Pods first, because service account permissions and projected tokens associated with those workloads can become reachable from any node where they run.
- Review Pods that do not need Kubernetes API access and confirm token mounting is disabled, then flag any unnecessary token mount for remediation.

Remediation:

Apply the following steps to reduce unnecessary service account token mounting in EKS:

- Set `automountServiceAccountToken` to `false` for `Pods` and `ServiceAccounts` that do not require Kubernetes API access. This reduces unnecessary default token auto-mounting.
- Update workloads to use a dedicated `serviceAccountName` only where a service account token is operationally required, instead of inheriting token behavior from the namespace `default` service account. Each application should use its own dedicated service account.
- For workloads that need AWS access, use IRSA or EKS Pod Identity with a dedicated service account rather than broad default token mounting. Both models associate AWS permissions to a Kubernetes service account for a specific workload boundary.
- Revalidate workloads after the change to confirm that only Pods with a documented Kubernetes API or workload identity need continue to receive service account tokens.

Default Value:

By default, all pods get a service account token mounted in them.

References:

1. https://docs.aws.amazon.com/eks/latest/best-practices/identity-and-access-management.html
2. https://docs.aws.amazon.com/eks/latest/userguide/iam-roles-for-service-accounts.html
3. https://docs.aws.amazon.com/eks/latest/userguide/pod-identities.html
4. https://docs.aws.amazon.com/eks/latest/userguide/service-accounts.html

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	4.8 Uninstall or Disable Unnecessary Services on Enterprise Assets and Software Uninstall or disable unnecessary services on enterprise assets and software, such as an unused file sharing service, web application module, or service function.		●	●
v7	14.7 Enforce Access Control to Data through Automated Tools Use an automated tool, such as host-based Data Loss Prevention, to enforce access controls to data even when data is copied off a system.			●

4.1.7 Standardize EKS cluster access by using access entries, policies, and scoped permissions (Automated)

Profile Applicability:

- Level 1

Description:

EKS provides the Cluster Access Management API, access entries, and associated access policies as the recommended way to manage IAM principal access to the Kubernetes API. This model is preferred over the deprecated `aws-auth` ConfigMap for cluster access management, supports cluster-wide or namespace-scoped permissions, and can also reference Kubernetes groups when native `Role`, `ClusterRole`, `RoleBinding`, or `ClusterRoleBinding` objects are needed for custom authorization. This access model applies to IAM users and roles that reach the cluster API, while Pod workload identity remains a separate service-account-based mechanism such as IRSA or EKS Pod Identity.

Rationale:

This approach improves access control consistency because EKS access entries are managed through the EKS API rather than through manual `aws-auth` ConfigMap edits, and AWS identifies access entries as the best way to grant IAM principals access to the Kubernetes API. An access entry associates an IAM user or role with Kubernetes permissions in one of two ways: by attaching one or more associated access policies, or by specifying Kubernetes group names that are then authorized through native Kubernetes RBAC objects. The AWS-managed access policies contain Kubernetes permissions, not IAM permissions, and each policy association can be scoped either to the entire cluster or to one or more Kubernetes namespaces, including wildcard namespace patterns.

This model also makes effective permissions easier to reason about in mixed authorization paths. A single access entry can have multiple associated access policies, and if Kubernetes groups are also specified, the IAM principal receives the union of all permissions granted by those policies plus any permissions granted through `Role`, `ClusterRole`, `RoleBinding`, or `ClusterRoleBinding` objects bound to those groups. AWS also notes that permissions granted by associated access policies are not shown by `kubectl auth can-i --list`, which means standardizing on access entries, explicit policy associations, and namespace-scoped assignments helps reduce hidden overpermissioning, authorization drift, and dependence on deprecated `aws-auth` mappings.

Impact:

If cluster access is not standardized in EKS, organizations can accumulate a mix of deprecated `aws-auth` mappings, uneven access-entry adoption, and overly broad cluster-scoped permissions that are harder to review, audit, and narrow to namespace scope. That increases the chance that an IAM principal receives duplicated or broader Kubernetes access than intended, and it makes access changes more error-prone over time.

Audit:

Review the cluster authentication mode to confirm EKS access entries are enabled, then review access entries and associated access policies to verify that cluster access is being managed through the Cluster Access Management API with cluster-wide or namespace-scoped permissions as appropriate. Treat API and `API_AND_CONFIG_MAP` as evidence that access entries are available, but treat `CONFIG_MAP` as legacy-only authentication. If the cluster is in `API_AND_CONFIG_MAP` mode, also review whether legacy `aws-auth` mappings are still present and whether access has been migrated to access entries and scoped policy associations.

To check if the Cluster Access Manager API is active on your Amazon EKS cluster, you can use the following AWS CLI command:

```
aws eks describe-cluster \
  --name $CLUSTER_NAME \
  --query "cluster.accessConfig" \
  --output json
```

The command queries the `cluster.accessConfig` property, which indicates the authentication mode of the cluster.

Expected output for `authenticationMode`:

- **API** = access entries enabled, ConfigMap auth removed
- **API_AND_CONFIG_MAP** = access entries enabled, legacy ConfigMap auth still also available
- **CONFIG_MAP** = legacy aws-auth only, no access-entry-based access management

Additional checks:

Lists the IAM principal ARNs that currently have EKS access entries on the cluster. Use it to identify which users or roles are being managed through the EKS access entry model.

```
aws eks list-access-entries --cluster-name $CLUSTER_NAME
```

Lists the EKS access policies associated with one specific access entry and shows the access scope, such as cluster-wide or namespace-scoped. Use it to identify what level of Kubernetes access that principal has been granted.

Command line statement to extract arn for $PRINCIPAL_ARN variable:

```
aws sts get-caller-identity --query Arn --output text
aws eks list-associated-access-policies \
  --cluster-name $CLUSTER_NAME \
  --principal-arn $PRINCIPAL_ARN
```

Remediation:

Log in to the AWS Management Console, navigate Amazon Elastic Kubernetes Service > Clusters and click on your EKS cluster.

Go to the **Access** tab and click on **Manage** in the **Access Configuration** section.

Under Cluster Authentication Mode for Cluster Access settings choose one of the following.

- **EKS API** *The cluster will source authenticated IAM principals only from EKS access entry APIs*
- **EKS API and ConfigMap** *The cluster will source authenticated IAM principals from both EKS access entry APIs and the aws-auth ConfigMap*

Default Value:

`EKS API` is selected by default during EKS Cluster creation but can be changed during initial configuration

References:

1. https://docs.aws.amazon.com/eks/latest/userguide/access-entries.html
2. https://docs.aws.amazon.com/eks/latest/userguide/access-policies.html
3. https://docs.aws.amazon.com/eks/latest/best-practices/cluster-access-management.html
4. https://docs.aws.amazon.com/eks/latest/userguide/setting-up-access-entries.html

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	4.8 Uninstall or Disable Unnecessary Services on Enterprise Assets and Software Uninstall or disable unnecessary services on enterprise assets and software, such as an unused file sharing service, web application module, or service function.		●	●
v7	14.7 Enforce Access Control to Data through Automated Tools Use an automated tool, such as host-based Data Loss Prevention, to enforce access controls to data even when data is copied off a system.			●

4.1.8 Limit use of the Bind, Impersonate and Escalate permissions in the Kubernetes cluster (Manual)

Profile Applicability:

- Level 1

Description:

In EKS, `bind`, `impersonate`, and `escalate` should be limited to identities with a documented administrative need, because these permissions belong to native Kubernetes RBAC and affect who can obtain or act with Kubernetes permissions. This review should cover custom `Role` and `ClusterRole` definitions, `RoleBinding` and `ClusterRoleBinding` assignments, and any service accounts that receive those permissions, especially for broadly deployed components such as DaemonSets or CSI drivers. AWS recommends using access entries and associated access policies for most cluster access and using native Kubernetes RBAC only when the predefined access policies do not meet the requirement.

Rationale:

This setting is important because EKS supports two authorization paths for IAM principals that reach the Kubernetes API: associated access policies and native Kubernetes RBAC through Kubernetes groups specified on an access entry. Access policies are AWS-managed Kubernetes permission sets, cannot be modified, and should be used for most common access patterns. When they do not meet a requirement, administrators must fall back to custom Kubernetes RBAC objects, which is where powerful verbs such as `bind`, `impersonate`, and `escalate` can appear and where least-privilege review becomes especially important.

The risk is higher because these permissions operate on the authorization model itself rather than on routine resource access. If a user impersonates a Kubernetes user or group, EKS access-policy permissions do not apply and authorization is evaluated through Kubernetes RBAC only. AWS also recommends reviewing and scoping RBAC for service accounts, especially for DaemonSets and CSI drivers, because those tokens and permissions are effectively present on every node where those Pods run. In that model, assigning `bind`, `impersonate`, or `escalate` to a broadly used service account or administrative role can make effective access harder to reason about and easier to expand beyond the intended boundary.

Impact:

If `bind`, `impersonate`, or `escalate` are granted too broadly in EKS, a user or service account can turn limited access into broader Kubernetes authority, weaken namespace and role boundaries, and increase the blast radius of a compromised identity or workload. The risk is greater when the affected identity is tied to broadly deployed service accounts, because the resulting permissions can become reachable from many nodes and can also combine with workload IAM access where IRSA or EKS Pod Identity is in use.

Audit:

Review EKS RBAC to identify any users, groups, or service accounts granted `bind`, `escalate`, or `impersonate` through custom Kubernetes `Role` or `ClusterRole` objects. Determine which subjects receive those permissions through `RoleBinding` or `ClusterRoleBinding` assignments, and validate that each assignment is required for a defined administrative function. If the cluster uses access entries with Kubernetes groups, review those group mappings as well, because EKS access entries are the recommended way to grant IAM principals access to the Kubernetes API and can reference Kubernetes groups for native RBAC authorization. Flag any cluster-scoped or service-account assignment that lacks a documented need, especially for broadly deployed service accounts such as DaemonSets or CSI drivers.

Identify custom `Role` and `ClusterRole` objects that include `bind`, `escalate`, or `impersonate`:

```
kubectl get roles,clusterroles -A -o json | jq -r '
  .items[]
  | select(.metadata.labels["kubernetes.io/bootstrapping"] != "rbac-
defaults")
  | select(any(.rules[]?; any(.verbs[]?; .=="bind" or .=="escalate" or
.=="impersonate")))
  | [.kind, (.metadata.namespace // "-"), .metadata.name]
  | @tsv
'
```

Map those roles to the subjects that receive them through bindings:

```
(echo -e
"BINDING_KIND\tBINDING_NS\tBINDING_NAME\tROLE_KIND\tROLE_NAME\tSUBJECT_KIND\t
SUBJECT_NS\tSUBJECT_NAME"; \
kubectl get rolebindings,clusterrolebindings -A -o json | jq -r '
  .items[] as $b
  | ($b.subjects // [])[]
  | [
      $b.kind,
      ($b.metadata.namespace // "-"),
      $b.metadata.name,
      $b.roleRef.kind,
      $b.roleRef.name,
      .kind,
      (.namespace // "-"),
      .name
    ]
  | @tsv
') | column -ts $'\t'
```

Remediation:

Remove `bind`, `escalate`, and `impersonate` permissions from any custom `Role` or `ClusterRole` that does not require them for an approved administrative function. In EKS, use access entries and associated access policies for standard cluster access wherever possible, and reserve custom Kubernetes RBAC for cases where the predefined access policies do not meet the requirement. Where these permissions are still necessary, keep them scoped as narrowly as possible and avoid assigning them to workload service accounts.

- Update each affected custom `Role` to remove unnecessary `bind`, `escalate`, or `impersonate` permissions.
- Update each affected custom `ClusterRole` so it retains only the specific elevated permission that is explicitly required.
- Replace broad assignments with lower-privilege roles, and reduce scope from cluster-wide to namespace-scoped where possible. EKS access policies support scoped assignments, and custom RBAC should be used only when those policies do not meet the requirement.
- Review access entries that specify Kubernetes groups, and remove or narrow any group mapping that reaches RBAC roles with these permissions. When you use Kubernetes groups on an access entry, you must create and manage the related native RBAC objects yourself.
- Do not assign these permissions to workload service accounts unless there is a documented and approved need. Review DaemonSet and CSI driver service accounts first, because their RBAC permissions can be reachable from every node where those Pods run.
- Revalidate access after the change to confirm that only approved administrative identities retain these permissions.

Default Value:

In EKS, `bind`, `escalate`, and `impersonate` should not be assumed to exist by default in customer-managed RBAC. When these permissions appear in custom `Role`, `ClusterRole`, `RoleBinding`, or `ClusterRoleBinding` objects, they should be treated as explicit configuration and reviewed accordingly.

References:

1. https://docs.aws.amazon.com/eks/latest/userguide/rbac-hardening.html
2. https://docs.aws.amazon.com/eks/latest/userguide/access-entries.html
3. https://docs.aws.amazon.com/eks/latest/userguide/create-k8s-group-access-entry.html
4. https://docs.aws.amazon.com/eks/latest/userguide/default-roles-users.html

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	4.8 Uninstall or Disable Unnecessary Services on Enterprise Assets and Software Uninstall or disable unnecessary services on enterprise assets and software, such as an unused file sharing service, web application module, or service function.		●	●
v7	14.7 Enforce Access Control to Data through Automated Tools Use an automated tool, such as host-based Data Loss Prevention, to enforce access controls to data even when data is copied off a system.			●

4.1.9 Minimize access to create persistent volumes (Manual)

Profile Applicability:

- Level 1

Description:

Permission to create persistent storage in EKS should be granted only where it is required. In EKS, this includes direct creation of `PersistentVolume` objects and, more commonly, the ability to trigger dynamic provisioning through `PersistentVolumeClaim` and `StorageClass` use, because EKS storage integrations such as the Amazon EBS CSI driver and EKS Auto Mode can automatically provision backing storage when applications request it. For service accounts, this remains an RBAC concern because CSI drivers and other storage components operate through Kubernetes identities and permissions inside the cluster.

Rationale:

Minimizing this access is important because persistent storage in EKS is not just a short-lived workload resource. The Amazon EBS CSI driver manages the lifecycle of Amazon EBS volumes for Kubernetes volumes, and EKS Auto Mode uses `StorageClass` definitions to automatically provision EBS volumes when applications request persistent storage. `StorageClass` settings can also influence technical characteristics such as volume type, encryption, IOPS, and KMS key usage, so the ability to create storage can affect durability, performance, encryption, and cost rather than only enabling routine application writes.

This access is also sensitive because storage creation commonly depends on CSI driver service accounts and related RBAC. CSI drivers often require broad RBAC permissions because they interact with nodes, persistent volumes, and storage APIs, and that DaemonSet service account tokens and permissions are effectively present on every node where those Pods run. In EKS Auto Mode, AWS-managed block storage policies include permissions to create and manage persistent volumes, update persistent volume claims, and read storage classes, which shows that storage provisioning rights are part of a broader control plane and service-account permission path that should be tightly scoped.

Impact:

If persistent volume creation is granted too broadly in EKS, a user or service account can provision durable storage that outlives Pod execution, expands the cluster data footprint, and triggers creation of backing AWS storage resources without a justified need. When those permissions are tied to broadly deployed CSI components or overprivileged service accounts, a compromised workload can create long-lived storage paths that increase persistence, cost, and the scope of unauthorized activity.

Audit:

Review EKS RBAC to identify any users, groups, or service accounts that can create `PersistentVolume`, `PersistentVolumeClaim`, or `StorageClass` resources. In EKS, this review should give particular attention to `PersistentVolumeClaim` and `StorageClass` permissions because persistent storage is commonly provisioned dynamically through CSI drivers and EKS storage integrations rather than by manually creating `PersistentVolume` objects. Also review the service accounts and bindings used by storage components, because CSI drivers commonly hold broad RBAC permissions related to nodes, persistent volumes, and storage APIs.

Remediation:

Remove persistent-storage creation rights from any user, group, or service account that does not require them, and replace broad assignments with the lowest-privilege role and narrowest scope possible. In EKS, prioritize `PersistentVolumeClaim` and `StorageClass` permissions because persistent storage is commonly provisioned dynamically through CSI drivers rather than by manually creating `PersistentVolume` objects. Limit direct `PersistentVolume` and `StorageClass` administration to approved administrators, and allow workloads to use only the `PersistentVolumeClaim` access required for approved storage classes. After changes are made, verify that required applications can still provision storage through the intended CSI driver path.

- Remove unnecessary create access to `PersistentVolume` objects wherever possible, because direct PV creation is less common than PVC-driven dynamic provisioning in EKS.
- Reduce `PersistentVolumeClaim` and `StorageClass` creation rights to the smallest set of approved users, groups, and service accounts. `StorageClass` settings can affect how backing storage is provisioned and managed, so broad access creates more than routine workload write capability.
- Review CSI driver service accounts and bound RBAC first, especially for storage components such as the EBS CSI driver and EFS CSI driver, because CSI drivers commonly hold broad RBAC permissions related to nodes, persistent volumes, and storage APIs.
- Revalidate application storage provisioning after the change to confirm that only approved workloads can create claims and obtain persistent storage through the approved storage classes.

Default Value:

By default, EKS does not use one dedicated setting for persistent-volume creation; access depends on RBAC, CSI driver deployment, and `StorageClass` configuration.

References:

1. https://docs.aws.amazon.com/eks/latest/userguide/ebs-csi.html
2. https://docs.aws.amazon.com/eks/latest/userguide/efs-csi.html

3. https://docs.aws.amazon.com/eks/latest/userguide/rbac-hardening.html
4. https://docs.aws.amazon.com/eks/latest/userguide/access-policy-permissions.html

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	6.8 Define and Maintain Role-Based Access Control Define and maintain role-based access control, through determining and documenting the access rights necessary for each role within the enterprise to successfully carry out its assigned duties. Perform access control reviews of enterprise assets to validate that all privileges are authorized, on a recurring schedule at a minimum annually, or more frequently.			●

4.1.10 Minimize access to the proxy sub-resource of nodes (Manual)

Profile Applicability:

- Level 1

Description:

Access to the `nodes/proxy` subresource in EKS should be limited to identities with a documented node-administration or troubleshooting need. This review should cover custom `Role` and `ClusterRole` rules, `RoleBinding` and `ClusterRoleBinding` assignments, and any service accounts that receive node subresource access. In EKS, access entries are the preferred way to grant IAM principals access to the Kubernetes API, while custom native RBAC remains the path for permissions that aren't covered by the predefined access policies, which is where sensitive node subresource access is typically introduced.

Rationale:

This control is important because `nodes/proxy` is a node-facing permission, not an ordinary namespace-scoped workload permission. In EKS, nodes run operational components such as the kubelet daemon and the `kube-proxy` add-on, and cluster components and add-ons can rely on RBAC to function. Since predefined EKS access policies don't cover every custom permission need, organizations that use Kubernetes groups on access entries or bind service accounts to custom RBAC objects can still introduce sensitive node subresource permissions through native Kubernetes authorization.

The risk is higher when these permissions are assigned to broadly deployed service accounts. DaemonSet Pods run on every node in the cluster, and their service account tokens and RBAC permissions are present on every node. An unauthorized process on a node may be able to access service account tokens from other Pods on that node, including DaemonSet Pods. EKS control plane logging records API server, audit, and authenticator activity from the control plane, so keeping node-proxy access tightly scoped helps preserve a clearer boundary between routine cluster API access and privileged node-adjacent operations.

Impact:

If `nodes/proxy` access is granted too broadly in EKS, a user or service account can gain an unnecessary node-adjacent access path that weakens separation between namespace-level operations and node administration. When those permissions are tied to broadly deployed service accounts, the blast radius can extend across many nodes and make unintended access harder to review, contain, and recover from.

Audit:

Review EKS RBAC to identify custom `Role` and `ClusterRole` objects that grant access to the `nodes/proxy` subresource, and validate that each permission is required for an approved troubleshooting, maintenance, or platform-administration function. Focus this review on customer-managed RBAC because EKS creates its own default RBAC identities for cluster components, while customer-defined `node-proxy` access is usually introduced through native Kubernetes RBAC. If the cluster uses access entries with Kubernetes groups, review those mappings as well, because access entries are the preferred way to grant IAM principals access to the Kubernetes API and Kubernetes groups on an access entry still rely on manually managed RBAC objects.

This command returns each custom `Role` or `ClusterRole` whose RBAC rules include `nodes/proxy`, along with the namespace, role name, and allowed verbs. It excludes Kubernetes bootstrapped default roles and `system:-prefixed ClusterRole` objects so the output stays focused on customer-managed RBAC that may require review.

```
kubectl get roles,clusterroles -A -o json | jq -r '
  .items[]
  | select(any(.rules[]?; any(.resources[]?; .=="nodes/proxy")))
  | select(
      (.kind=="Role")
      or
      (
        .kind=="ClusterRole"
        and (.metadata.labels["kubernetes.io/bootstrapping"] != "rbac-
defaults")
        and (.metadata.name | startswith("system:") | not)
      )
    )
  | [
      .kind,
      (.metadata.namespace // "-"),
      .metadata.name,
      (
        [.rules[]?
         | select(any(.resources[]?; .=="nodes/proxy"))
         | "verbs=" + ((.verbs // []) | join(","))
        ] | join("; ")
      )
    ]
  | @tsv
'
```

Remediation:

Reduce custom RBAC access to the `nodes/proxy` API path so it is limited to approved administrative identities that require node-level access. In EKS, use access entries and associated access policies for standard cluster access wherever possible, and reserve custom Kubernetes RBAC for cases where the predefined access policies do not meet the requirement. Keep `nodes/proxy` limited to tightly controlled troubleshooting, maintenance, or platform-administration use cases, and avoid assigning it to workload service accounts.

- Remove `nodes/proxy` from any custom `Role` or `ClusterRole` where node-level proxying is not explicitly required.
- Where some node-related access is still needed, replace `nodes/proxy` with narrower permissions on only the specific resources and verbs required, and reduce scope from `ClusterRole` to namespace-scoped `Role` where the function allows it.
- Remove any `RoleBinding` or `ClusterRoleBinding` that grants `nodes/proxy` to workload service accounts, CI/CD identities, or broad operator groups unless there is an approved exception tied to a defined operational function.
- Review access entries that use Kubernetes groups, and remove or narrow any group mapping that reaches RBAC roles with `nodes/proxy` access.
- Keep any remaining `nodes/proxy` assignment only on dedicated support or platform-admin identities, and document the purpose, scope, and owner of each assignment.

- Revalidate the updated roles and bindings after the change to confirm that required node-support tasks still work and that no unnecessary custom RBAC objects retain `nodes/proxy` access.

Default Value:

By default, EKS does not expose a separate cluster setting for `nodes/proxy` access comes from RBAC assignments, while EKS-created platform roles should be distinguished from customer-managed roles during review.

References:

1. https://docs.aws.amazon.com/eks/latest/userguide/rbac-hardening.html
2. https://docs.aws.amazon.com/eks/latest/userguide/access-entries.html
3. https://docs.aws.amazon.com/eks/latest/userguide/create-k8s-group-access-entry.html
4. https://docs.aws.amazon.com/eks/latest/userguide/default-roles-users.html

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	6.8 Define and Maintain Role-Based Access Control Define and maintain role-based access control, through determining and documenting the access rights necessary for each role within the enterprise to successfully carry out its assigned duties. Perform access control reviews of enterprise assets to validate that all privileges are authorized, on a recurring schedule at a minimum annually, or more frequently.			●

4.1.11 Minimize access to webhook configuration objects (Manual)

Profile Applicability:

- Level 1

Description:

Access to `ValidatingWebhookConfiguration` and `MutatingWebhookConfiguration` objects in EKS should be limited to tightly controlled cluster-administrative identities. These objects define admission webhooks that intercept Kubernetes API requests before they are accepted, and they apply at cluster scope rather than namespace scope. Write access to them should not be broadly granted to general users, namespace operators, CI/CD identities, or workload service accounts.

Rationale:

This setting matters because admission webhooks sit directly in the API request path. The EKS control plane page explains that validating and mutating webhooks can inspect or change objects before they are accepted, and it also states that poor webhook configuration can destabilize the control plane by blocking cluster-critical operations. The workloads page adds that dynamic admission webhooks are invoked in sequence for API requests, each webhook has a default timeout of 10 seconds, and multiple or slow webhooks can increase request latency.

The risk is amplified because these configuration objects control where admission callbacks are sent and how broadly they apply. The control plane page calls out catch-all webhook patterns that match all API groups, versions, operations, resources, and scopes as something to avoid, and it recommends a fail-open policy with a timeout shorter than 30 seconds when availability risk must be reduced. The access policy permissions page also shows that when predefined access policies do not meet a requirement, authorization falls back to native Kubernetes RBAC through group mappings, which is where broad write access to webhook configuration objects can be introduced for users or service accounts.

Impact:

If access to webhook configuration objects is granted too broadly in EKS, a user or service account can change how cluster-wide API requests are validated or mutated, which can reject requests, alter admitted resources, increase API latency, and in poorly designed cases interfere with cluster-critical operations. Mutating webhooks can also increase etcd churn by creating additional stored resource versions during repeated mutations, which raises the operational impact of overprivileged access.

Audit:

Audit this control by identifying who can manage cluster-scoped `ValidatingWebhookConfiguration` and `MutatingWebhookConfiguration` resources, and confirm that webhook definitions are limited to approved admission use cases in EKS. Review whether cluster access is managed through access entries and associated access policies or through Kubernetes groups and native RBAC, because EKS access entries are the primary cluster access model and Kubernetes-group authorization still depends on manually managed `Role`, `ClusterRole`, `RoleBinding`, and `ClusterRoleBinding` objects. Confirm that webhook configurations exist only where operationally required, that their rules and scope are narrowly defined, and that write access to manage them is restricted to tightly controlled administrative identities. Admission webhooks sit directly in the API request path, and broad or unhealthy webhook configurations can increase API latency or interfere with cluster operations.

- Identify whether cluster access is governed through access entries and associated access policies, through Kubernetes groups and native RBAC, or through a mixed model.
- Review cluster-scoped roles for permissions on `validatingwebhookconfigurations` and `mutatingwebhookconfigurations`, and trace those permissions through bindings to the exact users, groups, and service accounts that receive them.
- Review existing webhook configurations to confirm their rules, selectors, and scope are narrowly defined and not written as broad catch-all configurations that match all resources or operations.
- Review any service accounts that can modify webhook configuration objects, especially broadly deployed components, because node-wide or widely deployed service account permissions increase the reach of a compromised workload.

This command lists all validating and mutating webhook configuration objects in the cluster so they can be reviewed for presence and expected use:

```
kubectl get validatingwebhookconfigurations,mutatingwebhookconfigurations -o
wide
```

Remediation:

Remove permission to create, update, patch, or delete `ValidatingWebhookConfiguration` and `MutatingWebhookConfiguration` objects from broad `ClusterRole` and `Role` assignments, and rebind that access only to tightly controlled cluster-administrative identities. In EKS, use access entries and associated access policies for standard cluster access wherever possible, and reserve native Kubernetes RBAC for cases where the predefined access policies do not meet the requirement. Keep webhook-configuration management limited to a small set of approved administrative identities, because these objects sit directly in the admission path and operate at cluster scope.

- Remove `create`, `update`, `patch`, and `delete` permissions on `validatingwebhookconfigurations` and `mutatingwebhookconfigurations` from broad custom `ClusterRole` and `Role` assignments.
- Rebind any remaining webhook-management access only to dedicated platform-admin identities, and keep the scope and ownership of each assignment documented.
- Review access entries that use Kubernetes groups, and remove or narrow any group mapping that reaches RBAC roles with webhook-configuration write access.
- Remove webhook-configuration management rights from workload service accounts, CI/CD identities, and broadly deployed components unless there is a defined and approved operational need.
- Review each existing webhook configuration and narrow its `rules`, `namespaceSelector`, `objectSelector`, and failure behavior so it applies only to the required namespaces, resources, and operations. Avoid broad catch-all patterns, and where availability risk must be reduced, use a fail-open policy with a timeout shorter than 30 seconds.
- Revalidate admission behavior after the change to confirm required controllers still function and that webhook scope is not broad enough to increase API latency or interfere with cluster operations.

Default Value:

By default, EKS does not expose a dedicated setting for webhook configuration access; management of webhook configuration objects comes from access entries, Kubernetes groups, and RBAC permissions, while webhook behavior itself is defined by the webhook configuration objects present in the cluster.

References:

1. https://docs.aws.amazon.com/eks/latest/best-practices/control-plane.html
2. https://docs.aws.amazon.com/eks/latest/userguide/access-entries.html
3. https://docs.aws.amazon.com/eks/latest/userguide/access-policy-permissions.html
4. https://docs.aws.amazon.com/eks/latest/userguide/rbac-hardening.html

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	6.8 Define and Maintain Role-Based Access Control Define and maintain role-based access control, through determining and documenting the access rights necessary for each role within the enterprise to successfully carry out its assigned duties. Perform access control reviews of enterprise assets to validate that all privileges are authorized, on a recurring schedule at a minimum annually, or more frequently.			●

4.1.12 Minimize access to the service account token creation (Manual)

Profile Applicability:

- Level 1

Description:

Access to service account token creation in EKS should be limited to tightly controlled administrative or platform identities. Service accounts are a core Kubernetes identity type in EKS, projected service account tokens are used by workloads to authenticate, and Pods that do not need Kubernetes API access can avoid unnecessary token exposure by disabling automatic token mounting. Permission to request or create service account tokens should not be broadly granted to namespace operators, CI/CD identities, or general workload service accounts.

Rationale:

This setting is important because a service account token is an authentication credential. In EKS, the `BoundServiceAccountTokenVolume` feature is enabled by default, which allows workloads to request JSON web tokens that are audience-bound, time-bound, and key-bound, with a one-hour expiration. EKS also supports projected service account tokens for authentication flows such as IRSA, where a Pod uses a web identity token associated with a Kubernetes service account to obtain AWS credentials through `AssumeRoleWithWebIdentity`.

The risk increases when token creation is available to identities that do not need it. A Pod can authenticate to the Kubernetes API with an auto-mounted token, exposed tokens on a node can be accessed by unauthorized processes, and for workloads tied to IRSA or EKS Pod Identity, token exposure can extend beyond Kubernetes RBAC into AWS IAM credentials. Keeping token creation tightly scoped reduces the number of identities that can mint usable service account credentials and strengthens the service account as a deliberate workload boundary instead of a broadly reusable access path.

Impact:

If service account token creation is granted too broadly in EKS, a compromised user or service account can mint valid tokens for targeted service accounts and authenticate with the Kubernetes permissions of those identities. When the targeted service account is also tied to IRSA or EKS Pod Identity, that token path can also lead to AWS credentials, increasing the blast radius beyond the cluster.

Audit:

Begin by confirming that service account token creation in EKS is limited to identities with an approved administrative or workload identity need. Review token issuance through native Kubernetes RBAC, not through access entries, because access entries govern IAM principal access to the Kubernetes API while service account tokens remain Kubernetes credentials used by workloads and service accounts. Also confirm that token use aligns with expected Kubernetes API access, IRSA, or EKS Pod Identity behavior rather than ad hoc credential minting.

- Review `Role` and `ClusterRole` objects for permission to create the `serviceaccounts/token` subresource or any equivalent token-issuance path.
- Trace those permissions through `RoleBinding` and `ClusterRoleBinding` assignments to the exact users, groups, and service accounts that inherit them.
- Give extra scrutiny to DaemonSet and other broadly deployed service accounts, because their tokens and RBAC permissions are present on every node where those Pods run.
- Confirm each assignee has a documented need for token issuance, and that the permission is not exposed through broad custom roles, CI/CD identities, default service accounts, or general workload service accounts.
- Validate that current token use matches expected workload identity behavior such as Kubernetes API access, IRSA, or EKS Pod Identity. For IRSA, the projected service account token is the web identity token used for `AssumeRoleWithWebIdentity`.
- Flag any unnecessary token-creation path for remediation, especially where Pods could mount tokens by default without a defined operational need.

Remediation:

Review all `Role` and `ClusterRole` definitions for access to the `serviceaccounts/token` subresource, and remove that permission from any user, group, CI/CD identity, or workload service account that does not have a documented operational requirement. In EKS, service account token issuance remains a Kubernetes RBAC concern, while IAM principal access to the Kubernetes API is managed separately through access entries and associated access policies. Token-creation rights should remain limited to tightly controlled administrative or platform identities, because projected service account tokens are usable Kubernetes credentials and can also participate in workload identity flows such as IRSA.

- Review `Role` and `ClusterRole` objects for access to `serviceaccounts/token`, and remove that permission from broad operational roles, CI/CD identities, default service accounts, and general workload service accounts.
- Reassign token-creation rights only to approved administrative or platform identities with a documented need to issue service account tokens.
- Validate that required workload identity flows still function correctly for Pods that use projected service account tokens, including IRSA and service-account-based AWS access patterns.

- Recheck `RoleBinding` and `ClusterRoleBinding` assignments to confirm no unintended user, group, or service account still inherits token-creation rights. Give extra scrutiny to broadly deployed service accounts such as DaemonSets because their tokens and permissions are present on every node where those Pods run.

Default Value:

By default service account token creation is not controlled by a separate built-in cluster setting. It is governed by native Kubernetes RBAC, and any permission to create the `serviceaccounts/token` subresource should be treated as explicit configuration and reviewed accordingly. Projected service account tokens are enabled by default for workloads, but the right to issue them still depends on RBAC grants.

References:

1. https://docs.aws.amazon.com/eks/latest/userguide/service-accounts.html
2. https://docs.aws.amazon.com/eks/latest/userguide/iam-roles-for-service-accounts.html
3. https://docs.aws.amazon.com/eks/latest/best-practices/identity-and-access-management.html
4. https://docs.aws.amazon.com/eks/latest/userguide/rbac-hardening.html

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	6.8 Define and Maintain Role-Based Access Control Define and maintain role-based access control, through determining and documenting the access rights necessary for each role within the enterprise to successfully carry out its assigned duties. Perform access control reviews of enterprise assets to validate that all privileges are authorized, on a recurring schedule at a minimum annually, or more frequently.			●

4.2 Pod Security Standards

Pod Security Standards (PSS) are the Kubernetes-defined security profiles for controlling what pods are allowed to do at runtime. They define three cumulative levels, `privileged`, `baseline`, and `restricted`, and in EKS they are applied through the built-in Pod Security Admission (PSA) controller at the namespace level. PSA uses namespace labels to set `enforce`, `audit`, and `warn` behavior, so teams can block noncompliant pods, surface violations without blocking, or use both during rollout. AWS currently recommends applying at least the `baseline` profile cluster-wide and the `restricted` profile for workload namespaces.

For current EKS practice, PSS with PSA is the native starting point for pod-level security hardening, while policy engines such as Kyverno or OPA Gatekeeper are used when you need more granular, cross-resource, or organization-specific controls. `PodSecurityPolicy` is no longer part of the supported model: Kubernetes deprecated it in v1.21 and removed it in v1.25, and AWS points to PSA and policy-based alternatives as the replacement path. A practical EKS approach is to start namespaces with `warn` and `audit`, evaluate existing workloads for violations, and then move to `enforce` once manifests have been updated to meet the selected profile.

4.2.1 Minimize the admission of privileged containers (Automated)

Profile Applicability:

- Level 1

Description:

Minimize the admission of privileged containers in EKS by enforcing Pod Security Standards through Pod Security Admission or an equivalent policy engine, and restrict privileged workloads to only the limited system or platform namespaces that truly require them. AWS recommends enforcing at least the baseline profile cluster-wide and the `restricted` profile for workload namespaces because this reduces the ability to create privileged containers outside approved areas of the cluster.

Do not generally permit containers to run with `securityContext.privileged: true` in EKS clusters, except for narrowly approved system or platform workloads.

Rationale:

In EKS, a privileged container inherits all Linux capabilities associated with root on the host, which significantly weakens container isolation and increases the chance that a compromised workload can interact with the underlying node in unsafe ways. AWS notes that containers rarely need this level of access and that Pod Security Standards are the built-in Kubernetes mechanism for replacing older `PodSecurityPolicy` controls and validating pod specifications before they are persisted. Using baseline or restricted enforcement helps block privilege escalation patterns during admission rather than relying only on runtime detection after the pod has started.

If you need to run privileged containers, this should be defined in a separate policy and you should carefully check to ensure that only limited service accounts and users are given permission to use that policy.

Impact:

Workloads that depend on privileged mode may be denied admission in namespaces enforcing Pod Security Standards, so certain node-level agents, device-access workloads, or other specialized platform pods may need redesign, tighter scoping, or isolated placement. On EKS Fargate, privileged containers are not supported at all, which further limits these deployment patterns.

Pods with `spec.containers[].securityContext.privileged: true`, `spec.initContainers[].securityContext.privileged: true`, or `spec.ephemeralContainers[].securityContext.privileged: true` will not be permitted in namespaces enforcing the `baseline` or `restricted` Pod Security Standard.

Audit:

Review the effective Pod Security Admission configuration for every namespace in the EKS cluster and verify that workload namespaces are set to a policy level that disallows privileged containers. In current EKS guidance, AWS recommends enforcing at least the `baseline` profile cluster-wide and the `restricted` profile for workload namespaces to limit creation of privileged containers outside system namespaces.

As part of the audit, confirm that workload namespaces are not unlabeled, not set to `privileged`, and not relying only on `warn` or `audit` where blocking is required. Kubernetes documents that `enforce` rejects violating pods, while `warn` and `audit` report violations without preventing admission.

Then inspect existing pods for admitted use of `spec.containers[].securityContext.privileged`, `spec.initContainers[].securityContext.privileged`, and `spec.ephemeralContainers[].securityContext.privileged`. Under the Pod Security Standards, those fields must be `unset` or `false` for the relevant non-privileged profiles.

```
kubectl get pods -A -o json | jq -r '
  .items[]
  | .metadata.namespace as $ns
  | .metadata.name as $pod
  | def scan($type; $list):
      $list[]?
      | select(.securityContext.privileged == true)
      | "\($ns)/\($pod)\t\($type)=\(.name)\tprivileged=true";
    scan("container"; .spec.containers),
    scan("initContainer"; .spec.initContainers),
    scan("ephemeralContainer"; .spec.ephemeralContainers)
'
```

Expected Output:

```
kube-system/aws-node-9vxcs        container=aws-eks-nodeagent
privileged=true
kube-system/aws-node-9vxcs        initContainer=aws-vpc-cni-init
       privileged=true
kube-system/aws-node-dgzj8        container=aws-eks-nodeagent
privileged=true
kube-system/aws-node-dgzj8        initContainer=aws-vpc-cni-init
       privileged=true
kube-system/kube-proxy-dn9w5   container=kube-proxy
privileged=true
kube-system/kube-proxy-wdm9m   container=kube-proxy
privileged=true
```

OR

```
kubectl get pods --all-namespaces -o json | jq '.items[] |
select(.metadata.namespace != "kube-system" and
.spec.containers[]?.securityContext?.privileged == true) | {pod:
.metadata.name, namespace: .metadata.namespace, container:
.spec.containers[].name}'
```

When creating a Pod Security Policy, ["kube-system"] namespaces are excluded by default.

This command uses jq, a command-line JSON processor, to parse the JSON output from kubectl get pods and filter out pods where any container has the `securityContext.privileged` flag set to `true.` Please note that you might need to adjust the command depending on your specific requirements and the structure of your pod specifications.

Remediation:

Configure Pod Security Admission on workload namespaces so privileged containers are denied when new pods are created. In EKS, Pod Security Standards are applied through namespace labels, and current AWS guidance is to enforce at least `baseline` cluster-wide and `restricted` for workload namespaces. For this control, both `baseline` and `restricted` disallow privileged containers, but restricted is typically the stronger choice for user workloads. Only `enforce` blocks noncompliant pods. `warn` and `audit` are useful during rollout, but they do not remediate the issue by themselves.

- Label each workload namespace to enforce restricted.

```
kubectl label ns NAMESPACE pod-security.kubernetes.io/enforce=restricted --
overwrite
```

- Add `warn` and `audit` labels during transition so violations are surfaced before or alongside enforcement.

```
kubectl label ns NAMESPACE pod-security.kubernetes.io/warn=restricted --
overwrite
kubectl label ns NAMESPACE pod-security.kubernetes.io/audit=restricted --
overwrite
```

- Update workload manifests so no regular container, init container, or ephemeral container uses `securityContext.privileged: true`. Under the Pod Security Standards, those fields must be unset or `false` for non-privileged profiles.
- Redeploy or restart affected workloads after the manifest changes so replacement pods are evaluated under the enforced namespace policy. Pod Security Admission evaluates pods when they are created.
- Review system namespaces separately and do not apply the same labels everywhere without checking platform requirements. AWS frames this control around limiting privileged containers outside system namespaces.

Default Value:

By default, there are no restrictions on the creation of privileged containers.

References:

1. https://docs.aws.amazon.com/eks/latest/best-practices/pod-security.html
2. https://docs.aws.amazon.com/eks/latest/userguide/rbac-hardening.html
3. https://docs.aws.amazon.com/eks/latest/userguide/security-best-practices.html
4. https://docs.aws.amazon.com/eks/latest/best-practices/security.html

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	5.4 Restrict Administrator Privileges to Dedicated Administrator Accounts Restrict administrator privileges to dedicated administrator accounts on enterprise assets. Conduct general computing activities, such as internet browsing, email, and productivity suite use, from the user's primary, non-privileged account.	●	●	●
v7	4.3 Ensure the Use of Dedicated Administrative Accounts Ensure that all users with administrative account access use a dedicated or secondary account for elevated activities. This account should only be used for administrative activities and not internet browsing, email, or similar activities.	●	●	●

4.2.2 Minimize the admission of containers wishing to share the host process ID namespace (Automated)

Profile Applicability:

- Level 1

Description:

Minimize the admission of pods that request host process ID namespace sharing by restricting use of `spec.hostPID: true` in EKS workload namespaces. In the EKS Pod Security Standards model, this is enforced through Pod Security Admission or an equivalent policy engine, and AWS recommends enforcing at least the `baseline` profile cluster-wide and the `restricted` profile for workload namespaces. Because the `baseline` profile prohibits `hostPID` and `restricted` inherits that restriction, general user workloads should not be permitted to use host PID sharing.

Do not generally permit containers to be run with the `hostPID` flag set to true.

Rationale:

AWS notes that Linux containers rely on namespaces to maintain isolation, and specifically warns that granting a container access to host resources or namespaces such as the host PID namespace reduces the effectiveness of that isolation. In EKS, the Pod Security Standards are the built-in Kubernetes mechanism for controlling this at admission time, and AWS documents that the `baseline` profile prevents known privilege escalations by prohibiting `hostPID`, while `restricted` inherits the same control and adds tighter hardening. Applying those profiles through Pod Security Admission allows EKS to reject new pods that request host PID sharing instead of relying only on runtime detection after deployment.

Impact:

Pods that set `spec.hostPID: true` will be denied in namespaces enforcing the EKS `baseline` or `restricted` Pod Security Standard, so diagnostic, node-management, or other specialized workloads that depend on host PID sharing may need a tightly scoped exception path or separate placement.

Pods defined with `spec.hostPID: true` will not be permitted unless they are run under a specific policy.

Audit:

Review the effective Pod Security Admission configuration for every namespace in the EKS cluster and verify that workload namespaces enforce a profile that disallows `spec.hostPID: true`. In current EKS guidance, AWS recommends enforcing at least the baseline profile cluster-wide and the restricted profile for workload namespaces. AWS also documents that the baseline policy prohibits hostPID, and that PSA enforce mode prevents pods with Pod Security Standards violations from being applied.

Do not rely only on a live pod search as the primary audit. A pod query shows workloads that are already running with host PID sharing, but it does not prove that new pods requesting `hostPID` would be denied. Also, do not reference `PodSecurityPolicy` as the current control model for EKS. AWS documents Pod Security Standards and Pod Security Admission as the supported replacement path. Any namespace exemptions should be treated as explicit configuration choices, not assumed defaults.

Inspect existing pods for admitted use of host PID sharing:

```
kubectl get pods -A -o json | jq -r '
  .items[]
  | select(.spec.hostPID == true)
  | "\(.metadata.namespace)/\(.metadata.name)\thostPID=true"
'
```

This command identifies pods already running with `spec.hostPID: true`, which is important because tightening PSA does not remove existing pods that were admitted earlier.

Remediation:

Configure Pod Security Admission on workload namespaces so pods that request spec.hostPID: true are denied at admission time. In current EKS guidance, AWS recommends applying Pod Security Standards with at least the baseline profile cluster-wide and the restricted profile for workload namespaces. AWS also states that the baseline profile prohibits `hostPID`, while restricted inherits that restriction. For remediation, use `enforce` because AWS documents that only `enforce` rejects violating pods. `warn` and `audit` can help during rollout, but they do not block admission.

- Label each workload namespace to enforce `restricted`.

```
kubectl label ns NAMESPACE pod-security.kubernetes.io/enforce=restricted --
overwrite
```

- Add `warn` and `audit` during transition if you want visibility into violations before or alongside enforcement.

```
kubectl label ns NAMESPACE pod-security.kubernetes.io/warn=restricted --
overwrite
kubectl label ns NAMESPACE pod-security.kubernetes.io/audit=restricted --
overwrite
```

- Update workload manifests to remove `spec.hostPID: true` then redeploy the affected workloads so new pods are evaluated under the enforced policy.
- Review any exceptions separately and keep them tightly scoped to approved platform namespaces or workloads. AWS documents exemptions as explicit configuration, not something to assume by default.
- Check for already running pods that still use host PID sharing, because AWS notes that changing a namespace to a more restrictive enforced profile does not delete existing pods.

```
kubectl get pods -A -o json | jq -r '
  .items[]
  | select(.spec.hostPID == true)
  | "\(.metadata.namespace)/\(.metadata.name)\thostPID=true"
'
```

Default Value:

By default, there are no restrictions on the creation of `hostPID` containers.

References:

1. https://docs.aws.amazon.com/eks/latest/best-practices/pod-security.html
2. https://docs.aws.amazon.com/eks/latest/userguide/rbac-hardening.html
3. https://docs.aws.amazon.com/eks/latest/best-practices/security.html
4. https://docs.aws.amazon.com/eks/latest/userguide/security.html

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	5.4 Restrict Administrator Privileges to Dedicated Administrator Accounts Restrict administrator privileges to dedicated administrator accounts on enterprise assets. Conduct general computing activities, such as internet browsing, email, and productivity suite use, from the user's primary, non-privileged account.	●	●	●
v7	4.3 Ensure the Use of Dedicated Administrative Accounts Ensure that all users with administrative account access use a dedicated or secondary account for elevated activities. This account should only be used for administrative activities and not internet browsing, email, or similar activities.	●	●	●

4.2.3 Minimize the admission of containers wishing to share the host IPC namespace (Automated)

Profile Applicability:

- Level 1

Description:

Minimize the admission of pods that request `spec.hostIPC: true` in EKS so that host IPC namespace sharing is limited to tightly controlled, explicitly approved platform use cases. In the EKS Pod Security Standards model, this is done through Pod Security Admission or an equivalent policy engine, and AWS recommends enforcing at least the `baseline` profile cluster-wide and the r`estricted` profile for workload namespaces. Because AWS documents that the `baseline` profile prohibits `hostIPC` and `restricted` inherits the baseline restrictions, general workload namespaces should not permit host IPC sharing.

Do not generally permit containers to be run with the `hostIPC` flag set to true.

Rationale:

AWS states that pod specification settings can strengthen or weaken the cluster security posture and that a primary goal is preventing a process running in a container from escaping container isolation boundaries and gaining access to the underlying host. In EKS, Pod Security Admission evaluates pod specifications at the namespace level using `enforce`, `audit`, and `warn` modes, and AWS documents that `enforce` rejects violating pods while `warn` and `audit` only report violations. Since `hostIPC` is prohibited by the `baseline` profile and inherited by `restricted`, applying those profiles to workload namespaces provides an admission-time control that blocks pod specs requesting host IPC sharing before they are persisted and started.

Impact:

Pods with `spec.hostIPC: true` will be denied in namespaces that enforce the EKS `baseline` or `restricted` Pod Security Standard, so specialized workloads that depend on host IPC sharing will need a tightly scoped exception path or separate placement. Also, if a namespace is moved to a more restrictive profile, AWS notes that existing noncompliant pods are not automatically deleted, so only newly created or restarted pods are blocked by the updated enforced policy.

Pods defined with `spec.hostIPC: true` will not be permitted unless they are run under a specific policy`

Audit:

Review the effective Pod Security Admission configuration for every namespace in the EKS cluster and verify that workload namespaces enforce a profile that disallows `spec.hostIPC: true`. Current EKS guidance is to apply the Kubernetes Pod Security Standards with at least the `baseline` profile cluster-wide and the `restricted` profile for workload namespaces. AWS documents that the baseline profile prohibits `hostIPC`, and that restricted inherits the baseline restrictions.

Do not rely only on a live pod search as the primary audit. A pod query can identify workloads that are already running with host IPC sharing, but it does not prove that new pods requesting `hostIPC` would be denied. AWS documents that Pod Security Admission uses namespace labels and that only enforce rejects violating pods, while warn and audit only surface violations.

Inspect running pods for admitted use of host IPC sharing:

```
kubectl get pods -A -o json | jq -r '
  .items[]
  | select(.spec.hostIPC == true)
  | "\(.metadata.namespace)/\(.metadata.name)\thostIPC=true"
'
```

This identifies pods already running with `spec.hostIPC: true`, which matters because AWS notes that changing a namespace to a more restrictive enforced profile does not delete existing pods that were previously admitted.

Remediation:

Configure Pod Security Admission on workload namespaces so pods that request `spec.hostIPC: true` are denied at admission time. In current EKS guidance, the baseline Pod Security Standard prohibits `hostIPC`, restricted inherits that restriction, and AWS recommends using Pod Security Standards in EKS to limit risky pod settings through namespace-based enforcement. For remediation, use `enforce` because AWS documents that only `enforce` rejects violating pods. `warn` and `audit` can help during rollout, but they do not block admission.

- Label each workload namespace to enforce restricted.

```
kubectl label ns NAMESPACE pod-security.kubernetes.io/enforce=restricted --
overwrite
```

- Add warn and audit during transition if you want visibility into violations before or alongside enforcement.

```
kubectl label ns NAMESPACE pod-security.kubernetes.io/warn=restricted --
overwrite
kubectl label ns NAMESPACE pod-security.kubernetes.io/audit=restricted --
overwrite
```

- Update workload manifests to remove `spec.hostIPC: true`, then redeploy the affected workloads so new pods are evaluated under the enforced policy.
- Review any exceptions separately and keep them tightly scoped to approved platform namespaces or workloads. AWS documents exemptions as explicit configuration, not something to assume by default.
- Check for already running pods that still use host IPC sharing, because AWS notes that changing a namespace to a more restrictive enforced profile does not delete existing pods.

```
kubectl get pods -A -o json | jq -r '
  .items[]
  | select(.spec.hostIPC == true)
  | "\(.metadata.namespace)/\(.metadata.name)\thostIPC=true"
'
```

Default Value:

By default, there are no restrictions on the creation of `hostIPC` containers.

References:

1. https://docs.aws.amazon.com/eks/latest/best-practices/pod-security.html
2. https://docs.aws.amazon.com/eks/latest/userguide/rbac-hardening.html
3. https://docs.aws.amazon.com/eks/latest/userguide/security.html
4. https://docs.aws.amazon.com/eks/latest/best-practices/security.html

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	5.4 Restrict Administrator Privileges to Dedicated Administrator Accounts Restrict administrator privileges to dedicated administrator accounts on enterprise assets. Conduct general computing activities, such as internet browsing, email, and productivity suite use, from the user's primary, non-privileged account.	●	●	●
v7	4.3 Ensure the Use of Dedicated Administrative Accounts Ensure that all users with administrative account access use a dedicated or secondary account for elevated activities. This account should only be used for administrative activities and not internet browsing, email, or similar activities.	●	●	●

4.2.4 Minimize the admission of containers wishing to share the host network namespace (Automated)

Profile Applicability:

- Level 1

Description:

Minimize the admission of pods that request `spec.hostNetwork: true` in EKS so that sharing the host network namespace is limited to tightly controlled, explicitly approved platform scenarios. In EKS, AWS recommends applying Kubernetes Pod Security Standards with the built-in Pod Security Admission controller or a policy engine, using at least the `baseline` profile cluster-wide and the `restricted` profile for workload namespaces. AWS documents that the baseline profile prohibits `hostNetwork`, and `restricted` inherits the `baseline` restrictions, so general workload namespaces should not permit host network sharing.

Do not generally permit containers to be run with the `hostNetwork` flag set to true.

Rationale:

AWS states that granting a container access to host resources or namespaces such as the host network namespace is strongly discouraged because it reduces the effectiveness of container isolation. In EKS on EC2 worker nodes, pods configured with `hostNetwork: true` always have access to the EC2 Instance Metadata Service, even when IAM roles for service accounts are enabled, which increases exposure to node-level metadata and credentials paths that ordinary pods can be restricted from using. Applying Pod Security Standards in `enforce` mode addresses this at admission time by rejecting pods whose specs request host network sharing before those pods are created.

Impact:

Pods with `spec.hostNetwork: true` will be denied in namespaces that enforce the EKS `baseline` or `restricted` Pod Security Standard, so workloads that depend on host networking need a tightly scoped exception path or separate placement. On EKS Fargate, pods cannot specify `HostNetwork` at all.

Pods defined with `spec.hostNetwork: true` will not be permitted unless they are run under a specific policy.

Audit:

Review the effective Pod Security Admission configuration for every namespace in the EKS cluster and verify that workload namespaces enforce a profile that disallows `spec.hostNetwork: true`. In current EKS guidance, AWS recommends enforcing at least the `baseline` profile cluster-wide and the `restricted` profile for workload namespaces. AWS documents that the `baseline` profile prohibits `hostNetwork`, and Kubernetes documents that `enforce` mode blocks pods that violate the configured Pod Security Standard.

Inspect running pods for admitted use of host network sharing:

```
kubectl get pods -A -o json | jq -r '
  .items[]
  | select(.spec.hostNetwork == true)
  | "\(.metadata.namespace)/\(.metadata.name)\thostNetwork=true"
'
```

This pod query identifies pods already running with `spec.hostNetwork: true`, which matters because AWS notes that changing a namespace to a more restrictive enforced profile does not delete existing pods that were previously admitted.

Remediation:

Configure Pod Security Admission on workload namespaces so pods that request `spec.hostNetwork: true` are denied at admission time. Use `enforce` on workload namespaces with `restricted`, or at minimum `baseline`, and reserve any exceptions for narrowly approved platform workloads. After updating namespace enforcement, remove `hostNetwork: true` from affected manifests and redeploy those workloads so new pods are evaluated under the enforced policy. AWS also notes that pods with `hostNetwork: true` always have IMDS access on EKS EC2 nodes, which is another reason to minimize this setting.

- Label each workload namespace to enforce restricted.

```
kubectl label ns NAMESPACE pod-security.kubernetes.io/enforce=restricted --
overwrite
```

- Add warn and audit during rollout if you want visibility into violations, but do not rely on them for remediation because they do not block pod creation.

```
kubectl label ns NAMESPACE pod-security.kubernetes.io/warn=restricted --
overwrite
kubectl label ns NAMESPACE pod-security.kubernetes.io/audit=restricted --
overwrite
```

- Update manifests to remove `spec.hostNetwork: true`, then redeploy the affected workloads.

- Review any required exceptions separately and keep them tightly scoped to approved platform namespaces or workloads. AWS's examples show exemptions as explicit configuration, not something to assume broadly.
- Check for already running pods that still use host networking, because changing a namespace to a more restrictive profile does not remove existing pods automatically.

```
kubectl get pods -A -o json | jq -r '
  .items[]
  | select(.spec.hostNetwork == true)
  | "\(.metadata.namespace)/\(.metadata.name)\thostNetwork=true"
'
```

Default Value:

By default, there are no restrictions on the creation of `hostNetwork` containers.

References:

1. https://docs.aws.amazon.com/eks/latest/best-practices/pod-security.html
2. https://docs.aws.amazon.com/eks/latest/userguide/security.html
3. https://docs.aws.amazon.com/eks/latest/userguide/iam-roles-for-service-accounts.html
4. https://docs.aws.amazon.com/eks/latest/best-practices/security.html

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	5.4 Restrict Administrator Privileges to Dedicated Administrator Accounts Restrict administrator privileges to dedicated administrator accounts on enterprise assets. Conduct general computing activities, such as internet browsing, email, and productivity suite use, from the user's primary, non-privileged account.	●	●	●
v7	4.3 Ensure the Use of Dedicated Administrative Accounts Ensure that all users with administrative account access use a dedicated or secondary account for elevated activities. This account should only be used for administrative activities and not internet browsing, email, or similar activities.	●	●	●

4.2.5 Minimize the admission of containers with allowPrivilegeEscalation (Automated)

Profile Applicability:

- Level 1

Description:

Minimize the admission of containers with `securityContext.allowPrivilegeEscalation: true` in EKS so that general workload namespaces do not permit processes to gain more privileges than their parent process. In current EKS guidance, AWS recommends applying Pod Security Standards through Pod Security Admission or a policy engine, using at least the baseline profile cluster-wide and the `restricted` profile for workload namespaces. AWS's Pod Security guidance shows that workloads violating `restricted` are rejected unless the container explicitly sets `securityContext.allowPrivilegeEscalation=false`.

It's important to note that these rights are still constrained by the overall container sandbox, and this setting does not relate to the use of privileged containers.

Rationale:

AWS states that pod specification settings can strengthen or weaken the cluster security posture and that a primary goal is to prevent a process running in a container from escaping container runtime isolation and gaining access to the underlying host. AWS also explains that privileged escalation allows a process to change the security context under which it runs, such as through sudo or binaries with the `SUID` or `SGID` bit. In EKS, this can be controlled at admission time by setting `securityContext.allowPrivilegeEscalation=false` in the pod spec or by using Pod Security Standards or policy-as-code so noncompliant API requests can be mutated or rejected before they are persisted.

Impact:

In namespaces enforcing the restricted Pod Security Standard, containers that do not set `securityContext.allowPrivilegeEscalation=false` can be denied at admission, so workloads that rely on privilege-changing behavior will need manifest updates or a tightly scoped exception path. AWS also notes that `warn` and `audit` modes only report violations, while `enforce` is the mode that blocks pod creation.

Pods defined with `spec.allowPrivilegeEscalation: true` will not be permitted unless they are run under a specific policy.

Audit:

Review the effective Pod Security Admission configuration for every namespace in the EKS cluster and verify that workload namespaces enforce a profile that requires `securityContext.allowPrivilegeEscalation=false` for containers. In current EKS guidance, AWS recommends applying at least the `baseline` profile cluster-wide and the `restricted` profile for workload namespaces. For this specific control, the `restricted` profile is the relevant check, and only `enforce` blocks noncompliant pods. If your cluster uses a policy-as-code engine instead of Pod Security Admission, review the equivalent validation policy that rejects containers which do not explicitly set `allowPrivilegeEscalation=false`.

Inspect running pods for containers, init containers, and ephemeral containers where `allowPrivilegeEscalation` is either `true` or `unset`, because restricted requires the value to be explicitly `false`:

```
kubectl get pods -A -o json | jq -r '
  .items[]
  | .metadata.namespace as $ns
  | .metadata.name as $pod
  | def scan($type; $list):
      $list[]?
      | select((.securityContext.allowPrivilegeEscalation // "unset") !=
false)
      |
"\($ns)/\($pod)\t\($type)=\(.name)\tallowPrivilegeEscalation=\(.securityConte
xt.allowPrivilegeEscalation // "unset")";
    scan("container"; .spec.containers),
    scan("initContainer"; .spec.initContainers),
    scan("ephemeralContainer"; .spec.ephemeralContainers)
'
```

This pod query identifies workloads already admitted that would not satisfy the restricted requirement, which matters because tightening namespace enforcement does not remove pods that are already running.

Remediation:

Configure Pod Security Admission on workload namespaces so containers that do not explicitly set `securityContext.allowPrivilegeEscalation=false` are denied at admission time. In EKS, use `restricted` for workload namespaces, or an equivalent policy engine, because AWS shows that `restricted` requires `allowPrivilegeEscalation=false`. Use enforce for actual remediation, since `warn` and `audit` only report violations and do not block pod creation.

- Label each workload namespace to enforce restricted.

```
kubectl label ns NAMESPACE pod-security.kubernetes.io/enforce=restricted --
overwrite
```

- Add `warn` and `audit` during rollout if you want visibility into violations before or alongside enforcement.

```
kubectl label ns NAMESPACE pod-security.kubernetes.io/warn=restricted --
overwrite
kubectl label ns NAMESPACE pod-security.kubernetes.io/audit=restricted --
overwrite
```

- Update workload manifests so every regular container, init container, and ephemeral container explicitly sets:

```
securityContext:
   allowPrivilegeEscalation: false
```

- Redeploy the affected workloads so new pods are evaluated under the enforced policy.
- Review any exceptions separately and keep them tightly scoped, since AWS documents exemptions as explicit configuration rather than something to assume by default. Existing pods are not deleted automatically when a namespace is tightened to a more restrictive profile.

Default Value:

By default, there are no restrictions on contained process ability to escalate privileges, within the context of the container.

References:

1. https://docs.aws.amazon.com/eks/latest/best-practices/pod-security.html
2. https://docs.aws.amazon.com/eks/latest/best-practices/security.html
3. https://docs.aws.amazon.com/eks/latest/userguide/security-best-practices.html
4. https://docs.aws.amazon.com/eks/latest/userguide/security.html

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	5.4 Restrict Administrator Privileges to Dedicated Administrator Accounts Restrict administrator privileges to dedicated administrator accounts on enterprise assets. Conduct general computing activities, such as internet browsing, email, and productivity suite use, from the user's primary, non-privileged account.	●	●	●

Controls Version	Control	IG 1	IG 2	IG 3
v7	4.3 Ensure the Use of Dedicated Administrative Accounts Ensure that all users with administrative account access use a dedicated or secondary account for elevated activities. This account should only be used for administrative activities and not internet browsing, email, or similar activities.	●	●	●

4.3 Amazon VPC CNI plugin for Kubernetes

The Amazon VPC CNI plugin for Kubernetes is the EKS networking add-on that provides native VPC networking for Pods by assigning VPC IP addresses and configuring Pod networking on each node. AWS describes it as the default EKS-supported CNI for EC2 worker nodes and the component used to provide Pod networking features such as IP assignment and, in supported versions, network policy enforcement.

4.3.1 Ensure VPC CNI network policy support is enabled (Automated)

Profile Applicability:

- Level 1

Description:

The Amazon VPC CNI plugin for Kubernetes can provide native Kubernetes `NetworkPolicy` enforcement in EKS when network policy support is enabled on a supported VPC CNI add-on version. This capability lets EKS apply Pod-to-Pod traffic restrictions through the VPC CNI data path instead of leaving Pod network communication broadly open by default.

Rationale:

Kubernetes `NetworkPolicy` objects do not provide traffic isolation in EKS unless the Amazon VPC CNI network policy feature is enabled and the cluster is running a supported VPC CNI add-on version. Policy enforcement is implemented on the node through the `aws-node` DaemonSet, with startup behavior controlled by `NETWORK_POLICY_ENFORCING_MODE`. In standard mode, newly started Pods come up with default-allow traffic until their active policies are attached. In strict mode, Pods start with default-deny behavior and only gain connectivity allowed by policy, which means required dependencies such as CoreDNS must already be permitted. Enforcement uses eBPF on the node, which improves the efficiency and visibility of policy processing compared with relying only on sequential packet filtering rules.

Impact:

If VPC CNI network policy support is not enabled, Pod-to-Pod traffic remains unrestricted by this feature, so namespace segmentation and least-privilege traffic controls are not enforced through the Amazon VPC CNI plugin. Even when enabled, the feature applies to Amazon EC2 Linux nodes, not Fargate or Windows nodes, and policies are applied only to the Pod's primary interface.

Audit:

Review the current Amazon VPC CNI configuration for the cluster and verify that network policy support is enabled. In EKS, this feature is not enabled by default, and a valid audit must confirm both that the cluster is running a supported VPC CNI version and that the VPC CNI configuration has network policy turned on. For Amazon EKS add-ons, the key setting is `enableNetworkPolicy=true` on the `vpc-cni` add-on. For self-managed installs, review the aws-node DaemonSet and confirm the network policy agent is enabled in the manifest.

Use this command for an Amazon EKS managed add-on audit:

```
aws eks describe-addon \
  --cluster-name $CLUSTER_NAME \
  --addon-name vpc-cni \
  --query
'addon.{status:status,version:addonVersion,configurationValues:configurationV
alues}' \
  --output json
```

Confirm the add-on is `ACTIVE`, the version is supported for network policy use, and `configurationValues` includes `enableNetworkPolicy` set to `true`. The EKS CLI returns the add-on version and the configuration values you supplied, which makes this the most direct audit for managed vpc-cni.

If the cluster uses a self-managed VPC CNI, use these checks:

```
kubectl describe daemonset aws-node -n kube-system | grep amazon-k8s-cni:
kubectl get daemonset aws-node -n kube-system -o yaml | grep -- '--enable-
network-policy='
```

The first command verifies the installed VPC CNI version. The second checks whether the network policy agent is enabled in the `aws-node` DaemonSet manifest. AWS's current setup guidance requires both a supported VPC CNI version and network policy enabled in the VPC CNI configuration.

Remediation:

Enable network policy support in the Amazon VPC CNI plugin for Kubernetes so EKS can enforce Kubernetes `NetworkPolicy` objects for Pod traffic. This feature is not enabled by default on newly provisioned clusters, and remediation requires a supported VPC CNI version plus network policy enabled in the VPC CNI configuration. For managed add-ons, set `enableNetworkPolicy` to `true` on the `vpc-cni` add-on. For self-managed installations, enable the network policy controller in the `amazon-vpc-cni` ConfigMap and set the `aws-node` DaemonSet argument `--enable-network-policy=true`.

AWS CLI, managed add-on: Update the managed `vpc-cni` add-on and enable the feature. AWS's current guide shows v1.14.0-eksbuild.3 or later in the example.

```
aws eks update-addon \
  --cluster-name $CLUSTER_NAME \
  --addon-name vpc-cni \
  --addon-version v1.14.0-eksbuild.3 \
  --resolve-conflicts PRESERVE \
  --configuration-values '{"enableNetworkPolicy":"true"}'
```

Command line, self-managed VPC CNI: Edit the amazon-vpc-cni ConfigMap and add:

```
enable-network-policy-controller: "true"
```

Then edit the `aws-node` DaemonSet and set:

```
--enable-network-policy=true
```

Apply the required Kubernetes `NetworkPolicy` objects. Standard mode starts Pods as default-allow, while strict mode starts them as default-deny and requires necessary traffic such as DNS to be explicitly permitted.

Default Value:

The default for this control is disabled. When you first provision an EKS cluster, VPC CNI network policy functionality is not enabled by default. AWS says you must use a supported VPC CNI add-on version and set the network policy flag to `true` to enable it.

References:

1. https://docs.aws.amazon.com/eks/latest/userguide/cni-network-policy-configure.html
2. https://docs.aws.amazon.com/eks/latest/best-practices/network-security.html
3. https://docs.aws.amazon.com/eks/latest/userguide/cni-network-policy.html
4. https://docs.aws.amazon.com/eks/latest/userguide/managing-vpc-cni.html

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	7.4 Perform Automated Application Patch Management Perform application updates on enterprise assets through automated patch management on a monthly, or more frequent, basis.	●	●	●
v7	18.4 Only Use Up-to-date And Trusted Third-Party Components Only use up-to-date and trusted third-party components for the software developed by the organization.		●	●

4.3.2 Ensure required workloads are covered by Kubernetes network policies (Manual)

Profile Applicability:

- Level 1

Description:

The Amazon VPC CNI plugin for Kubernetes can enforce Kubernetes network policies in EKS, but workload isolation depends on the relevant workloads actually being covered by `NetworkPolicy` or `ClusterNetworkPolicy` rules. `NetworkPolicy` provides namespace-scoped pod traffic segmentation, while `ClusterNetworkPolicy` can apply a common security posture across all or a selected subset of workloads in the cluster.

Rationale:

Turning on VPC CNI network policy support only provides the enforcement mechanism. Traffic restrictions are not realized unless policies are defined for the workloads that need them. Pod-to-Pod communication is allowed by default, and AWS recommends a least-privilege approach that starts with a default deny policy and then adds explicit ingress and egress allow rules. Namespace-scoped policies control traffic within a namespace, while cluster-level policies centralize controls across multiple namespaces or workload groups. When no matching policies allow the traffic, the evaluation model ends in deny-by-default.

Impact:

If required workloads are not covered by Kubernetes network policies, their traffic remains effectively unsegmented even when Amazon VPC CNI network policy support is enabled, which leaves unnecessary communication paths open. Coverage also has practical limits because policies apply to Amazon EC2 Linux nodes, not Fargate or Windows nodes, and they apply only to a Pod's primary network interface.

Audit:

Review the Kubernetes network policy model in the EKS cluster and verify that every required workload is selected by either a namespace-scoped `NetworkPolicy` or a cluster-scoped `ClusterNetworkPolicy`. A separate policy in every namespace is not required, because cluster-scoped policies can apply a shared posture across all workloads or a selected subset. Workloads that are not matched by policy remain effectively unsegmented because Pod traffic is allowed by default until policy restricts it.

- Identify all `NetworkPolicy` and `ClusterNetworkPolicy` objects intended to govern workload traffic. `ClusterNetworkPolicy` can cover all workloads or selected workloads across namespaces.

- Map each required workload to the policy or policies that select it, and confirm no required workload falls outside policy coverage. Unmatched workloads remain default-allow.
- Review selectors and ingress and egress rules to confirm least-privilege design. A strong pattern is default deny first, then explicit allow rules for only necessary flows such as DNS and approved service-to-service traffic.
- Confirm the workloads are in supported enforcement scope. Native VPC CNI network policies apply to Amazon EC2 Linux nodes, not Fargate or Windows nodes, and they apply only to a Pod's primary interface.
- Check for workload types that may not be covered as expected, including standalone Pods, because this feature applies policies only to Pods created by Kubernetes Deployments.

Review policy objects, map them to required workloads, and confirm no required workload is left outside enforceable network policy coverage.

Remediation:

Create and apply Kubernetes `NetworkPolicy` or `ClusterNetworkPolicy` objects so every required workload is explicitly covered by policy. Start with a default deny posture and then add only the ingress and egress rules needed for approved traffic such as DNS and required service-to-service flows. Use namespace-scoped`NetworkPolicy` where application-level segmentation is sufficient, and use `ClusterNetworkPolicy` where a shared security posture must apply across multiple namespaces or workload groups. Keep in mind that coverage depends on the Amazon VPC CNI network policy feature being enabled, applies to Amazon EC2 Linux nodes, and is limited to a Pod's primary interface.

Default Value:

By default, network policies are not created.

References:

1. https://docs.aws.amazon.com/eks/latest/userguide/cni-network-policy.html
2. https://docs.aws.amazon.com/eks/latest/best-practices/network-security.html
3. https://docs.aws.amazon.com/eks/latest/userguide/cni-network-policy-configure.html
4. https://docs.aws.amazon.com/eks/latest/userguide/auto-net-pol.html

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	4.4 Implement and Manage a Firewall on Servers Implement and manage a firewall on servers, where supported. Example implementations include a virtual firewall, operating system firewall, or a third-party firewall agent.	●	●	●
v7	14.1 Segment the Network Based on Sensitivity Segment the network based on the label or classification level of the information stored on the servers, locate all sensitive information on separated Virtual Local Area Networks (VLANs).		●	●
v7	14.2 Enable Firewall Filtering Between VLANs Enable firewall filtering between VLANs to ensure that only authorized systems are able to communicate with other systems necessary to fulfill their specific responsibilities.		●	●

4.4 Secrets Management

Secrets management in EKS covers both protecting secret data stored by the cluster and securely delivering application secrets to Pods. EKS provides envelope encryption for Kubernetes API data, and a stronger application pattern is often to store secrets in AWS Secrets Manager and deliver them to Pods with IAM-scoped access by using EKS Pod Identity or IRSA, often through the Secrets Store CSI Driver.

A practical approach is to use Kubernetes Secrets only when needed for compatibility, prefer mounted secrets over environment variables, and keep rotation, access control, and auditing in AWS wherever possible.

4.4.1 Prefer exposing secrets as mounted files rather than environment variables (Automated)

Profile Applicability:

- Level 1

Description:

Secrets for EKS workloads are better exposed as mounted files than as environment variables. Kubernetes secrets can be consumed either way, and AWS supports file-based delivery from AWS Secrets Manager and Parameter Store through the AWS Secrets and Configuration Provider with the Secrets Store CSI Driver.

Rationale:

Environment variable values can unintentionally appear in logs, while secrets mounted as volumes are instantiated as `tmpfs` volumes and are automatically removed from the node when the Pod is deleted. File-based delivery also aligns with the AWS Secrets and Configuration Provider model, which mounts Secrets Manager and Parameter Store values into EKS Pods as files and can work with secret rotation through the CSI Driver rotation reconciler.

Impact:

Workloads that currently read secrets from environment variables may need manifest and application changes to consume secrets from mounted file paths instead, using Kubernetes secret volumes or the Secrets Store CSI Driver integration with AWS secrets stores.

Audit:

Review workload manifests and identify any Pod, Deployment, DaemonSet, StatefulSet, Job, or CronJob that exposes secrets through environment variables instead of mounted files. Check for both direct secret references through `env[].valueFrom.secretKeyRef` and bulk secret imports through `envFrom[].secretRef`, because either pattern exposes secret data as environment variables. Prefer workloads that consume secrets through mounted Secret volumes or file-based delivery from AWS secrets integrations instead.

Run this command to list workloads that reference secrets through either environment-variable pattern:

```
kubectl get deploy,ds,sts,job,cronjob,pod -A -o json | jq -r '
  .items[]
  | .kind as $kind
  | .metadata.namespace as $ns
  | .metadata.name as $name
  | (if $kind == "Pod" then .spec else .spec.template.spec end) as $spec
  | select([ $spec.containers[]?, $spec.initContainers[]? ] |
any(.env[]?.valueFrom.secretKeyRef? or .envFrom[]?.secretRef?))
  | "\($ns)\t\($kind)/\($name)"
'
```

Remediation:

Update workloads to consume secrets as mounted files instead of environment variables. Replace `env[].valueFrom.secretKeyRef` and `envFrom[].secretRef` usage with Kubernetes Secret volumes or the AWS Secrets and Configuration Provider with the Secrets Store CSI Driver, then update the application to read the secret from the mounted file path. This reduces the chance of secret values appearing in logs and aligns with the AWS-recommended file-based delivery pattern for EKS workloads.

Replace secret environment variables with mounted secret files and update the application to read from the mounted path.

Default Value:

By default, secrets are not defined

References:

1. https://docs.aws.amazon.com/eks/latest/best-practices/data-encryption-and-secrets-management.html
2. https://docs.aws.amazon.com/eks/latest/userguide/manage-secrets.html
3. https://docs.aws.amazon.com/secretsmanager/latest/userguide/ascp-eks-installation.html
4. https://docs.aws.amazon.com/secretsmanager/latest/userguide/integrating_ascp_irsa.html

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	3.10 Encrypt Sensitive Data in Transit Encrypt sensitive data in transit. Example implementations can include: Transport Layer Security (TLS) and Open Secure Shell (OpenSSH).		●	●
v7	14.4 Encrypt All Sensitive Information in Transit Encrypt all sensitive information in transit.		●	●

Controls Version	Control	IG 1	IG 2	IG 3
v7	14.8 Encrypt Sensitive Information at Rest Encrypt all sensitive information at rest using a tool that requires a secondary authentication mechanism not integrated into the operating system, in order to access the information.			●

4.4.2 Consider external secret storage (Manual)

Profile Applicability:

- Level 1

Description:

External secret storage for EKS means keeping application secrets outside native Kubernetes Secret objects and retrieving them from a managed service such as AWS Secrets Manager when workloads need them. This approach lets EKS workloads consume secrets through integrations such as the AWS Secrets and Configuration Provider with the Secrets Store CSI Driver, instead of relying only on Kubernetes to store and distribute sensitive values.

Rationale:

Storing high-value credentials outside Kubernetes reduces dependence on cluster-resident secret objects and shifts lifecycle functions such as encryption, access control, rotation, and auditability to a dedicated secrets service. In EKS, Pods can retrieve secrets from Secrets Manager by using IAM-scoped access through EKS Pod Identity or IRSA, and the AWS Secrets and Configuration Provider can mount those values into Pods as files through the Secrets Store CSI Driver. This model supports automatic rotation workflows and tighter secret-to-workload scoping than broadly distributing static values through Kubernetes manifests.

Impact:

Moving to external secret storage usually requires manifest and application changes so workloads fetch or mount secrets from Secrets Manager instead of reading only native Kubernetes Secret objects, and it may require Pod Identity or IRSA permissions plus CSI-based integration components. In return, secret rotation and access management can be handled centrally in AWS rather than separately inside each cluster.

Audit:

Review the EKS cluster's secrets architecture and verify that required workloads use an external secrets store such as AWS Secrets Manager instead of relying only on native Kubernetes Secret objects. The audit should confirm both the integration path and the workload-level access model, because external secret storage in EKS typically depends on the AWS Secrets and Configuration Provider, the Secrets Store CSI Driver, and Pod-scoped IAM through EKS Pod Identity or IRSA.

- Identify workloads that still consume high-value credentials from native Kubernetes Secrets and determine which of those should be moved to AWS Secrets Manager or Parameter Store.
- Confirm the external secret integration components are in use, including the AWS Secrets and Configuration Provider and Secrets Store CSI Driver, and

verify that workloads reference a `SecretProviderClass` where external secrets are mounted as files.
- Verify Pod-level access is scoped through EKS Pod Identity or IRSA so each workload can retrieve only the secrets it is authorized to read.
- Check whether rotation, access control, and audit requirements are being handled in the external store rather than duplicated through static secret values embedded in Kubernetes manifests or long-lived native Secret objects.

Remediation:

Move high-value application secrets out of native Kubernetes Secret objects and into an external store such as AWS Secrets Manager. Then update EKS workloads to retrieve those secrets through the AWS Secrets and Configuration Provider with the Secrets Store CSI Driver, using Pod-scoped IAM through EKS Pod Identity or IRSA. This shifts encryption, access control, rotation, and auditing to AWS instead of leaving those functions primarily in cluster-resident Secret objects.

- Create the required secrets in AWS Secrets Manager and define workload access with least-privilege IAM policies.
- Install and configure the AWS Secrets and Configuration Provider and the Secrets Store CSI Driver for EKS.
- Update workload manifests to mount external secrets through a `SecretProviderClass` instead of reading only native Kubernetes Secret objects.
- Enable rotation and update the application to read secrets from the mounted path so secret lifecycle management is handled centrally in AWS.

Move secrets to AWS Secrets Manager, mount them into Pods, and scope access with Pod Identity or IRSA.

Default Value:

By default, no external secret management is configured.

References:

1. https://docs.aws.amazon.com/eks/latest/best-practices/data-encryption-and-secrets-management.html
2. https://docs.aws.amazon.com/eks/latest/userguide/manage-secrets.html
3. https://docs.aws.amazon.com/secretsmanager/latest/userguide/ascp-eks-installation.html
4. https://docs.aws.amazon.com/secretsmanager/latest/userguide/integrating_ascp_irsa.html

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	3.11 Encrypt Sensitive Data at Rest Encrypt sensitive data at rest on servers, applications, and databases containing sensitive data. Storage-layer encryption, also known as server-side encryption, meets the minimum requirement of this Safeguard. Additional encryption methods may include application-layer encryption, also known as client-side encryption, where access to the data storage device(s) does not permit access to the plain-text data.		●	●
v7	14.8 Encrypt Sensitive Information at Rest Encrypt all sensitive information at rest using a tool that requires a secondary authentication mechanism not integrated into the operating system, in order to access the information.			●

4.5 General Policies

These policies relate to general cluster management topics, like namespace best practices and policies applied to pod objects in the cluster.

4.5.1 Create administrative boundaries between resources using namespaces (Manual)

Profile Applicability:

- Level 1

Description:

Namespaces create administrative boundaries in EKS by dividing a cluster into logical partitions for teams, applications, or tenants. They are a core building block for soft multi-tenancy, and several controls used to separate workloads, including quotas, network policies, and service accounts, are scoped to the namespace.

Rationale:

Namespaces matter operationally because many access and governance controls in Kubernetes and EKS can be applied at that scope. Kubernetes `Role` and `RoleBinding` objects are used to delegate permissions within a namespace, and EKS access policies can be scoped to one or more specific namespaces instead of the entire cluster. That allows platform teams to grant administrative access to only the namespaces a team or workload should manage, rather than granting cluster-wide permissions.

The namespace boundary becomes stronger when it is combined with namespace-scoped resource and traffic controls. `ResourceQuota` and `LimitRange` can constrain CPU, memory, and object consumption within a namespace, while network policies can restrict Pod-to-Pod communication and support a least-privilege model between tenant or application namespaces. A namespace alone is still only a logical grouping construct, so effective administrative separation depends on pairing it with RBAC, access scoping, quotas, service accounts, and network isolation.

Impact:

Creating administrative boundaries with namespaces usually requires teams to move workloads out of the `default` namespace and implement namespace-specific RBAC, access scopes, quotas, and network policies. This improves delegation and organization, but namespaces by themselves do not provide full isolation.

Audit:

Verify that namespaces are present and are being used as intentional administrative boundaries for teams, applications, or environments in the EKS cluster. Namespaces are the core logical partition for soft multi-tenancy, and they should be paired with scoped access controls and namespace-level governance such as quotas, limits, and network policies. A namespace should be considered adequately administered only when it has a clear workload or team purpose, appropriately scoped access, and the namespace-level controls required for that use case.

- List all namespaces and confirm each has a defined administrative, application, or environment purpose. Namespaces are the base partitioning construct for soft multi-tenancy in EKS.
- Verify that user and workload access is scoped to the intended namespaces through Kubernetes `Role` and `RoleBinding` objects and, where used, EKS access entries and access policies rather than broad cluster-wide grants. EKS access policies can be scoped to specific namespaces.
- Review whether namespace-level controls such as `ResourceQuota`, `LimitRange`, service accounts, and network policies are applied where required to support separation, resource governance, and east-west traffic restrictions.
- Confirm that cluster-wide access is limited to identities that truly require it, because namespace boundaries alone do not prevent cross-tenant or cross-workload access.

Inventory checks for EKS Clusters:

```
kubectl get namespaces

kubectl get rolebindings,roles -A
kubectl get clusterrolebindings,clusterroles

kubectl get resourcequota,limitrange,networkpolicy -A

aws eks list-access-entries --cluster-name $CLUSTER_NAME
aws eks list-associated-access-policies --cluster-name $CLUSTER_NAME --
principal-arn $PRINCIPAL_ARN
```

Note: Command line statement to extract arn for $PRINCIPAL_ARN varriable:

```
aws sts get-caller-identity --query Arn --output text
```

Remediation:

Create separate namespaces for teams, applications, or environments that require their own administrative scope, and move workloads out of the default namespace into that managed namespace structure. Make the namespace boundary meaningful by scoping access to the intended namespaces, applying namespace-level governance such as quotas, limits, service accounts, and network policies, and reducing cluster-wide permissions that bypass the intended separation model. When stronger isolation is required for mutually untrusted workloads, use separate clusters rather than relying on namespaces alone.

1. Create dedicated namespaces for each team, application, or environment that needs its own administrative boundary.
2. Move user workloads out of the `default` namespace and into the appropriate managed namespaces.
3. Rebind access with namespace-scoped `Role` and `RoleBinding` objects and, where used, scope EKS access policies to one or more specific namespaces instead of the whole cluster.

4. Apply `ResourceQuota`, `LimitRange`, service accounts, and network policies to reinforce namespace-level governance and traffic separation.
5. Remove unused namespaces and reduce unnecessary cluster-wide permissions or access entries that bypass the intended namespace boundary.

Create dedicated namespaces, move workloads out of `default`, scope access by namespace, and apply quotas, limits, service accounts, and network policies to enforce the administrative boundary.

Default Value:

By default, Kubernetes starts with four initial namespaces:

- default — the default namespace for objects that do not specify another namespace
- kube-system — the namespace for objects created by the Kubernetes system
- kube-public — a namespace readable by all clients, mostly reserved for cluster usage
- kube-node-lease — the namespace that holds Lease objects for each node so kubelets can send heartbeats

References:

1. https://docs.aws.amazon.com/eks/latest/best-practices/tenant-isolation.html
2. https://docs.aws.amazon.com/eks/latest/userguide/access-policies.html
3. https://docs.aws.amazon.com/eks/latest/userguide/cluster-auth.html
4. https://docs.aws.amazon.com/wellarchitected/latest/saas-lens/amazon-eks-saas.html

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	12.8 Establish and Maintain Dedicated Computing Resources for All Administrative Work Establish and maintain dedicated computing resources, either physically or logically separated, for all administrative tasks or tasks requiring administrative access. The computing resources should be segmented from the enterprise's primary network and not be allowed internet access.			●
v7	12.1 Maintain an Inventory of Network Boundaries Maintain an up-to-date inventory of all of the organization's network boundaries.	●	●	●

4.5.2 The default namespace should not be used (Automated)

Profile Applicability:

- Level 1

Description:

Avoid placing application workloads in the `default` namespace, and instead deploy them into dedicated namespaces that reflect the intended team, application, or environment boundary. In EKS, namespaces are the primary logical partition for organizing workloads and supporting soft multi-tenancy in a shared cluster.

Rationale:

Least-privilege administration in EKS depends on using namespaces as intentional operational boundaries. Kubernetes `Role` and `RoleBinding` objects apply at namespace scope, and EKS access policies can be scoped to one or more specific namespaces instead of the whole cluster. When workloads remain in `default`, access delegation, ownership boundaries, and namespace-scoped governance become harder to separate cleanly.

Impact:

Using the `default` namespace for application workloads weakens administrative separation and makes namespace-scoped access control, resource governance, and network segmentation harder to apply consistently.

Audit:

Review the EKS cluster and verify that the `default` namespace is not being used for normal application administration or routine workload placement. Objects that do not explicitly specify a namespace are created in `default`, so the audit should confirm that teams, applications, and environments are instead using dedicated namespaces that align with namespace-scoped access control, quotas, limits, service accounts, and network policies. Namespaces are a soft multi-tenancy boundary in EKS, so the goal is to confirm that user workloads are intentionally placed outside `default` and governed through namespace-specific controls.

- Inventory the `default` namespace and identify any user-managed Pods, Services, ConfigMaps, Secrets, ServiceAccounts, Deployments, StatefulSets, DaemonSets, Jobs, CronJobs, Ingresses, Roles, RoleBindings, ResourceQuotas, LimitRanges, or NetworkPolicies that still exist there.

```
kubectl get
pod,svc,configmap,secret,serviceaccount,deploy,statefulset,daemonset,job,cron
job,ingress,role,rolebinding,resourcequota,limitrange,networkpolicy -n
default
```

- Verify that access is not effectively centered on `default`. Review Kubernetes `Role` and `RoleBinding` assignments for that namespace and, where EKS access entries are used, confirm that access policies are scoped to the intended namespaces rather than broadly applied cluster-wide. EKS access policies support namespace-scoped access.
- Confirm that user workloads have been placed into dedicated namespaces that reflect team, application, or environment boundaries, and that namespace-level controls such as quotas, limits, service accounts, and network policies are applied where required. Those controls are what make a namespace an actual administrative boundary instead of just a label.
- Check whether the namespace design reduces shared administrative scope and naming sprawl. AWS recommends using namespaces as the logical separation mechanism for soft multi-tenancy, while Kubernetes namespaces also provide the uniqueness boundary for namespaced resources.

Inventory the default namespace, identify user workloads still running there, and confirm access and governance are instead aligned to dedicated namespaces.

Remediation:

Move user workloads out of the `default` namespace and place them in dedicated namespaces that reflect the intended team, application, or environment boundary. In EKS, namespaces are the core logical partition for soft multi-tenancy, but they only become meaningful administrative boundaries when they are paired with namespace-scoped access, service accounts, quotas, limits, and network policies. Because namespaced objects are created in `default` when no namespace is specified, remediation should address both existing workloads and the deployment process that created them there.

- Create dedicated namespaces for each team, application, or environment that needs its own administrative scope, then redeploy or migrate user workloads from default into those namespaces.
- Update manifests, Helm values, GitOps definitions, and pipeline defaults so namespaced resources explicitly set `metadata.namespace` and are not created in `default` by omission.
- Re-scope access with namespace-scoped Kubernetes `Role` and `RoleBinding` objects and, where EKS access entries are used, associate EKS access policies to specific namespaces instead of the whole cluster.
- Apply namespace-level governance such as `ResourceQuota`, `LimitRange`, dedicated service accounts, and network policies so the namespace boundary supports resource control, identity separation, and east-west traffic isolation.

- Remove unnecessary user workloads and broad operational access from `default`, and use separate clusters rather than namespaces alone when stronger isolation is required for less-trusted workloads.

Move workloads out of `default`, enforce explicit namespaces in deployment workflows, and bind access and governance controls to the target namespaces.

Default Value:

Unless a namespace is specific on object creation, the `default` namespace will be used

References:

1. https://docs.aws.amazon.com/eks/latest/best-practices/tenant-isolation.html
2. https://docs.aws.amazon.com/eks/latest/userguide/access-policies.html
3. https://docs.aws.amazon.com/eks/latest/userguide/kubernetes-concepts.html
4. https://docs.aws.amazon.com/eks/latest/userguide/cluster-auth.html

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	16.7 Use Standard Hardening Configuration Templates for Application Infrastructure Use standard, industry-recommended hardening configuration templates for application infrastructure components. This includes underlying servers, databases, and web servers, and applies to cloud containers, Platform as a Service (PaaS) components, and SaaS components. Do not allow in-house developed software to weaken configuration hardening.		●	●
v7	5.1 Establish Secure Configurations Maintain documented, standard security configuration standards for all authorized operating systems and software.	●	●	●

5 Managed services

This section consists of security recommendations for the Amazon EKS. These recommendations are applicable for configurations that Amazon EKS customers own and manage.

5.1 Image Registry and Image Scanning

This section contains recommendations relating to container image registries and securing images in those registries, such as Amazon Elastic Container Registry (ECR).

5.1.1 Ensure Image Vulnerability Scanning using Amazon ECR image scanning or a third party provider (Automated)

Profile Applicability:

- Level 1

Description:

Container image vulnerability scanning helps strengthen the security of EKS workloads by identifying known software weaknesses in the images those workloads run. This supports a more secure software supply chain and helps reduce the chance that vulnerable images are promoted into Kubernetes environments.

Rationale:

EKS best practices recommend scanning images in Amazon ECR and rebuilding or deleting images with High or Critical findings. In AWS, this can be implemented with ECR Basic Scanning or with ECR Enhanced Scanning, where Enhanced Scanning integrates with Amazon Inspector for continuous scanning and expanded coverage that includes operating system and programming language package vulnerabilities. AWS also documents that enhanced scanning can show how images are used in EKS, which helps teams identify affected clusters and prioritize remediation for images that are actually running in production.

Vulnerabilities in container image software packages can be exploited to compromise workloads and potentially gain unauthorized access to cluster, node, or connected cloud resources. Amazon ECR and approved third-party tools can scan container images for known vulnerabilities.

Impact:

This control improves detection and prioritization of vulnerable container images, but it also adds operational overhead to review findings, rebuild images, and manage registry scan configuration. In Enhanced Scanning, coverage depends on registry scanning rules and monitoring windows, so repositories outside the configured filters or images outside the active rescan scope may not continue to receive scans.

Audit:

Verify that Amazon ECR vulnerability scanning is enabled for repositories that store images used by EKS workloads by reviewing the registry scanning configuration and, where needed, the repository scanning configuration. Confirm whether the registry is using Basic or Enhanced Scanning and that the applicable repositories are included in the scanning scope. If a third-party scanner is used instead, obtain equivalent evidence that EKS-bound images are scanned and that findings are available for review.

- Checks the registry-wide scanning setting for the Region. Use it to confirm the scan type and any registry scan rules:

```
aws ecr get-registry-scanning-configuration \
  --region $REGION_CODE
```

Expected output: JSON showing the `registryId` and `scanningConfiguration`. In that object, look for `scanType` such as `BASIC` or `ENHANCED`, and if rules are present, review `rules`, `repositoryFilters`, and `scanFrequency` to see which repositories are covered and how they are scanned.

- Checks the scanning configuration for the specific repository. Use it to confirm that the repository used by EKS is actually covered:

```
aws ecr batch-get-repository-scanning-configuration \
  --repository-names $REPO_NAME \
  --region $REGION_CODE
```

Expected output: JSON with `scanningConfigurations` for the requested repository and possibly failures if a repository name is invalid or not found. In `scanningConfigurations`, look for `repositoryName`, `repositoryArn`, `scanOnPush`, `scanFrequency`, and `appliedScanFilters` to confirm the effective scan behavior for that repository.

Remediation:

Enable image vulnerability scanning in ECR for the repositories that supply container images to EKS workloads. Configure scanning at the private registry level in each Region, because ECR manages scan behavior through registry settings. Prefer Enhanced Scanning so Amazon Inspector can continuously scan container images for operating system and programming language package vulnerabilities. If Basic Scanning is used instead, ensure the repositories that store EKS images are covered by scan on push rules or are manually scanned before image promotion and deployment. EKS guidance recommends regularly scanning images in ECR or with a commercial scanner, and AWS notes that the older repository-level `put-image-scanning-configuration` path is being deprecated in favor of registry-level configuration.

- Identify the ECR repositories and Regions that provide images to EKS workloads. This establishes which private registries must be included in the scanning scope.
- Configure the private registry scanning setting in each Region to use Enhanced or Basic scanning. Registry scanning settings are regional, so this must be set in each Region that stores images for EKS.

```
aws ecr put-registry-scanning-configuration \
  --scan-type ENHANCED \
  --region $REGION_CODE
```

- Prefer Enhanced Scanning and configure repository filters or rules so all repositories used by EKS are included. Enhanced Scanning integrates with Amazon Inspector, and the registry rules determine which repositories are scanned and at what frequency.

```
aws ecr put-registry-scanning-configuration \
  --scan-type ENHANCED \
  --rules
'[{"repositoryFilters":[{"filter":"prod","filterType":"WILDCARD"}],"scanFrequ
ency":"CONTINUOUS_SCAN"}]' \
  --region $REGION_CODE
```

- If Basic Scanning is used, configure scan on push for the required repositories. ECR supports Basic and Enhanced scanning, and scan-on-push behavior can be controlled through the registry scanning configuration.
- Review vulnerability findings and rebuild, replace, or prevent deployment of images with unacceptable risk before they are used by EKS workloads. This makes the control operational rather than leaving findings unaddressed.

Default Value:

Images are not scanned by Default.

References:

1. https://docs.aws.amazon.com/eks/latest/best-practices/image-security.html
2. https://docs.aws.amazon.com/AmazonECR/latest/userguide/image-scanning.html
3. https://docs.aws.amazon.com/AmazonECR/latest/userguide/image-scanning-enhanced-enabling.html
4. https://docs.aws.amazon.com/cli/latest/reference/ecr/put-registry-scanning-configuration.html

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	7.5 Perform Automated Vulnerability Scans of Internal Enterprise Assets Perform automated vulnerability scans of internal enterprise assets on a quarterly, or more frequent, basis. Conduct both authenticated and unauthenticated scans, using a SCAP-compliant vulnerability scanning tool.		●	●

Controls Version	Control	IG 1	IG 2	IG 3
v8	7.6 Perform Automated Vulnerability Scans of Externally-Exposed Enterprise Assets Perform automated vulnerability scans of externally-exposed enterprise assets using a SCAP-compliant vulnerability scanning tool. Perform scans on a monthly, or more frequent, basis.		●	●
v7	3.1 Run Automated Vulnerability Scanning Tools Utilize an up-to-date SCAP-compliant vulnerability scanning tool to automatically scan all systems on the network on a weekly or more frequent basis to identify all potential vulnerabilities on the organization's systems.		●	●
v7	3.2 Perform Authenticated Vulnerability Scanning Perform authenticated vulnerability scanning with agents running locally on each system or with remote scanners that are configured with elevated rights on the system being tested.		●	●

5.1.2 Ensure least-privilege access is enforced for Amazon ECR repositories used by EKS workloads (Manual)

Profile Applicability:

- Level 1

Description:

Least-privilege access for Amazon ECR repositories helps limit who can pull, push, delete, or administer container images used by EKS workloads. This reduces unnecessary exposure of the image registry and helps ensure that only the identities required to support cluster operation, image delivery, and approved build or release processes can access the repositories.

Rationale:

Amazon ECR access is enforced through a combination of IAM identity-based policies and ECR repository policies, and AWS recommends moving toward permissions that are specific to the use case rather than broad managed access. For EKS image pulls, worker node roles or Fargate pod execution roles only need the ECR actions required to authenticate and retrieve images, such as `ecr:GetAuthorizationToken`, `ecr:BatchGetImage`, `ecr:GetDownloadUrlForLayer`, and `ecr:BatchCheckLayerAvailability`. For image push, administration, or cross-account access, permissions should be scoped to the specific repository ARN and only the required actions should be granted.

Impact:

Enforcing this control usually requires separate IAM roles or policies for node image pulls, Fargate image pulls, CI/CD image publishing, and repository administration. If permissions are scoped too narrowly, EKS nodes, Fargate pods, or operational users may be unable to authenticate to ECR, pull images, or use repository functions as intended. When scoped correctly, the control reduces the risk of unauthorized image access or modification without disrupting required image delivery to EKS workloads.

Audit:

Review the IAM permissions and ECR repository policies that apply to repositories used by EKS workloads, and verify that access is limited to approved node roles, Fargate pod execution roles, CI/CD identities, and administrative roles. Confirm that pull access is limited to the minimum ECR actions required for image retrieval, that push or administrative access is scoped only to the repositories and actions required, and that no unnecessary wildcard or broad cross-account access is present. ECR access decisions can be affected by both IAM policies and repository policies, and EKS image pulls rely on the node role or Fargate pod execution role having the required ECR permissions.

- Identify the ECR repositories used by EKS workloads. Review only the repositories that actually supply images to the cluster.
- Review each repository policy for unnecessary principals or broad actions. Look for cross-account access, wildcard principals, or actions beyond the repository's intended use.

```
aws ecr get-repository-policy \
  --repository-name $REPO_NAME \
  --region $REGION_CODE
```

Expected output: JSON containing the repository policy text. Review the Principal, Action, and Resource entries to confirm access is limited to approved identities and required actions only.

- Review the IAM role used to pull images. For node-based workloads, review the node IAM role. For Fargate workloads, review the pod execution role. Confirm the role has only the pull permissions required for ECR access. AWS documents pull-related permissions such as `ecr:GetAuthorizationToken`, `ecr:BatchCheckLayerAvailability`, `ecr:BatchGetImage`, and `ecr:GetDownloadUrlForLayer`.

```
aws iam list-attached-role-policies \
  --role-name $ROLE_NAME
```

Expected output: JSON listing the managed policies attached to the role. Review the attached policy names and flag overly broad access such as full ECR administration where pull-only access would be sufficient.

- Flag broad access patterns. Examples include `ecr:*`, wildcard repository scope where it is not required, or broad managed policies where a narrower customer-managed policy would satisfy the use case. AWS recommends least-privilege identity-based policies tailored to the access requirement.

Remediation:

Enforce least-privilege access for ECR repositories used by EKS workloads by separating image pull, image push, and repository administration into different IAM roles and policy scopes. Grant node roles and Fargate pod execution roles only the ECR permissions needed to authenticate and pull images, such as `ecr:GetAuthorizationToken`, `ecr:BatchCheckLayerAvailability`, `ecr:BatchGetImage`, and `ecr:GetDownloadUrlForLayer`. Scope CI/CD and administrator access to the specific repositories and actions they require, and review both IAM identity-based policies and ECR repository policies because ECR authorization can be affected by both. AWS also provides the `AmazonEC2ContainerRegistryPullOnly` managed policy for pull-only use cases.

- Separate ECR access by role. Use different roles for node pulls, Fargate pulls, CI/CD pushes, and repository administration.

- Replace broad pull access with pull-only permissions where possible. Attach a narrower pull-only policy to runtime pull roles instead of broader ECR access.

```
aws iam attach-role-policy \
  --role-name $ROLE_NAME \
  --policy-arn arn:aws:iam::aws:policy/AmazonEC2ContainerRegistryPullOnly
```

- Scope push and admin access to specific repositories. Limit CI/CD and admin identities to the repository ARNs and ECR actions they actually need.
- Tighten repository policies only where needed. Use repository policies to restrict repository-level access, including approved cross-account access, but remember that `ecr:GetAuthorizationToken` must still come from IAM.

```
aws ecr set-repository-policy \
  --repository-name $REPO_NAME \
  --policy-text file://repo-policy.json \
  --region $REGION_CODE
```

- Keep runtime pull roles limited to pull actions only. Do not give node or Fargate pull roles push, delete, or policy-management rights unless there is a specific operational requirement.
- Retest image delivery after policy changes. Confirm workloads can still pull required images and that unauthorized identities can no longer modify or administer repositories outside their scope.

Default Value:

By default, IAM principals do not have access to Amazon ECR repositories unless permissions are explicitly granted through IAM and, where used, ECR repository policies. For EKS image pulls, the node role or Fargate pod execution role must be granted the required ECR authentication and pull permissions before images can be retrieved from ECR.

References:

1. https://docs.aws.amazon.com/AmazonECR/latest/userguide/repository-policies.html
2. https://docs.aws.amazon.com/AmazonECR/latest/userguide/ECR_on_EKS.html
3. https://docs.aws.amazon.com/AmazonECR/latest/userguide/security_iam_id-based-policy-examples.html
4. https://docs.aws.amazon.com/aws-managed-policy/latest/reference/AmazonEC2ContainerRegistryPullOnly.html

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	5.4 Restrict Administrator Privileges to Dedicated Administrator Accounts Restrict administrator privileges to dedicated administrator accounts on enterprise assets. Conduct general computing activities, such as internet browsing, email, and productivity suite use, from the user's primary, non-privileged account.	●	●	●
v7	4.3 Ensure the Use of Dedicated Administrative Accounts Ensure that all users with administrative account access use a dedicated or secondary account for elevated activities. This account should only be used for administrative activities and not internet browsing, email, or similar activities.	●	●	●

5.1.3 Restrict ECR access for EKS runtime identities to pull-only permissions (Manual)

Profile Applicability:

- Level 1

Description:

Restricting ECR access for EKS runtime identities to pull-only permissions helps ensure that the identities used to run workloads can retrieve required container images without also receiving broader registry visibility or administrative access. In EKS, this applies to worker node IAM roles and, for Fargate workloads, the pod execution role that pulls images from private ECR repositories.

Rationale:

The EKS runtime image-pull path requires only a small subset of ECR permissions, so granting broader registry visibility is unnecessary for node roles and Fargate pod execution roles. For node-based workloads, the required permissions are `ecr:GetAuthorizationToken`, `ecr:BatchCheckLayerAvailability`, `ecr:BatchGetImage`, and `ecr:GetDownloadUrlForLayer`. For Fargate, the pod execution role is used to pull images from private ECR repositories. A pull-only policy is therefore a closer least-privilege match for runtime identities because it supports registry authentication and layer retrieval without also granting repository inspection and inventory actions.

A read-only ECR policy is broader than the runtime pull path because it also allows actions such as `ecr:GetRepositoryPolicy`, `ecr:DescribeRepositories`, `ecr:ListImages`, `ecr:DescribeImages`, `ecr:GetLifecyclePolicy`, `ecr:ListTagsForResource`, and `ecr:DescribeImageScanFindings`. Those actions expose repository metadata, image inventory, lifecycle configuration, and scan-result visibility that are typically useful for administrators, auditors, or tooling, but are not required just to start workloads from private ECR images. Restricting runtime identities to pull-only permissions therefore reduces unnecessary registry exposure while still preserving the permissions required for image retrieval.

Impact:

This control reduces unnecessary exposure of ECR metadata and administrative surfaces to runtime identities, but the assigned role must still retain the permissions required to authenticate and pull images. If those pull permissions are removed or scoped incorrectly, nodes or Fargate-hosted pods will not be able to retrieve private ECR images, which can prevent workloads from starting successfully.

Audit:

Review the IAM role permissions and any ECR repository policies that apply to the runtime identities used by EKS to pull images, and verify that those identities are limited to pull-only access. Focus on the worker node IAM role for node-based workloads and the pod execution role for Fargate workloads. The expected pull path requires only `ecr:GetAuthorizationToken`, `ecr:BatchCheckLayerAvailability`, `ecr:BatchGetImage`, and `ecr:GetDownloadUrlForLayer`, while the broader `AmazonEC2ContainerRegistryReadOnly` policy also includes repository and image discovery actions that are not required just to pull images. Repository policies can also affect access, but `ecr:GetAuthorizationToken` must still come from IAM.

- Identify the runtime role in scope. Review the IAM role used by EKS worker nodes, or the Fargate pod execution role if the workload runs on Fargate. That is the identity that should be restricted to pull-only access.
- Review the managed policies attached to that role. Flag broad policies such as `AmazonEC2ContainerRegistryReadOnly` or `AmazonEC2ContainerRegistryFullAccess` when the role only needs to pull images. `AmazonEC2ContainerRegistryPullOnly` is the tighter managed-policy fit for runtime image pulls.

```
aws iam list-attached-role-policies \
  --role-name $ROLE_NAME
```

- Review inline policies on the same role if managed policies are not the whole story. Confirm they do not add broader ECR actions such as repository listing, repository-policy reads, scan-finding reads, push actions, delete actions, or wildcard `ecr:*` access. Least-privilege ECR policies should be tailored to the actual use case.
- Check for repository-level exceptions. If a repository policy is used, verify that it does not grant unnecessary principals or broader access than intended. A missing repository policy is not automatically a finding, because ECR access can be granted through IAM alone.

```
aws ecr get-repository-policy \
  --repository-name $REPO_NAME \
  --region $REGION_CODE
```

- Confirm the effective permission set is pull-only. The runtime role should be limited to registry authentication and image retrieval, not broader read, push, or administrative access. If broader permissions are present, mark the role for remediation.

Remediation:

Enforce pull-only ECR access for EKS runtime identities by limiting the worker node IAM role, and where applicable the Fargate pod execution role, to only the permissions required to authenticate to ECR and retrieve container image layers. The runtime pull path only needs `ecr:GetAuthorizationToken`, `ecr:BatchCheckLayerAvailability`, `ecr:BatchGetImage`, and `ecr:GetDownloadUrlForLayer`. Broader policies such as `AmazonEC2ContainerRegistryReadOnly` or `AmazonEC2ContainerRegistryFullAccess` expose additional repository visibility or administration capabilities that are not required just to start workloads.

- Identify the runtime roles in scope. Review the node IAM role for node-based workloads and the pod execution role for Fargate workloads, because those are the identities used to pull private ECR images.
- Remove broader ECR managed policies from runtime roles. Detach `AmazonEC2ContainerRegistryReadOnly`, `AmazonEC2ContainerRegistryFullAccess`, or similar broad policies from roles that only need image-pull access.

```
aws iam detach-role-policy \
  --role-name $ROLE_NAME \
  --policy-arn arn:aws:iam::aws:policy/AmazonEC2ContainerRegistryReadOnly
```

- Attach the pull-only managed policy to the runtime role. `AmazonEC2ContainerRegistryPullOnly` is the tighter managed-policy fit for runtime image pulls.

```
aws iam attach-role-policy \
  --role-name $ROLE_NAME \
  --policy-arn arn:aws:iam::aws:policy/AmazonEC2ContainerRegistryPullOnly
```

- Remove or replace inline policies that add extra ECR permissions. Keep runtime roles limited to the pull path and remove repository listing, repository-policy reads, scan-finding reads, push actions, delete actions, and wildcard `ecr:*` access unless there is a documented operational need.
- Check repository policies for unnecessary grants. Repository policies should not reintroduce broader access for runtime principals, and `ecr:GetAuthorizationToken` must still be granted through IAM even when a repository policy is used.
- Retest image pulls after the change. Confirm that nodes and Fargate workloads can still pull required private ECR images and that the runtime role no longer has broader repository visibility or administrative access than needed.

Default Value:

By default, EKS runtime identities do not automatically receive pull-only ECR access unless the node role or Fargate pod execution role is created with the required ECR pull permissions. When clusters and worker node groups are created with eksctl or the Getting Started CloudFormation templates, those image-pull permissions are applied to the worker node IAM role by default.

References:

1. https://docs.aws.amazon.com/AmazonECR/latest/userguide/ECR_on_EKS.html
2. https://docs.aws.amazon.com/aws-managed-policy/latest/reference/AmazonEC2ContainerRegistryPullOnly.html
3. https://docs.aws.amazon.com/eks/latest/userguide/create-node-role.html
4. https://docs.aws.amazon.com/AmazonECR/latest/userguide/repository-policies.html

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	5.4 Restrict Administrator Privileges to Dedicated Administrator Accounts Restrict administrator privileges to dedicated administrator accounts on enterprise assets. Conduct general computing activities, such as internet browsing, email, and productivity suite use, from the user's primary, non-privileged account.	●	●	●
v7	4.3 Ensure the Use of Dedicated Administrative Accounts Ensure that all users with administrative account access use a dedicated or secondary account for elevated activities. This account should only be used for administrative activities and not internet browsing, email, or similar activities.	●	●	●

5.1.4 Allow only approved container registries for images used by EKS workloads (Manual)

Profile Applicability:

- Level 1

Description:

Allowing only approved container registries for images used by EKS workloads helps reduce supply chain risk by limiting where workloads can source their images. This creates a controlled image origin model in which clusters admit workloads only from trusted registries and can pair that control with image-signature verification before containers are allowed to run.

Rationale:

A practical way to enforce this control in EKS is through policy-as-code admission controls such as Kyverno or Gatekeeper, which evaluate Kubernetes API requests before the requested change is persisted. Those policies can validate image-source rules, verify signatures and attested artifacts, and reject deployments that do not meet the required trust conditions. Signature verification workflows for EKS also require a Notation trust store and trust policy for the selected admission controller, which makes the registry restriction more than a naming convention and turns it into an admission-time enforcement point tied to trusted publishers and approved image sources.

Impact:

This control reduces the chance that workloads pull images from unapproved or untrusted registries, but it adds administrative overhead because the approved-registry policy must be maintained for platform add-ons, vendor components, and operational exceptions. If the allowlist is too narrow, admission can deny valid deployments until the required registry is explicitly added, as shown in GuardDuty troubleshooting for clusters using an `allowed-container-registries` admission policy.

Audit:

Review the image sources currently used by the cluster and the admission controls that govern new deployments, then verify that workloads are limited to approved registries only. The strongest implementation for this control is an admission-time policy, because policy-as-code solutions such as Kyverno and OPA Gatekeeper can intercept Kubernetes API requests before the change is persisted, and image-security guidance for EKS describes using admission controllers to verify image trust conditions before deployment. Include required AWS-managed image sources for platform components and add-ons in the approved list, because EKS add-ons and core components such as `kube-proxy`, `coredns`, and `aws-node` are pulled from region-specific ECR registries.

- Compare every running workload image registry hostname to the approved registry allowlist. Any image sourced from an unapproved registry is a finding.

```
kubectl get pods -A -o jsonpath='{range
.items[*]}{.metadata.namespace}{"\t"}{.metadata.name}{"\t"}{range
.spec.containers[*]}{.image}{" "}{end}{"\n"}{end}'
```

- Confirm the cluster uses an admission control path that evaluates image source before deployment is accepted, rather than relying only on CI/CD checks or documented standards.
- Check the approved-registry allowlist. Verify it includes internal ECR repositories, approved third-party registries if any, and the region-specific ECR registries required for EKS add-ons and platform components.
- Confirm an approved image is accepted and a test image from an unapproved registry is rejected at admission time.
- Verify any additional allowed registries are documented and intentionally approved, and where image signing or attestation is used, confirm the admission controller also validates those trust conditions for approved image sources.

Remediation:

Restrict EKS workloads to approved container registries by enforcing image-source controls at admission time, rather than relying only on CI/CD checks or written standards. Enforcement is typically implemented with a policy-as-code admission controller such as Kyverno or Gatekeeper so image references are checked before the deployment is persisted. The allowlist must also include the Region-specific ECR registries required for EKS platform components and add-ons, because EKS add-on images are pulled from Amazon ECR private repositories replicated per Region.

- Include internal ECR repositories, explicitly approved third-party registries, and the Region-specific ECR registries required for EKS add-ons and platform components in the approved registry allowlist.
- Configure Kyverno, Gatekeeper, or another validating admission path so image registry source is checked before the workload is admitted.
- Apply the policy to workload namespaces while accounting for system namespaces and approved operational exceptions so required platform components are not blocked.
- Configure the admission controller to verify signed images and attested artifacts when stronger supply chain controls are required, using a Notation trust store and trust policy.
- Confirm approved registries are allowed, unapproved registries are denied, and update the allowlist when new add-ons, vendor components, or trusted registries are introduced.

Default Value:

Container registries are not restricted by default and Kubernetes assumes your default CR is Docker Hub.

References:

1. https://docs.aws.amazon.com/eks/latest/best-practices/image-security.html
2. https://docs.aws.amazon.com/eks/latest/best-practices/pod-security.html
3. https://docs.aws.amazon.com/eks/latest/userguide/add-ons-images.html
4. https://docs.aws.amazon.com/eks/latest/userguide/image-verification.html

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	2.5 Allowlist Authorized Software Use technical controls, such as application allowlisting, to ensure that only authorized software can execute or be accessed. Reassess bi-annually, or more frequently.		●	●
v7	5.3 Securely Store Master Images Store the master images and templates on securely configured servers, validated with integrity monitoring tools, to ensure that only authorized changes to the images are possible.		●	●
v7	13.4 Only Allow Access to Authorized Cloud Storage or Email Providers Only allow access to authorized cloud storage or email providers.		●	●

5.2 Identity and Access Management (IAM)

This section contains recommendations relating to using AWS IAM with Amazon EKS.

5.2.1 Prefer using dedicated EKS Service Accounts (Automated)

Profile Applicability:

- Level 1

Description:

Using dedicated Kubernetes service accounts for EKS workloads helps isolate AWS permissions by application, job, or trust boundary instead of sharing a single identity across multiple workloads. This supports more controlled access to AWS services and reduces the blast radius if one workload is compromised or misconfigured.

Rationale:

A workload that needs AWS access should run with a Kubernetes service account that is associated to an IAM role through EKS Pod Identity or IRSA, and that association should be created for each unique permission set an application needs. With EKS Pod Identity, the IAM role trust policy is limited to the `pods.eks.amazonaws.com` service principal and can be further restricted with conditions tied to cluster, namespace, and service account context. With IRSA, the service account is associated to an IAM role through the cluster OIDC provider, and pods use the default AWS credential chain to obtain temporary credentials for that role. Using a dedicated service account per workload or per permission boundary prevents unrelated pods in the same namespace from inheriting the same AWS access and aligns permissions more closely to the actual application function.

Impact:

This control improves permission isolation and usually makes auditing easier, but it increases the number of service accounts, IAM roles, and role associations that must be created and maintained. If a pod is left on the namespace default service account, or if the wrong service account is referenced in the workload spec, the pod may fail to access required AWS resources or may run with a broader identity than intended.

Audit:

Review all workloads in the cluster and verify that applications use dedicated Kubernetes service accounts rather than relying on the namespace default service account. Confirm that the namespace default service account has only the safe default discovery access unless additional rights are explicitly required, and verify that pods which do not need Kubernetes API access are not automatically mounting service account tokens. For workloads that need AWS access, verify that the referenced service account is associated with the correct IAM role through EKS Pod Identity or IRSA, and that the permission scope matches the application's actual AWS access requirements. Also confirm that pods cannot bypass the dedicated service account model by inheriting broader credentials from the worker node instance profile.

- Review each namespace default service account and confirm it has no extra `RoleBindings` or `ClusterRoleBindings` beyond the safe default discovery access unless explicitly required.
- Review workload manifests and confirm pods explicitly set `serviceAccountName` when they need Kubernetes API or AWS access, rather than falling back to the namespace default service account.
- Verify `automountServiceAccountToken: false` is set for pods or default service accounts where the application does not need Kubernetes API access. Do not treat this as a blanket requirement for every workload.

Check pods for an explicit setting:

```
kubectl get pods -A \
  -o custom-
columns='NAMESPACE:.metadata.namespace,POD:.metadata.name,SERVICEACCOUNT:.spe
c.serviceAccountName,AUTOMOUNT:.spec.automountServiceAccountToken'
```

Check the default service account in each namespace:

```
kubectl get sa -A \
  -o custom-
columns='NAMESPACE:.metadata.namespace,NAME:.metadata.name,AUTOMOUNT:.automou
ntServiceAccountToken' | grep ' default '
```

- For workloads that access AWS services, verify the dedicated service account is associated to the intended IAM role through EKS Pod Identity or IRSA, and that each unique permission set uses its own service account boundary.
- Verify node instance profile access is restricted so pods cannot still obtain broader worker-node credentials after a service-account-based identity model is configured.

Remediation:

Use dedicated Kubernetes service accounts for EKS workloads and associate AWS permissions to those service accounts instead of relying on the namespace default service account or broad worker node IAM permissions. Prefer EKS Pod Identity where supported, and use IRSA where Pod Identity is not supported or where IRSA is specifically required. Create one dedicated service account and one least-privilege IAM role for each application or distinct permission boundary, and update workload manifests to reference the intended service account explicitly. Disable automatic service account token mounting for workloads that do not need Kubernetes API access. After migrating workloads to dedicated service accounts, restrict access to the worker node instance profile so pods cannot inherit broader node credentials.

- Create a dedicated Kubernetes service account for each application or distinct AWS permission boundary instead of sharing the namespace default service account.

- Associate that service account with a least-privilege IAM role by using EKS Pod Identity where supported, or IRSA where Pod Identity is not supported or not selected.
- Update workload manifests to set `spec.serviceAccountName` explicitly so the intended service account is always used.
- Set `automountServiceAccountToken: false` for pods or service accounts that do not need Kubernetes API access.
- Restrict worker node IMDS access so workloads use the dedicated service-account identity path and cannot also inherit the node instance profile.

Default Value:

By default, Pods use the namespace default service account, and token mounting is automatic unless explicitly disabled. A dedicated service account with AWS permissions exists only if you configure Pod Identity or IRSA for that workload.

References:

1. https://docs.aws.amazon.com/eks/latest/best-practices/identity-and-access-management.html
2. https://docs.aws.amazon.com/eks/latest/userguide/pod-identities.html
3. https://docs.aws.amazon.com/eks/latest/userguide/pod-id-association.html
4. https://docs.aws.amazon.com/eks/latest/userguide/iam-roles-for-service-accounts.html

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	12.8 Establish and Maintain Dedicated Computing Resources for All Administrative Work Establish and maintain dedicated computing resources, either physically or logically separated, for all administrative tasks or tasks requiring administrative access. The computing resources should be segmented from the enterprise's primary network and not be allowed internet access.			●
v7	4.3 Ensure the Use of Dedicated Administrative Accounts Ensure that all users with administrative account access use a dedicated or secondary account for elevated activities. This account should only be used for administrative activities and not internet browsing, email, or similar activities.	●	●	●

5.3 AWS EKS Key Management Service

This section contains recommendations relating to using AWS EKS Key Management.

5.3.1 Use customer managed KMS keys for EKS envelope encryption of Kubernetes secrets (Manual)

Profile Applicability:

- Level 1

Description:

Using customer managed KMS keys for EKS envelope encryption adds a customer-controlled protection layer for Kubernetes secrets stored through the Kubernetes API. This strengthens data-at-rest protection while allowing organizations to manage key ownership, rotation, policy, and audit visibility for the key used to protect secret data.

Rationale:

Envelope encryption adds a second encryption layer to Kubernetes secrets stored through the EKS control plane. Secret data is encrypted with a unique data encryption key, and that data encryption key is then encrypted with a key encryption key in AWS KMS before the object is persisted in `etcd`. This means the secret is stored in ciphertext in the backing datastore and is decrypted only through the Kubernetes API server path, which strengthens protection beyond the underlying storage encryption used for `etcd`.

In the current EKS model, envelope encryption protects Kubernetes API data by using KMS-backed key encryption and derived data encryption keys within the control plane encryption flow. A customer managed KMS key gives the customer direct control over key policy, rotation, audit visibility, and key lifecycle for the key that protects those data encryption keys. Where a customer managed key is associated for Kubernetes secrets, that same key can also serve as the key encryption key for Kubernetes API data protected by envelope encryption.

Impact:

Using a customer managed KMS key increases customer control, but it also adds KMS cost, key-policy administration, and lifecycle responsibility. If you use your own key, standard KMS charges apply, and deleting the KMS key used for EKS encryption can leave the cluster in a permanently degraded and unrecoverable state.

Audit:

Review the EKS cluster configuration and verify that Kubernetes secrets are protected with a customer managed KMS key, not only the default AWS-owned envelope-encryption key. For this control, the important check is whether the cluster is associated with a customer managed key for EKS envelope encryption. Also verify that the key is usable by the cluster, is in the same Region, and has not been disabled, deleted, or stripped of required grants or permissions. A customer managed key association should appear on the cluster, while current EKS clusters that rely only on the default AWS-owned key do not expose a customer key ARN for this purpose.

- Review the cluster encryption configuration and confirm a customer managed KMS key ARN is associated to the cluster for Kubernetes secrets protection. If no customer key ARN is associated, the cluster does not meet this control.

```
aws eks describe-cluster \
  --name $CLUSTER_NAME \
  --region $REGION_CODE \
  --query 'cluster.encryptionConfig'
```

- In the command output, verify that the encryption configuration includes `resources` with `secrets` and a `provider.keyArn` value for the customer managed KMS key. If the response is empty or no customer key ARN is present, this control is not satisfied.
- Review the selected KMS key and confirm it is symmetric, supports encrypt and decrypt operations, and exists in the same Region as the cluster. If the key is in another account, confirm the required principal still has access to use it.
- Verify the key is still enabled and that grants or key-policy permissions required by EKS have not been revoked. A disabled key, revoked grant, or deleted key can place the cluster into a degraded or unrecoverable state.
- Where continuous compliance monitoring is used, confirm the cluster would pass the `EKS_SECRETS_ENCRYPTED` AWS Config rule with the expected customer managed key ARN configured as a parameter.

Remediation:

Configure the EKS cluster to use a customer managed KMS key for envelope encryption of Kubernetes secrets so secret data stored through the Kubernetes API is protected with a customer-controlled key encryption key in AWS KMS. This requires an existing symmetric KMS key in the same Region as the cluster, with key policy and grant permissions that allow EKS to use the key.

The `associate-encryption-config` operation does not create a KMS key and does not encrypt arbitrary data directly. It associates an existing customer managed KMS key to the cluster so EKS can use it for envelope encryption of Kubernetes secrets.

After the association is made, the cluster update must complete successfully and the key must remain enabled and accessible. Disabling, deleting, or breaking access to the associated key can place the cluster in a degraded or unrecoverable state.

- Create or identify a customer managed symmetric KMS key that already exists in the same Region as the EKS cluster and is allowed for EKS use.
- Associate that existing KMS key to the cluster for `secrets` encryption. Replace the variables with real values before running the command. In this command, `CLUSTER_NAME` is the EKS cluster name, `REGION_CODE` is the cluster Region, and `KEY_ARN` is the full ARN of the existing KMS key.

```
aws eks associate-encryption-config \
  --cluster-name "$CLUSTER_NAME" \
  --region "$REGION_CODE" \
  --encryption-config
"[{\"resources\":[\"secrets\"],\"provider\":{\"keyArn\":\"${KEY_ARN}\"}}]"
```

- Wait for the cluster update to complete, because this operation runs as an asynchronous EKS update.
- Recheck the cluster configuration and confirm that `cluster.encryptionConfig` contains `resources` set to `secrets` and the expected `provider.keyArn`.
- Keep the associated KMS key enabled and do not delete it while the cluster depends on it.

Default Value:

By default, EKS applies envelope encryption to Kubernetes API data with an AWS owned KMS key, not a customer managed KMS key. A customer managed KMS key is used only if you explicitly associate one to the cluster for envelope encryption.

References:

1. https://docs.aws.amazon.com/eks/latest/userguide/envelope-encryption.html
2. https://docs.aws.amazon.com/eks/latest/userguide/enable-kms.html
3. https://docs.aws.amazon.com/eks/latest/best-practices/data-encryption-and-secrets-management.html
4. https://docs.aws.amazon.com/config/latest/developerguide/eks-secrets-encrypted.html

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	3.11 Encrypt Sensitive Data at Rest Encrypt sensitive data at rest on servers, applications, and databases containing sensitive data. Storage-layer encryption, also known as server-side encryption, meets the minimum requirement of this Safeguard. Additional encryption methods may include application-layer encryption, also known as client-side encryption, where access to the data storage device(s) does not permit access to the plain-text data.		●	●
v7	14.8 Encrypt Sensitive Information at Rest Encrypt all sensitive information at rest using a tool that requires a secondary authentication mechanism not integrated into the operating system, in order to access the information.			●

5.4 Cluster Networking

This section contains recommendations relating to network security configurations in Amazon EKS.

5.4.1 Restrict EKS control plane endpoint access with private access and public CIDR allowlists (Automated)

Profile Applicability:

- Level 1

Description:

This recommendation is about reducing exposure of the Kubernetes API server without necessarily making it private-only. The intended end state is that private endpoint access is enabled, and if the public endpoint remains enabled, it is narrowed to a small set of trusted source CIDR ranges instead of remaining open to all internet addresses. This keeps the control plane reachable for approved administration paths while lowering unnecessary internet exposure.

Rationale:

EKS exposes separate settings for the API server through `endpointPrivateAccess`, `endpointPublicAccess`, and `publicAccessCidrs`. When private access is enabled, API requests that originate inside the cluster VPC use the private VPC endpoint, and access to that private endpoint is governed by the cluster security group. When public access is enabled, the public endpoint remains internet reachable, but `publicAccessCidrs` can restrict it to explicit source ranges. If private access is not enabled, the CIDR allowlist must also account for the outbound source addresses used by nodes or Fargate pods, such as NAT gateway egress addresses, or those components can lose access to the API server. The CIDR allowlist affects the public endpoint only and does not control the private endpoint.

Restricting access to an authorized network can provide additional security benefits for your container cluster, including:

- Better protection from outsider attacks: Authorized networks provide an additional layer of security by limiting external access to a specific set of addresses you designate, such as those that originate from your premises. This helps protect access to your cluster in the case of a vulnerability in the cluster's authentication or authorization mechanism.
- Better protection from insider attacks: Authorized networks help protect your cluster from accidental leaks of master certificates from your company's premises. Leaked certificates used from outside Cloud Services and outside the authorized IP ranges (for example, from addresses outside your company) are still denied access.

Impact:

This recommendation gives you a balanced hardening option because it reduces public exposure while preserving approved remote administration paths. The tradeoff is operational maintenance of the CIDR allowlist and the risk of accidental lockout if trusted administrator IPs, NAT egress addresses, or other required source ranges are omitted. It is stricter than the default public endpoint model, but less restrictive than a private-only API design.

Audit:

Check for the following `endpointPrivateAccess` and `endpointPublicAccess` to be 'true'

```
aws eks describe-cluster \
  --name $CLUSTER_NAME \
  --region $REGION_CODE \
  --query
"cluster.resourcesVpcConfig.{endpointPrivateAccess:endpointPrivateAccess,endp
ointPublicAccess:endpointPublicAccess,publicAccessCidrs:publicAccessCidrs}" \
  --output json
```

Expected Output:

```
{
    "endpointPrivateAccess": true,
    "endpointPublicAccess": true,
    "publicAccessCidrs": [
        "203.0.113.5/32"
    ]
}
```

Note: Check for publicAccessCidrs is set to a valid IP address not set to 0.0.0.0/0

Remediation:

By enabling private endpoint access to the Kubernetes API server, all communication between your nodes and the API server stays within your VPC. You can also limit the IP addresses that can access your API server from the internet, or completely disable internet access to the API server.

With this in mind, you can update your cluster accordingly using the AWS CLI to ensure that Private Endpoint Access is enabled.

If you choose to also enable Public Endpoint Access then you should also configure a list of allowable CIDR blocks, resulting in restricted access from the internet. If you specify no CIDR blocks, then the public API server endpoint is able to receive and process requests from all IP addresses by defaulting to ['0.0.0.0/0'].

For example, the following command would enable private access to the Kubernetes API as well as limited public access over the internet from a single IP address (noting the /32 CIDR suffix):

```
aws eks update-cluster-config \
  --name $CLUSTER_NAME \
  --region $REGION_CODE \
  --resources-vpc-config endpointPrivateAccess=true,
endpointPublicAccess=true, publicAccessCidrs="203.0.113.5/32"
```

Note:

The CIDR blocks specified cannot include reserved addresses. There is a maximum number of CIDR blocks that you can specify. For more information, see the EKS Service Quotas link in the references section. For more detailed information, see the EKS Cluster Endpoint documentation link in the references section.

Default Value:

By default, Endpoint Public Access is disabled.

References:

1. https://docs.aws.amazon.com/eks/latest/userguide/cluster-endpoint.html
2. https://docs.aws.amazon.com/eks/latest/userguide/config-cluster-endpoint.html
3. https://docs.aws.amazon.com/AWSCloudFormation/latest/TemplateReference/aws-properties-eks-cluster-resourcesvpcconfig.html
4. https://docs.aws.amazon.com/cli/latest/reference/eks/update-cluster-config.html

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	4.4 Implement and Manage a Firewall on Servers Implement and manage a firewall on servers, where supported. Example implementations include a virtual firewall, operating system firewall, or a third-party firewall agent.	●	●	●
v8	9.3 Maintain and Enforce Network-Based URL Filters Enforce and update network-based URL filters to limit an enterprise asset from connecting to potentially malicious or unapproved websites. Example implementations include category-based filtering, reputation-based filtering, or through the use of block lists. Enforce filters for all enterprise assets.		●	●
v7	7.4 Maintain and Enforce Network-Based URL Filters Enforce network-based URL filters that limit a system's ability to connect to websites not approved by the organization. This filtering shall be enforced for each of the organization's systems, whether they are physically at an organization's facilities or not.		●	●

5.4.2 Enable private-only EKS control plane access and disable the public endpoint (Automated)

Profile Applicability:

- Level 1

Description:

This recommendation is the strict private-only version of API endpoint hardening. The intended end state is that the Kubernetes API server is reachable only from inside the cluster VPC or from a connected private network, and there is no internet-reachable public API endpoint at all. This is a pure control-plane exposure decision.

Rationale:

In EKS, disabling public access requires private access to be enabled. In that configuration, all traffic to the Kubernetes API server must come from within the cluster VPC or from a connected network path, and `kubectl` access from the internet is not available. The private endpoint is controlled by the cluster security group, while `publicAccessCidrs` becomes irrelevant because there is no public endpoint to filter. The cluster endpoint is resolved through DNS to a private IP address for private-only access, and administration from outside the VPC must come through connectivity such as VPN, Transit Gateway, or Direct Connect rather than through an internet path.

In a private cluster, the master node has two endpoints, a private and public endpoint. The private endpoint is the internal IP address of the master, behind an internal load balancer in the master's VPC network. Nodes communicate with the master using the private endpoint. The public endpoint enables the Kubernetes API to be accessed from outside the master's VPC network.

Although Kubernetes API requires an authorized token to perform sensitive actions, a vulnerability could potentially expose the Kubernetes publicly with unrestricted access. Additionally, an attacker may be able to identify the current cluster and Kubernetes API version and determine whether it is vulnerable to an attack. Unless required, disabling public endpoint will help prevent such threats, and require the attacker to be on the master's VPC network to perform any attack on the Kubernetes API.

Impact:

This recommendation controls for reducing direct exposure of the EKS control plane, but it requires deliberate private connectivity for administrators, automation, and any external systems that need Kubernetes API access. If that private access path is missing or incomplete, cluster operations, troubleshooting, and deployment tooling can be blocked even though the cluster itself is healthy.

Audit:

Check for private endpoint access to the Kubernetes API server

```
aws eks describe-cluster \
  --name $CLUSTER_NAME \
  --region $REGION_CODE \
  --query
"cluster.resourcesVpcConfig.{endpointPublicAccess:endpointPublicAccess,
endpointPrivateAccess:endpointPrivateAccess}" \
  --output json
```

Expected Output:

```
{
    "endpointPublicAccess": false,
    "endpointPrivateAccess": true
}
```

Remediation:

By enabling private endpoint access to the Kubernetes API server, all communication between your nodes and the API server stays within your VPC.

With this in mind, you can update your cluster accordingly using the AWS CLI to ensure that Private Endpoint Access is enabled.

For example, the following command would enable private access to the Kubernetes API and ensure that no public access is permitted:

```
aws eks update-cluster-config \
  --name $CLUSTER_NAME \
  --region $REGION_CODE \
  --resources-vpc-config
endpointPrivateAccess=true,endpointPublicAccess=false
```

Note: For more detailed information, see the EKS Cluster Endpoint documentation link in the references section.

Default Value:

By default, the Public Endpoint is disabled.

References:

1. https://docs.aws.amazon.com/eks/latest/userguide/cluster-endpoint.html
2. https://docs.aws.amazon.com/eks/latest/userguide/config-cluster-endpoint.html
3. https://docs.aws.amazon.com/AWSCloudFormation/latest/TemplateReference/aws-properties-eks-cluster-resourcesvpcconfig.html
4. https://docs.aws.amazon.com/cli/latest/reference/eks/update-cluster-config.html

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	4.4 Implement and Manage a Firewall on Servers Implement and manage a firewall on servers, where supported. Example implementations include a virtual firewall, operating system firewall, or a third-party firewall agent.	●	●	●
v7	12 Boundary Defense Boundary Defense			

5.4.3 Ensure clusters are created with Private Nodes (Manual)

Profile Applicability:

- Level 1

Description:

This recommendation is about the data plane, not the control plane. It means worker nodes are placed in private subnets and are not intended to have direct public internet exposure. In the common pattern, nodes run in private subnets while internet-facing ingress resources are placed in public subnets. This changes where compute runs and how node traffic exits the VPC, but it does not by itself determine whether the Kubernetes API endpoint is public, private, or both.

Rationale:

EKS managed node groups can be launched in either public or private subnets, so private nodes are a deliberate architecture choice rather than a built-in default. Placing nodes in private subnets reduces direct reachability of the worker instances and gives tighter control over egress paths. At the same time, those nodes still need access to required services such as ECR for image pulls, and that access must be provided through a NAT gateway or the necessary VPC endpoints. In a fully private cluster design with no outbound internet access, endpoint private access is required so nodes can register with the cluster, and additional VPC endpoints are needed for supporting services such as ECR, S3, STS, EKS Auth, and others depending on workload needs.

Impact:

This reduces direct internet exposure of the worker nodes and usually improves traffic control around the data plane, but it increases dependency on NAT design, VPC endpoints, subnet routing, and private management methods such as SSM or internal jump-host patterns. It also does not automatically make the control plane private. A cluster can have private nodes and still expose a public API endpoint unless the endpoint settings are hardened separately.

Audit:

Review the subnets used by each EKS node group and verify that worker nodes are deployed in private subnets, not public subnets. For this control, the deciding evidence is node subnet placement, public IP behavior, and subnet routing, not the control plane endpoint settings. A compliant private-node design places nodes in subnets that do not auto-assign public IP addresses and do not have a default route to an Internet Gateway. If outbound access is required, the design should use a NAT gateway or the necessary VPC endpoints instead.

Review each node group and identify the subnet IDs used by the worker nodes. If the node group is placed in public subnets, this control is not satisfied.

```
aws ec2 describe-subnets \
  --subnet-ids $(aws eks describe-cluster \
    --name $CLUSTER_NAME \
    --region $REGION_CODE \
    --query 'cluster.resourcesVpcConfig.subnetIds' \
    --output text) \
  --region $REGION_CODE \
  --query
'Subnets[].{SubnetId:SubnetId,MapPublicIpOnLaunch:MapPublicIpOnLaunch}' \
  --output table
```

Expected output: a table containing `MapPublicIpOnLaunch` values of `True` or `False` and the corresponding `SubnetId`. These are the subnets you must inspect next. Private-node compliance is not determined by the node group alone, but by whether these subnets are actually private.

- Verify the node subnets do not auto-assign public IPv4 addresses. Managed node groups in public subnets require automatic public IP assignment, so a `true` value here indicates a public-subnet design rather than private nodes.
- Verify the route table associated with each node subnet does not contain a default route such as `0.0.0.0/0` to an Internet Gateway. Private subnets can still have outbound access through a NAT gateway, but they should not route directly to an Internet Gateway.
- Verify the subnets identified for the node group are intended private subnets in the cluster VPC design and that nodes do not require direct public IP reachability for administration or workload traffic.
- Verify outbound dependencies for private nodes are satisfied through NAT or required VPC endpoints so nodes can still pull images and reach supporting services without direct public exposure.

Remediation:

For existing worker nodes that run in public subnets, remediate the nodes by replacing them with a new node group that uses only private subnets, then migrating workloads and retiring the original node group. Managed node group subnet placement is defined when the node group is created, so moving existing nodes to private subnets is handled as a replacement and migration activity rather than an in-place subnet change. The target private subnets should not auto-assign public IPv4 addresses and should not have a default route to an Internet Gateway. If outbound access is required for image pulls or AWS API calls, provide it through a NAT gateway or the required VPC endpoints instead.

- Create a new node group that uses only private subnet IDs. Do not reuse public subnets for the replacement worker nodes.
- Ensure the replacement subnets do not auto-assign public IPv4 addresses, so new worker instances are launched without public IPs.
- Ensure the route tables for those subnets do not send `0.0.0.0/0` directly to an Internet Gateway. If internet egress is needed, route it through a NAT gateway or

use the necessary VPC endpoints. ECR image pulls for private-subnet nodes require NAT or the ECR API, ECR DKR, and S3 endpoints.

- If a launch template is used, confirm its network interface settings do not assign public IP addresses or otherwise override the private-subnet design.
- Cordon and drain workloads from the existing public-subnet nodes, move scheduling to the new private-subnet node group, then delete the old node group after workloads are stable. Managed node groups support adding new node groups to an existing cluster and handle node lifecycle through replacement rather than manual node re-registration.

References:

1. https://docs.aws.amazon.com/eks/latest/userguide/managed-node-groups.html
2. https://docs.aws.amazon.com/eks/latest/best-practices/subnets.html
3. https://docs.aws.amazon.com/eks/latest/userguide/private-clusters.html
4. https://docs.aws.amazon.com/eks/latest/userguide/launch-templates.html

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	4.4 Implement and Manage a Firewall on Servers Implement and manage a firewall on servers, where supported. Example implementations include a virtual firewall, operating system firewall, or a third-party firewall agent.	●	●	●
v7	12 Boundary Defense Boundary Defense			

5.4.4 Use TLS certificates on EKS load balancer listeners for encrypted client connections (Manual)

Profile Applicability:

- Level 1

Description:

Using TLS certificates on EKS load balancer listeners protects client-to-load-balancer traffic in transit by presenting a trusted server certificate and negotiating encrypted HTTPS or TLS connections at the listener. In EKS, this commonly means configuring TLS on an NLB listener for Service resources or specifying certificate ARNs for ALB-based ingress so externally exposed application entry points do not rely on unencrypted HTTP.

Rationale:

EKS supports TLS configuration on both major load balancer paths. For NLB, EKS Auto Mode supports listener-level TLS through service annotations such as `aws-load-balancer-ssl-cert`, which specifies the ACM certificate ARN, and `aws-load-balancer-ssl-ports`, which identifies the listener ports that should use TLS. For ALB, EKS Auto Mode uses `IngressClassParams`, where certificate ARNs are part of the load balancer configuration for HTTP and HTTPS traffic. This matters because the listener is the point where the client session is negotiated, the certificate is presented, and encrypted traffic is established before requests are forwarded to targets. The network security guidance for EKS also makes clear that TLS can terminate at the load balancer, at ingress, or continue end to end to the pod depending on the security model, so requiring certificates on load balancer listeners is the baseline control for securing the external client connection path.

Impact:

This control improves protection against eavesdropping and tampering on client-facing connections, but it introduces certificate lifecycle and listener configuration requirements. The certificate ARN must match the intended domain names, the correct listener ports must be configured for TLS, and expired or missing certificates can break client access even when the workload itself is healthy. It also does not by itself guarantee end-to-end encryption beyond the load balancer, because traffic from the load balancer to the backend may still be unencrypted unless a separate backend TLS design is implemented.

Audit:

Review all EKS Service resources of type LoadBalancer and all Ingress resources that expose client traffic, and verify that client-facing listeners use HTTPS or TLS with a valid certificate. For NLB-backed Service resources, listener encryption is configured with annotations such as `aws-load-balancer-ssl-cert` and `aws-load-balancer-ssl-ports`. For ALB-backed ingress, HTTPS listeners require certificate configuration on the ALB path. A TCP listener on port 443 is a different design because it passes encrypted traffic through instead of terminating TLS at the load balancer.

- Identify all internet-facing or client-facing LoadBalancer Services and Ingress resources.
- Verify NLB services use `aws-load-balancer-ssl-cert` and `aws-load-balancer-ssl-ports` for listener encryption.
- Verify ALB listeners are configured for HTTPS and have the required certificate ARN association.
- Verify the certificate is valid for the intended hostname and that the listener design matches the intended TLS termination point.

Remediation:

Configure every client-facing EKS load balancer listener to use HTTPS or TLS with a valid certificate so client-to-load-balancer traffic is encrypted in transit. For NLB-backed Service resources, add the certificate ARN and encrypted listener ports in the service annotations. For ALB-backed ingress, configure HTTPS listeners and associate the required certificate ARN. If the design requires encryption beyond the load balancer, implement backend TLS separately because listener TLS alone protects only the client-facing connection.

- Add a valid certificate to each client-facing NLB or ALB listener that should serve encrypted traffic.
- For NLB services, set `aws-load-balancer-ssl-cert` to the certificate ARN and `aws-load-balancer-ssl-ports` to the required listener ports.
- For ALB ingress, configure HTTPS listeners and associate the required certificate ARN with the listener configuration.
- Replace or renew invalid certificates and implement backend TLS separately where end-to-end encryption is required.

Default Value:

By default, EKS does not automatically attach TLS certificates to client-facing load balancer listeners. TLS is used only when you explicitly configure an HTTPS or TLS listener and associate a certificate, while NLB services and ALB ingress can otherwise be created with non-TLS listener behavior unless you set the required certificate configuration.

References:

1. https://docs.aws.amazon.com/eks/latest/userguide/auto-configure-nlb.html
2. https://docs.aws.amazon.com/eks/latest/userguide/auto-configure-alb.html
3. https://docs.aws.amazon.com/elasticloadbalancing/latest/network/tls-listener-certificates.html
4. https://docs.aws.amazon.com/eks/latest/best-practices/network-security.html

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	3.10 Encrypt Sensitive Data in Transit Encrypt sensitive data in transit. Example implementations can include: Transport Layer Security (TLS) and Open Secure Shell (OpenSSH).		●	●
v7	14.4 Encrypt All Sensitive Information in Transit Encrypt all sensitive information in transit.		●	●

5.5 Authentication and Authorization

This section contains recommendations relating to authentication and authorization in Amazon EKS.

5.5.1 Use EKS access entries to manage IAM principal access to Kubernetes (Automated)

Profile Applicability:

- Level 1

Description:

Using EKS access entries centralizes how IAM principals are granted access to the Kubernetes API and lets cluster access be managed through Amazon EKS instead of relying on manual `aws-auth` ConfigMap updates. This gives a cleaner access-management model for users and roles that need to work with Kubernetes resources in the cluster.

Rationale:

An access entry binds an IAM principal, such as an IAM role or IAM user, to Kubernetes authorization in the cluster. To use access entries, the cluster authentication mode must be set to API or `API_AND_CONFIG_MAP`. Permissions can then be granted in two main ways: by associating EKS access policies to the access entry, or by mapping the principal to Kubernetes groups and continuing authorization through Kubernetes RBAC objects such as Roles, ClusterRoles, RoleBindings, and ClusterRoleBindings. Access policies can be scoped cluster-wide or to specific namespaces, while the Kubernetes-groups approach is better when highly customized RBAC is required. This model also avoids continued dependence on the deprecated `aws-auth` ConfigMap path for routine IAM principal access management.

Impact:

This control improves consistency and makes access easier to review, grant, and revoke, but it usually requires an authentication-mode update and a planned migration of any existing `aws-auth` ConfigMap entries. It can also require reworking current RBAC design if existing permissions are broad, inconsistent, or split between legacy ConfigMap mappings and Kubernetes RBAC.

Audit:

Review the cluster access model and verify IAM principal access to Kubernetes is managed through EKS access entries rather than relying only on legacy `aws-auth` ConfigMap mappings. Access entries are available only when the cluster authentication mode is set to `API` or `API_AND_CONFIG_MAP`, and permissions can then be granted through EKS access policies, Kubernetes groups for RBAC, or both.

Verify the cluster authentication mode is `API` or `API_AND_CONFIG_MAP`.

```
aws eks describe-cluster \
  --name $CLUSTER_NAME \
  --region $REGION_CODE \
  --query 'cluster.accessConfig.authenticationMode' \
  --output text
```

Expected output: API or API_AND_CONFIG_MAP

If the output is `CONFIG_MAP`, the cluster does not satisfy this audit step

- Review the cluster's access entries and confirm each required IAM role or user has an access entry, with no duplicate or unnecessary principals.
- Review the permissions attached to each access entry and confirm they are granted through the intended EKS access policies, Kubernetes groups, or both, with scope limited to the required namespaces or cluster-wide access level.
- Review any remaining `aws-auth` ConfigMap usage and flag legacy mappings that should be migrated or removed if access entries are now the intended management path.

Remediation:

Configure the cluster to use EKS access entries for IAM principal access management, then create and scope access entries for each required IAM role or user. This remediation typically starts by enabling the access-entry API through the cluster authentication mode, then moving existing access management from legacy `aws-auth` ConfigMap mappings to access entries with EKS access policies, Kubernetes RBAC group mappings, or both. Access entries require the cluster authentication mode to include the EKS API, and moving from `CONFIG_MAP` to `API_AND_CONFIG_MAP` is the recommended transition path.

Update the cluster authentication mode to `API_AND_CONFIG_MAP`

```
aws eks update-cluster-config \
  --name $CLUSTER_NAME \
  --region $REGION_CODE \
  --access-config authenticationMode=API_AND_CONFIG_MAP
```

- Create an access entry for each IAM principal that needs Kubernetes access. Each access entry maps one existing IAM role or IAM user to cluster access.
- Associate the required EKS access policies to the access entry, or map the principal to Kubernetes groups and authorize it through RBAC objects such as Roles, ClusterRoles, RoleBindings, and ClusterRoleBindings. Scope access to specific namespaces where possible.
- After validating access through the new model, remove or reduce legacy `aws-auth` ConfigMap mappings that are no longer needed so cluster access is managed consistently through access entries.

Default Value:

By default, the cluster authentication mode depends on how the EKS cluster was created. Clusters created through the EKS API, AWS SDKs, or CloudFormation default to `CONFIG_MAP`, while clusters created through the AWS Management Console default to `API_AND_CONFIG_MAP`. Only `API` and `API_AND_CONFIG_MAP` support EKS access entries.

References:

1. https://docs.aws.amazon.com/eks/latest/userguide/access-entries.html
2. https://docs.aws.amazon.com/eks/latest/userguide/setting-up-access-entries.html
3. https://docs.aws.amazon.com/eks/latest/userguide/creating-access-entries.html
4. https://docs.aws.amazon.com/eks/latest/userguide/access-policies.html

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	6.8 Define and Maintain Role-Based Access Control Define and maintain role-based access control, through determining and documenting the access rights necessary for each role within the enterprise to successfully carry out its assigned duties. Perform access control reviews of enterprise assets to validate that all privileges are authorized, on a recurring schedule at a minimum annually, or more frequently.			●
v7	16.2 Configure Centralized Point of Authentication Configure access for all accounts through as few centralized points of authentication as possible, including network, security, and cloud systems.		●	●

Appendix: Summary Table

CIS Benchmark Recommendation		Set Correctly	
		Yes	No
1	**Control Plane Components**		
2	**Control Plane Configuration**		
2.1	**Logging**		
2.1.1	Enable EKS control plane logging (Automated)	☐	☐
3	**Worker Nodes**		
3.1	**Worker Node Configuration Files**		
3.1.1	Ensure that the kubeconfig file permissions are set to 644 or more restrictive (Automated)	☐	☐
3.1.2	Ensure that the kubeconfig file ownership is set to root:root (Automated)	☐	☐
3.1.3	Ensure that the kubelet configuration file has permissions set to 644 or more restrictive (Automated)	☐	☐
3.1.4	Ensure that the kubelet configuration file ownership is set to root:root (Automated)	☐	☐
3.2	**Kubelet**		
3.2.1	Ensure kubelet anonymous authentication is disabled (Automated)	☐	☐
3.2.2	Ensure kubelet authorization mode is set to Webhook (Automated)	☐	☐
3.2.3	Ensure the kubelet X.509 client CA file is configured (Automated)	☐	☐
3.2.4	Ensure the kubelet read-only port is disabled (Automated)	☐	☐

CIS Benchmark Recommendation		Set Correctly	
		Yes	No
3.2.5	Ensure the kubelet iptables utility chains are enabled (Automated)	☐	☐
3.2.6	Ensure the kubelet event recording QPS is set to an appropriate value (Automated)	☐	☐
3.2.7	Ensure the kubelet client certificate rotation is enabled (Automated)	☐	☐
3.2.8	Ensure serverTLSBootstrap and RotateKubeletServerCertificate are enabled (Automated)	☐	☐
4	**Policies**		
4.1	**RBAC and Service Accounts**		
4.1.1	Ensure that the cluster-admin role is only used where required (Manual)	☐	☐
4.1.2	Ensure access to Secrets is minimized and granted only where required (Manual)	☐	☐
4.1.3	Minimize wildcard use in Roles and ClusterRoles (Manual)	☐	☐
4.1.4	Minimize access to create pods (Manual)	☐	☐
4.1.5	Ensure that default service accounts are not actively used. (Automated)	☐	☐
4.1.6	Ensure that Service Account Tokens are only mounted where necessary (Automated)	☐	☐
4.1.7	Standardize EKS cluster access by using access entries, policies, and scoped permissions (Automated)	☐	☐
4.1.8	Limit use of the Bind, Impersonate and Escalate permissions in the Kubernetes cluster (Manual)	☐	☐
4.1.9	Minimize access to create persistent volumes (Manual)	☐	☐

CIS Benchmark Recommendation		Set Correctly	
		Yes	No
4.1.10	Minimize access to the proxy sub-resource of nodes (Manual)	☐	☐
4.1.11	Minimize access to webhook configuration objects (Manual)	☐	☐
4.1.12	Minimize access to the service account token creation (Manual)	☐	☐
4.2	**Pod Security Standards**		
4.2.1	Minimize the admission of privileged containers (Automated)	☐	☐
4.2.2	Minimize the admission of containers wishing to share the host process ID namespace (Automated)	☐	☐
4.2.3	Minimize the admission of containers wishing to share the host IPC namespace (Automated)	☐	☐
4.2.4	Minimize the admission of containers wishing to share the host network namespace (Automated)	☐	☐
4.2.5	Minimize the admission of containers with allowPrivilegeEscalation (Automated)	☐	☐
4.3	**Amazon VPC CNI plugin for Kubernetes**		
4.3.1	Ensure VPC CNI network policy support is enabled (Automated)	☐	☐
4.3.2	Ensure required workloads are covered by Kubernetes network policies (Manual)	☐	☐
4.4	**Secrets Management**		
4.4.1	Prefer exposing secrets as mounted files rather than environment variables (Automated)	☐	☐
4.4.2	Consider external secret storage (Manual)	☐	☐
4.5	**General Policies**		

CIS Benchmark Recommendation		Set Correctly	
		Yes	No
4.5.1	Create administrative boundaries between resources using namespaces (Manual)	☐	☐
4.5.2	The default namespace should not be used (Automated)	☐	☐
5	**Managed services**		
5.1	**Image Registry and Image Scanning**		
5.1.1	Ensure Image Vulnerability Scanning using Amazon ECR image scanning or a third party provider (Automated)	☐	☐
5.1.2	Ensure least-privilege access is enforced for Amazon ECR repositories used by EKS workloads (Manual)	☐	☐
5.1.3	Restrict ECR access for EKS runtime identities to pull-only permissions (Manual)	☐	☐
5.1.4	Allow only approved container registries for images used by EKS workloads (Manual)	☐	☐
5.2	**Identity and Access Management (IAM)**		
5.2.1	Prefer using dedicated EKS Service Accounts (Automated)	☐	☐
5.3	**AWS EKS Key Management Service**		
5.3.1	Use customer managed KMS keys for EKS envelope encryption of Kubernetes secrets (Manual)	☐	☐
5.4	**Cluster Networking**		
5.4.1	Restrict EKS control plane endpoint access with private access and public CIDR allowlists (Automated)	☐	☐
5.4.2	Enable private-only EKS control plane access and disable the public endpoint (Automated)	☐	☐
5.4.3	Ensure clusters are created with Private Nodes (Manual)	☐	☐

CIS Benchmark Recommendation		Set Correctly	
		Yes	No
5.4.4	Use TLS certificates on EKS load balancer listeners for encrypted client connections (Manual)	☐	☐
5.5	**Authentication and Authorization**		
5.5.1	Use EKS access entries to manage IAM principal access to Kubernetes (Automated)	☐	☐

Appendix: CIS Controls v7 IG 1 Mapped Recommendations

Recommendation		Set Correctly	
		Yes	No
2.1.1	Enable EKS control plane logging	☐	☐
3.2.1	Ensure kubelet anonymous authentication is disabled	☐	☐
3.2.2	Ensure kubelet authorization mode is set to Webhook	☐	☐
3.2.6	Ensure the kubelet event recording QPS is set to an appropriate value	☐	☐
4.1.1	Ensure that the cluster-admin role is only used where required	☐	☐
4.1.5	Ensure that default service accounts are not actively used.	☐	☐
4.2.1	Minimize the admission of privileged containers	☐	☐
4.2.2	Minimize the admission of containers wishing to share the host process ID namespace	☐	☐
4.2.3	Minimize the admission of containers wishing to share the host IPC namespace	☐	☐
4.2.4	Minimize the admission of containers wishing to share the host network namespace	☐	☐
4.2.5	Minimize the admission of containers with allowPrivilegeEscalation	☐	☐
4.5.1	Create administrative boundaries between resources using namespaces	☐	☐
4.5.2	The default namespace should not be used	☐	☐
5.1.2	Ensure least-privilege access is enforced for Amazon ECR repositories used by EKS workloads	☐	☐
5.1.3	Restrict ECR access for EKS runtime identities to pull-only permissions	☐	☐
5.2.1	Prefer using dedicated EKS Service Accounts	☐	☐

Appendix: CIS Controls v7 IG 2 Mapped Recommendations

Recommendation		Set Correctly	
		Yes	No
2.1.1	Enable EKS control plane logging	☐	☐
3.1.1	Ensure that the kubeconfig file permissions are set to 644 or more restrictive	☐	☐
3.1.2	Ensure that the kubeconfig file ownership is set to root:root	☐	☐
3.1.3	Ensure that the kubelet configuration file has permissions set to 644 or more restrictive	☐	☐
3.1.4	Ensure that the kubelet configuration file ownership is set to root:root	☐	☐
3.2.1	Ensure kubelet anonymous authentication is disabled	☐	☐
3.2.2	Ensure kubelet authorization mode is set to Webhook	☐	☐
3.2.3	Ensure the kubelet X.509 client CA file is configured	☐	☐
3.2.4	Ensure the kubelet read-only port is disabled	☐	☐
3.2.5	Ensure the kubelet iptables utility chains are enabled	☐	☐
3.2.6	Ensure the kubelet event recording QPS is set to an appropriate value	☐	☐
3.2.7	Ensure the kubelet client certificate rotation is enabled	☐	☐
3.2.8	Ensure serverTLSBootstrap and RotateKubeletServerCertificate are enabled	☐	☐
4.1.1	Ensure that the cluster-admin role is only used where required	☐	☐
4.1.2	Ensure access to Secrets is minimized and granted only where required	☐	☐
4.1.3	Minimize wildcard use in Roles and ClusterRoles	☐	☐
4.1.4	Minimize access to create pods	☐	☐
4.1.5	Ensure that default service accounts are not actively used.	☐	☐
4.2.1	Minimize the admission of privileged containers	☐	☐
4.2.2	Minimize the admission of containers wishing to share the host process ID namespace	☐	☐

Recommendation		Set Correctly	
		Yes	No
4.2.3	Minimize the admission of containers wishing to share the host IPC namespace	☐	☐
4.2.4	Minimize the admission of containers wishing to share the host network namespace	☐	☐
4.2.5	Minimize the admission of containers with allowPrivilegeEscalation	☐	☐
4.3.1	Ensure VPC CNI network policy support is enabled	☐	☐
4.3.2	Ensure required workloads are covered by Kubernetes network policies	☐	☐
4.4.1	Prefer exposing secrets as mounted files rather than environment variables	☐	☐
4.5.1	Create administrative boundaries between resources using namespaces	☐	☐
4.5.2	The default namespace should not be used	☐	☐
5.1.1	Ensure Image Vulnerability Scanning using Amazon ECR image scanning or a third party provider	☐	☐
5.1.2	Ensure least-privilege access is enforced for Amazon ECR repositories used by EKS workloads	☐	☐
5.1.3	Restrict ECR access for EKS runtime identities to pull-only permissions	☐	☐
5.1.4	Allow only approved container registries for images used by EKS workloads	☐	☐
5.2.1	Prefer using dedicated EKS Service Accounts	☐	☐
5.4.1	Restrict EKS control plane endpoint access with private access and public CIDR allowlists	☐	☐
5.4.4	Use TLS certificates on EKS load balancer listeners for encrypted client connections	☐	☐
5.5.1	Use EKS access entries to manage IAM principal access to Kubernetes	☐	☐

Appendix: CIS Controls v7 IG 3 Mapped Recommendations

Recommendation		Set Correctly	
		Yes	No
2.1.1	Enable EKS control plane logging	☐	☐
3.1.1	Ensure that the kubeconfig file permissions are set to 644 or more restrictive	☐	☐
3.1.2	Ensure that the kubeconfig file ownership is set to root:root	☐	☐
3.1.3	Ensure that the kubelet configuration file has permissions set to 644 or more restrictive	☐	☐
3.1.4	Ensure that the kubelet configuration file ownership is set to root:root	☐	☐
3.2.1	Ensure kubelet anonymous authentication is disabled	☐	☐
3.2.2	Ensure kubelet authorization mode is set to Webhook	☐	☐
3.2.3	Ensure the kubelet X.509 client CA file is configured	☐	☐
3.2.4	Ensure the kubelet read-only port is disabled	☐	☐
3.2.5	Ensure the kubelet iptables utility chains are enabled	☐	☐
3.2.6	Ensure the kubelet event recording QPS is set to an appropriate value	☐	☐
3.2.7	Ensure the kubelet client certificate rotation is enabled	☐	☐
3.2.8	Ensure serverTLSBootstrap and RotateKubeletServerCertificate are enabled	☐	☐
4.1.1	Ensure that the cluster-admin role is only used where required	☐	☐
4.1.2	Ensure access to Secrets is minimized and granted only where required	☐	☐
4.1.3	Minimize wildcard use in Roles and ClusterRoles	☐	☐
4.1.4	Minimize access to create pods	☐	☐
4.1.5	Ensure that default service accounts are not actively used.	☐	☐
4.1.6	Ensure that Service Account Tokens are only mounted where necessary	☐	☐

Recommendation		Set Correctly	
		Yes	No
4.1.7	Standardize EKS cluster access by using access entries, policies, and scoped permissions	☐	☐
4.1.8	Limit use of the Bind, Impersonate and Escalate permissions in the Kubernetes cluster	☐	☐
4.2.1	Minimize the admission of privileged containers	☐	☐
4.2.2	Minimize the admission of containers wishing to share the host process ID namespace	☐	☐
4.2.3	Minimize the admission of containers wishing to share the host IPC namespace	☐	☐
4.2.4	Minimize the admission of containers wishing to share the host network namespace	☐	☐
4.2.5	Minimize the admission of containers with allowPrivilegeEscalation	☐	☐
4.3.1	Ensure VPC CNI network policy support is enabled	☐	☐
4.3.2	Ensure required workloads are covered by Kubernetes network policies	☐	☐
4.4.1	Prefer exposing secrets as mounted files rather than environment variables	☐	☐
4.4.2	Consider external secret storage	☐	☐
4.5.1	Create administrative boundaries between resources using namespaces	☐	☐
4.5.2	The default namespace should not be used	☐	☐
5.1.1	Ensure Image Vulnerability Scanning using Amazon ECR image scanning or a third party provider	☐	☐
5.1.2	Ensure least-privilege access is enforced for Amazon ECR repositories used by EKS workloads	☐	☐
5.1.3	Restrict ECR access for EKS runtime identities to pull-only permissions	☐	☐
5.1.4	Allow only approved container registries for images used by EKS workloads	☐	☐
5.2.1	Prefer using dedicated EKS Service Accounts	☐	☐
5.3.1	Use customer managed KMS keys for EKS envelope encryption of Kubernetes secrets	☐	☐
5.4.1	Restrict EKS control plane endpoint access with private access and public CIDR allowlists	☐	☐

Recommendation		Set Correctly	
		Yes	No
5.4.4	Use TLS certificates on EKS load balancer listeners for encrypted client connections	☐	☐
5.5.1	Use EKS access entries to manage IAM principal access to Kubernetes	☐	☐

Appendix: CIS Controls v7 Unmapped Recommendations

Recommendation		Set Correctly	
		Yes	No
4.1.9	Minimize access to create persistent volumes	☐	☐
4.1.10	Minimize access to the proxy sub-resource of nodes	☐	☐
4.1.11	Minimize access to webhook configuration objects	☐	☐
4.1.12	Minimize access to the service account token creation	☐	☐

Appendix: CIS Controls v8 IG 1 Mapped Recommendations

Recommendation		Set Correctly	
		Yes	No
2.1.1	Enable EKS control plane logging	☐	☐
3.1.1	Ensure that the kubeconfig file permissions are set to 644 or more restrictive	☐	☐
3.1.2	Ensure that the kubeconfig file ownership is set to root:root	☐	☐
3.1.3	Ensure that the kubelet configuration file has permissions set to 644 or more restrictive	☐	☐
3.1.4	Ensure that the kubelet configuration file ownership is set to root:root	☐	☐
3.2.1	Ensure kubelet anonymous authentication is disabled	☐	☐
3.2.2	Ensure kubelet authorization mode is set to Webhook	☐	☐
3.2.6	Ensure the kubelet event recording QPS is set to an appropriate value	☐	☐
4.1.1	Ensure that the cluster-admin role is only used where required	☐	☐
4.1.2	Ensure access to Secrets is minimized and granted only where required	☐	☐
4.1.3	Minimize wildcard use in Roles and ClusterRoles	☐	☐
4.1.5	Ensure that default service accounts are not actively used.	☐	☐
4.2.1	Minimize the admission of privileged containers	☐	☐
4.2.2	Minimize the admission of containers wishing to share the host process ID namespace	☐	☐
4.2.3	Minimize the admission of containers wishing to share the host IPC namespace	☐	☐
4.2.4	Minimize the admission of containers wishing to share the host network namespace	☐	☐
4.2.5	Minimize the admission of containers with allowPrivilegeEscalation	☐	☐
4.3.1	Ensure VPC CNI network policy support is enabled	☐	☐

Recommendation		Set Correctly	
		Yes	No
4.3.2	Ensure required workloads are covered by Kubernetes network policies	☐	☐
5.1.2	Ensure least-privilege access is enforced for Amazon ECR repositories used by EKS workloads	☐	☐
5.1.3	Restrict ECR access for EKS runtime identities to pull-only permissions	☐	☐
5.4.1	Restrict EKS control plane endpoint access with private access and public CIDR allowlists	☐	☐
5.4.2	Enable private-only EKS control plane access and disable the public endpoint	☐	☐
5.4.3	Ensure clusters are created with Private Nodes	☐	☐

Appendix: CIS Controls v8 IG 2 Mapped Recommendations

Recommendation		Set Correctly	
		Yes	No
2.1.1	Enable EKS control plane logging	☐	☐
3.1.1	Ensure that the kubeconfig file permissions are set to 644 or more restrictive	☐	☐
3.1.2	Ensure that the kubeconfig file ownership is set to root:root	☐	☐
3.1.3	Ensure that the kubelet configuration file has permissions set to 644 or more restrictive	☐	☐
3.1.4	Ensure that the kubelet configuration file ownership is set to root:root	☐	☐
3.2.1	Ensure kubelet anonymous authentication is disabled	☐	☐
3.2.2	Ensure kubelet authorization mode is set to Webhook	☐	☐
3.2.3	Ensure the kubelet X.509 client CA file is configured	☐	☐
3.2.4	Ensure the kubelet read-only port is disabled	☐	☐
3.2.5	Ensure the kubelet iptables utility chains are enabled	☐	☐
3.2.6	Ensure the kubelet event recording QPS is set to an appropriate value	☐	☐
3.2.7	Ensure the kubelet client certificate rotation is enabled	☐	☐
3.2.8	Ensure serverTLSBootstrap and RotateKubeletServerCertificate are enabled	☐	☐
4.1.1	Ensure that the cluster-admin role is only used where required	☐	☐
4.1.2	Ensure access to Secrets is minimized and granted only where required	☐	☐
4.1.3	Minimize wildcard use in Roles and ClusterRoles	☐	☐
4.1.5	Ensure that default service accounts are not actively used.	☐	☐
4.1.6	Ensure that Service Account Tokens are only mounted where necessary	☐	☐
4.1.7	Standardize EKS cluster access by using access entries, policies, and scoped permissions	☐	☐

Recommendation		Set Correctly	
		Yes	No
4.1.8	Limit use of the Bind, Impersonate and Escalate permissions in the Kubernetes cluster	☐	☐
4.2.1	Minimize the admission of privileged containers	☐	☐
4.2.2	Minimize the admission of containers wishing to share the host process ID namespace	☐	☐
4.2.3	Minimize the admission of containers wishing to share the host IPC namespace	☐	☐
4.2.4	Minimize the admission of containers wishing to share the host network namespace	☐	☐
4.2.5	Minimize the admission of containers with allowPrivilegeEscalation	☐	☐
4.3.1	Ensure VPC CNI network policy support is enabled	☐	☐
4.3.2	Ensure required workloads are covered by Kubernetes network policies	☐	☐
4.4.1	Prefer exposing secrets as mounted files rather than environment variables	☐	☐
4.4.2	Consider external secret storage	☐	☐
4.5.2	The default namespace should not be used	☐	☐
5.1.1	Ensure Image Vulnerability Scanning using Amazon ECR image scanning or a third party provider	☐	☐
5.1.2	Ensure least-privilege access is enforced for Amazon ECR repositories used by EKS workloads	☐	☐
5.1.3	Restrict ECR access for EKS runtime identities to pull-only permissions	☐	☐
5.1.4	Allow only approved container registries for images used by EKS workloads	☐	☐
5.3.1	Use customer managed KMS keys for EKS envelope encryption of Kubernetes secrets	☐	☐
5.4.1	Restrict EKS control plane endpoint access with private access and public CIDR allowlists	☐	☐
5.4.2	Enable private-only EKS control plane access and disable the public endpoint	☐	☐
5.4.3	Ensure clusters are created with Private Nodes	☐	☐
5.4.4	Use TLS certificates on EKS load balancer listeners for encrypted client connections	☐	☐

Appendix: CIS Controls v8 IG 3 Mapped Recommendations

Recommendation		Set Correctly	
		Yes	No
2.1.1	Enable EKS control plane logging	☐	☐
3.1.1	Ensure that the kubeconfig file permissions are set to 644 or more restrictive	☐	☐
3.1.2	Ensure that the kubeconfig file ownership is set to root:root	☐	☐
3.1.3	Ensure that the kubelet configuration file has permissions set to 644 or more restrictive	☐	☐
3.1.4	Ensure that the kubelet configuration file ownership is set to root:root	☐	☐
3.2.1	Ensure kubelet anonymous authentication is disabled	☐	☐
3.2.2	Ensure kubelet authorization mode is set to Webhook	☐	☐
3.2.3	Ensure the kubelet X.509 client CA file is configured	☐	☐
3.2.4	Ensure the kubelet read-only port is disabled	☐	☐
3.2.5	Ensure the kubelet iptables utility chains are enabled	☐	☐
3.2.6	Ensure the kubelet event recording QPS is set to an appropriate value	☐	☐
3.2.7	Ensure the kubelet client certificate rotation is enabled	☐	☐
3.2.8	Ensure serverTLSBootstrap and RotateKubeletServerCertificate are enabled	☐	☐
4.1.1	Ensure that the cluster-admin role is only used where required	☐	☐
4.1.2	Ensure access to Secrets is minimized and granted only where required	☐	☐
4.1.3	Minimize wildcard use in Roles and ClusterRoles	☐	☐
4.1.4	Minimize access to create pods	☐	☐
4.1.5	Ensure that default service accounts are not actively used.	☐	☐
4.1.6	Ensure that Service Account Tokens are only mounted where necessary	☐	☐

Recommendation		Set Correctly	
		Yes	No
4.1.7	Standardize EKS cluster access by using access entries, policies, and scoped permissions	☐	☐
4.1.8	Limit use of the Bind, Impersonate and Escalate permissions in the Kubernetes cluster	☐	☐
4.1.9	Minimize access to create persistent volumes	☐	☐
4.1.10	Minimize access to the proxy sub-resource of nodes	☐	☐
4.1.11	Minimize access to webhook configuration objects	☐	☐
4.1.12	Minimize access to the service account token creation	☐	☐
4.2.1	Minimize the admission of privileged containers	☐	☐
4.2.2	Minimize the admission of containers wishing to share the host process ID namespace	☐	☐
4.2.3	Minimize the admission of containers wishing to share the host IPC namespace	☐	☐
4.2.4	Minimize the admission of containers wishing to share the host network namespace	☐	☐
4.2.5	Minimize the admission of containers with allowPrivilegeEscalation	☐	☐
4.3.1	Ensure VPC CNI network policy support is enabled	☐	☐
4.3.2	Ensure required workloads are covered by Kubernetes network policies	☐	☐
4.4.1	Prefer exposing secrets as mounted files rather than environment variables	☐	☐
4.4.2	Consider external secret storage	☐	☐
4.5.1	Create administrative boundaries between resources using namespaces	☐	☐
4.5.2	The default namespace should not be used	☐	☐
5.1.1	Ensure Image Vulnerability Scanning using Amazon ECR image scanning or a third party provider	☐	☐
5.1.2	Ensure least-privilege access is enforced for Amazon ECR repositories used by EKS workloads	☐	☐
5.1.3	Restrict ECR access for EKS runtime identities to pull-only permissions	☐	☐
5.1.4	Allow only approved container registries for images used by EKS workloads	☐	☐
5.2.1	Prefer using dedicated EKS Service Accounts	☐	☐

Recommendation		Set Correctly	
		Yes	No
5.3.1	Use customer managed KMS keys for EKS envelope encryption of Kubernetes secrets	☐	☐
5.4.1	Restrict EKS control plane endpoint access with private access and public CIDR allowlists	☐	☐
5.4.2	Enable private-only EKS control plane access and disable the public endpoint	☐	☐
5.4.3	Ensure clusters are created with Private Nodes	☐	☐
5.4.4	Use TLS certificates on EKS load balancer listeners for encrypted client connections	☐	☐
5.5.1	Use EKS access entries to manage IAM principal access to Kubernetes	☐	☐

Appendix: CIS Controls v8 Unmapped Recommendations

Recommendation		Set Correctly	
		Yes	No
	No unmapped recommendations to CIS Controls v8	☐	☐

Appendix: Change History

Date	Version	Changes for this version
5/25/2026	2.0.0	Recommendation 5.5.1 updated audit and remediation commands
5/25/2026	2.0.0	Recommendation 5.4.3 updated audit and remediation commands
5/25/2026	2.0.0	Recommendation 5.4.4 has been removed – replaced by VPC CNI plugin.
5/25/2026	2.0.0	Recommendation 4.3.1 updated audit and remediation commands
5/25/2026	2.0.0	Recommendation 4.1.6 updated audit and remediation commands
5/25/2026	2.0.0	Recommendation 4.1.5 updated audit and remediation commands
5/25/2026	2.0.0	Recommendation 3.2.5 has been removed as the setting has been fully deprecated and no longer effective.
5/25/2026	2.0.0	Recommendation 3.2.4 prose has been updated
5/25/2026	2.0.0	Recommendation 3.2.3 prose has been updated
5/25/2026	2.0.0	Recommendation 3.2.2 prose has been updated
5/25/2026	2.0.0	Recommendation 3.2.1 prose has been updated
5/25/2026	2.0.0	Recommendation 3.1.4 prose has been updated
5/25/2026	2.0.0	Recommendation 3.1.3 prose has been updated
5/25/2026	2.0.0	Recommendation 3.1.2 prose has been updated
5/25/2026	2.0.0	Recommendation 3.1.1 prose has been updated

Date	Version	Changes for this version
5/25/2026	2.0.0	Recommendation 2.1.2 was determined to be a duplicate and has been removed.
5/25/2026	2.0.0	Recommendation 2.1.1 prose has been updated
10/14/2025	1.8.0	AAC updated and tested against Latest Cluster available v1.34
10/12/2025	1.8.0	AAC testing on versions 1.32,1.33, 1.34
10/12/2025	1.8.0	Benchmark updated to v1.32, 1.33 & 1.34
9/17/2025	1.8.0	Added recommendation 4.1.9 “Minimize access to create persistent volumes”
9/17/2025	1.8.0	Added recommendation 4.1.10 “ Minimize access to the proxy sub-resource of nodes
9/17/2025	1.8.0	Added recommendation 4.1.11 “Minimize access to webhook configuration objects”
9/17/2025	1.8.0	Added recommendation 4.1.12 “Minimize access to the service account token creation”
9/17/2025	1.8.0	Modified recommendation 4.1.6 to align with Kubernetes Benchmark v1.13
9/16/2025	1.8.0	Modified recommendation 4.1.1 to align with Kubernetes Benchmark v1.13
9/1/2025	1.8.0	Modified recommendation 4.1.2 to align with Kubernetes Benchmark v1.13
9/1/2025	1.8.0	Modified recommendation 4.1.3 to align with Kubernetes Benchmark v1.13
9/1/2025	1.8.0	Modified recommendation 4.1.4 to align with Kubernetes Benchmark v1.13
9/1/2025	1.8.0	Modified recommendation 4.1.5 to align with Kubernetes Benchmark v1.13
5/1/2025	1.7.0	AAC updated and tested against Latest Cluster available v1.32
5/1/2025	1.7.0	AAC testing on versions 1.30,1.31, 1.32
4/14/2025	1.7.0	3.1.3 AAC test script edited

Date	Version	Changes for this version
4/14/2025	1.7.0	3.1.1 AAC test script edited.